



תוכן העניינים

תוכן העניינים.....	2
2. תקציר:.....	4
2.1. הרקע לפרויקט:.....	4
2.2. תהליך המחקר:.....	5
2.3. סקירת ספרות:.....	5
2.4. אתגרים מרכזיים:.....	6
2.4.1. הבעיה האלגוריתמית:.....	6
2.4.2. הסיבות לבחירת הנושא:.....	6
2.4.3. מוטיבציה לעבודה:.....	7
2.4.4. על איזה צורך הפרויקט עונה:.....	7
2.4.5. הצגת פתרונות לבעיה:.....	7
3. מטרות/ יעדים:.....	8
4. אתגרים:.....	8
5. מדדי הצלחה למערכת:.....	9
6. רקע תיאורטי:.....	9
6.1. מושגים עיקריים בתחום הדעת של הבעיה:.....	10
6.2. נושאים רלוונטיים בתחום הדעת של הבעיה:.....	10
6.3. סקירה מקצועית של הבעיה בעולם:.....	11
6.4. נושאים רלוונטיים במדעי המחשב בהקשר לבעיה:.....	12
7. תיאור פתרונות קיימים לבעיה:.....	12
8. ניתוח חלופות מערכת:.....	13
9. תיאור החלופה הנבחרת ונימוקים:.....	14
10. אפיון המערכת:.....	15
10.1. ניתוח דרישות המערכת:.....	16
10.2. מודול המערכת:.....	17
10.3. אפיון פונקציונלי:.....	18
10.4. ביצועים עיקריים:.....	19
10.5. אילוצים:.....	19
11. תיאור הארכיטקטורה:.....	20
11.1. הארכיטקטורה של הפתרון המוצע:.....	20
11.2. תיאור הרכיבים בפתרון:.....	20
11.3. תיאור תהליכים של מערכת ההפעלה:.....	21
11.4. ארכיטקטורת רשת:.....	21

בארכיטקטורה API 11.5 תיאור	22
11.6 תיאור פרוטוקולי התקשורת	22
11.7 שרת- לקוח	22
12. תיאור תהליכי אבטחת מידע במערכת	23
12.1 תיאור התקיפה או ההגנה	23
12.2 תיאור הצפנות	23
13. למידת מכונה	24
13.1 הליך איסוף הנתונים	24
13.2 טיוב ואיזון נתונים	25
13.3 סוג הלמידה הנבחר	25
13.4 נוסחאות ורשתות למידה	26
13.5 כיוון, ייעול וניטור מדדי ביצוע	27
14. תיאור התוכנה	28
14.1 API פירוט	28
14.2 סביבת עבודה	30
14.3 שפות תכנות	31
15. Use cases/UML של המערכת המוצעת. ניתוח ותרשים	32
15.1 העיקריים של המערכת UC תיאור ה-	32
15.2 case use (עבור כל הפונקציות העיקריות בפרויקט. הצגת מקרה)	33
15.3 מבנה נתונים בהם השתמש	35
15.4 חישוב יעילות האלגוריתם	36
15.5 הקשרים בין היחידות השונות	38
15.6 עץ מודולים	39
15.7 Use cases Diagram	40
15.8 Use cases רשימת	40
15.9 UML תרשים	42
15.10 Design class diagram	42
15.11 תרשים מחלקות.	44
15.12 תיאור המחלקות המוצעות	44
16. תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם	48
17. עבור כל מסך באפליקציה	48
17.1 מה תפקידו של המסך	48
17.2 תיאור המסך	48
17.3 צילום מסך	48
18. הסבר על תפקידי אלמנטים בתצוגת הפרויקט	53
19. הודעות למשתמש	54

20. ממשק משתמש	55
21. קוד התוכנית	56
21.1 קלט	56
21.2 פלט	56
21.3 פונקציות עיקריות	56
22. תיאור מסד הנתונים	71
23. מדריך למשתמש	80
24. בדיקות והערכה	81
25. מסקנות	81
26. פיתוחים עתידיים	82
27. ביבליוגרפיה	82

2. תקציר:

הפרויקט "MindMatch AI" עוסק בפיתוח מערכת חכמה לביצוע ראיונות עבודה אוטומטיים באופן מקוון. המערכת מאפשרת לחברות ליצור משרה פתוחה, להזין תיאור חופשי של התפקיד והדרישות, ולקבל שאלות ראיון מותאמות למשרה. לאחר מכן מועמדים יכולים לצפות במשרות פתוחות, להצטרף לראיון, ולענות על שאלות באופן טקסטואלי או קולי. המערכת תומכת בקלט טקסטואלי וקולי, ממירה תשובות קוליות לטקסט, מנתחת את תוכן התשובות, בודקת את מידת הרלוונטיות שלהן לשאלה, ומפעילה מודלים שונים לניתוח מאפיינים כגון ניסיון, יכולת חשיבה ויכולות מעשיות. בסיום התהליך המערכת מפיקה ציון התאמה סופי למועמד באמצעות גרף ראיות משוקלל, המתחשב בחשיבות מאפייני המשרה, באיכות תשובות המועמד, באמינות התשובות ובדפוסים שחוזרים לאורך הראיון. בנוסף מוצג לחברה סיכום מילולי הכולל חוזקות, חולשות יחסיות והסבר על התאמת המועמד לתפקיד. הפרויקט נועד לייעל את תהליך הגיוס, לצמצם תלות בהערכה ידנית בלבד, ולאפשר תהליך מובנה, עקבי ומבוסס נתונים יותר.

2.1. הרקע לפרויקט:

בעשור האחרון חלה התקדמות משמעותית בתחום הבינה המלאכותית והמערכות החכמות, אשר מאפשרות לבצע תהליכים מורכבים שבעבר דרשו התערבות אנושית. אחד התחומים המרכזיים שהושפעו מכך הוא תחום משאבי האנוש, ובפרט תהליכי גיוס עובדים. תהליך גיוס עובדים כולל שלבים רבים, כאשר אחד המרכזיים שבהם הוא ראיון העבודה. שלב זה דורש השקעת זמן ומשאבים רבים מצד הארגון, כולל תיאום מועדים, נוכחות מראיינים וביצוע הערכה סובייקטיבית של מועמדים. בנוסף, תהליכי ראיון אנושיים מושפעים לעיתים מהטיות שונות, כגון רושם ראשוני, מצב רוח או העדפות אישיות, דבר שעלול לפגוע באובייקטיביות ההחלטה. לאור זאת, עולה צורך במערכת חכמה שתאפשר ביצוע ראיונות באופן אוטומטי, תוך ניתוח נתונים אובייקטיבי והפחתת התלות בגורם האנושי. מערכת כזו תאפשר לחברות לבצע ראיונות מכל מקום ובכל זמן, לייעל את תהליך המיון ולשפר את איכות קבלת ההחלטות.

2.2 תהליך המחקר:

במסגרת המחקר המקדים לפרוייקט, בוצעה למידה מעמיקה של תחום גיוס העובדים ותהליכי ראיונות עבודה. המחקר כלל הבנה של אופן ביצוע ראיונות בארגונים שונים, סוגי שאלות מקובלות, קריטריונים להערכת מועמדים, והקשיים המרכזיים בתהליך זה.

נמצא כי מראיינים עושים שימוש במספר סוגי שאלות עיקריים:

- שאלות מקצועיות הבודקות ידע וניסיון.
- שאלות התנהגותיות, כגון: "ספר על מקרה בו התמודדת עם קונפליקט בצוות".
- שאלות הבוחנות איכות חשיבה ופתרון בעיות.
- שאלות פתוחות הבודקות יכולת ביטוי, סדר מחשבתי ורמת ביטחון.

במהלך המחקר עלה כי הערכת מועמדים מושפעת במידה רבה מהתרשמות אישית של המראיין, ולעיתים קשה לבצע הערכה עקבית ואובייקטיבית בין מועמדים שונים. בנוסף, נמצא כי תשובות רבות אינן נבחנות רק לפי התוכן העובדתי שלהן, אלא גם לפי אופן הניסוח, רמת הפירוט, מידת האחריות האישית שהמועמד מתאר, והיכולת להסביר תהליכים בצורה ברורה. בנוסף, נבחנו מאפייני אישיות מרכזיים הנבדקים במהלך ראיונות, כגון: אחריות, יוזמה, תקשורת בין-אישית, יכולת ביטוי, רמת מוטיבציה והתאמה לסביבת עבודה. במהלך המחקר זוהו שישה צירים מרכזיים להערכת מועמדים:

- תכונות אופי ודפוסי התנהגות.
- איכות חשיבה.
- יכולות ביצוע בפועל.
- רמת ניסיון.
- מוטיבציה.
- התאמה לסביבה ארגונית.

נמצא כי שילוב בין צירים אלו מאפשר יצירת תמונה רחבה ומדויקת יותר של המועמד, מעבר לבדיקה טכנית של ידע בלבד. כמו כן, נלמדו עקרונות בתחום עיבוד שפה טבעית (NLP), במטרה להבין כיצד ניתן לנתח תשובות של מועמדים בצורה אוטומטית. במסגרת זו נבחנו שיטות לניתוח משמעות טקסט, זיהוי קשר בין שאלה לתשובה, וסיווג מאפיינים מתוך שפה חופשית. המחקר אפשר הבנה רחבה של תחום הבעיה, והיווה בסיס לפיתוח הפתרון המוצע בפרוייקט.

2.3 סקירת ספרות:

תחום ניתוח השפה הטבעית (Natural Language Processing) מהווה אחד התחומים המרכזיים במחקר הבינה המלאכותית, ומתמקד ביכולת של מערכות מחשב להבין, לנתח ולהפיק משמעות מטקסטים בשפה אנושית. מחקרים בתחום מצביעים על מעבר משיטות סטטיסטיות פשוטות למודלים מתקדמים מבוססי למידת מכונה ורשתות נוירונים עמוקות. בפרט, מחקר ה-Transformer של Vaswani ואחרים מגולל (2017), אשר הוצג במאמר *Attention Is All You Need*, הוביל למהפכה בתחום ואפשר הבנה הקשרית של טקסט תוך התחשבות ביחסים בין מילים בתוך המשפט ומחוצה לו. בהמשך לכך, פותחו מודלים מתקדמים כגון BERT של Google Research ומודל RoBERTa של Facebook AI, אשר שיפרו משמעותית את היכולת לבצע הבנת שפה טבעית במשימות שונות, כולל סיווג טקסט, מענה לשאלות והסקת משמעות. לצד זאת, תחום זיהוי הדיבור (Speech Recognition) עבר התפתחות משמעותית. מחקרים ומודלים כגון Whisper של OpenAI הצליחו להגיע לרמות דיוק גבוהות גם בתנאים מורכבים. הכוללים רעשי רקע, מבטאים שונים ושונות בין דוברים.

עם זאת, מערכות אלו מתמקדות בעיקר בהמרת דיבור לטקסט, ואינן מבצעות הבנה מלאה של משמעות התוכן. בתחום הפסיכולוגיה החישובית, פותחו מודלים לניתוח אישיות מתוך טקסט, ביניהם מודל Big Five, אשר מתאר חמישה ממדים מרכזיים של אישיות: פתיחות לחוויות, מצפוניות, מוחצנות, נעימות ורגישות רגשית. מחקרים בתחום מראים כי ניתן להסיק מאפיינים אישיותיים מתוך דפוסי שפה, סגנון כתיבה ואופן ניסוח. בנוסף, מחקר בשם *Semantic Answer Similarity for Evaluating Question Answering Models*, אשר פורסם במסגרת 2021 ACL, הציג גישה למדידת דמיון סמנטי בין תשובות באמצעות מודלי Transformer. המחקר הראה כי הערכת משמעות של תשובות יעילה ומדויקת יותר מהשוואת מילות מפתח בלבד. למרות ההתקדמות בתחום, מחקרים מצביעים על כך שמערכות רבות עדיין מתקשות לבצע הבנה סמנטית עמוקה של טקסט, ובפרט מתקשות להסביר את תהליך קבלת ההחלטות שלהן. בנוסף, מרבית הפתרונות מתמקדים במשימות סיווג או חיזוי, ואינם מספקים ניתוח מפורט של הסיבות לתוצאה. הפרויקט הנוכחי מתמודד עם פערים אלו באמצעות שילוב בין הבנת טקסט ברמה סמנטית, הסקת סיבות לאי התאמה בין שאלה לתשובה, ואבחון מאפיינים אישיותיים מתוך טקסט חופשי.

2.4 אתגרים מרכזיים:

הפרויקט מציב מספר אתגרים אלגוריתמיים מרכזיים, בעיקר בתחום ניתוח טקסט חופשי, הבנת שפה טבעית והתאמת מידע לא מובנה לדרישות משרה מוגדרות. המערכת נדרשת להבין תיאור משרה חופשי, לזהות מתוכו מאפיינים מקצועיים חשובים, ולהתאים אליו שאלות ראיון רלוונטיות מתוך מאגר שאלות. אתגר מרכזי נוסף הוא ניתוח תשובות מועמדים ברמה סמנטית. המערכת צריכה לזהות האם תשובה עונה על השאלה מבחינה רעיונית, גם ללא התאמה מילולית ישירה, ולהבחין בין תשובה רלוונטית, תשובה חלקית, תשובה כללית מדי או תשובה שאינה עונה על השאלה ועוד... בנוסף, קיימת בעיה אלגוריתמית של שילוב מספר תוצאות אבחון לציון התאמה סופי. המערכת אינה מסתמכת על תשובה אחת או על מודל יחיד, אלא משלבת ראיות ממספר תשובות ומודלים שונים, תוך התחשבות בחשיבות מאפייני המשרה, באמינות התשובות ובדפוסים החוזרים לאורך הראיון.

2.4.1 הבעיה האלגוריתמית:

הבעיה האלגוריתמית המרכזית בפרויקט היא קבלת תשובות חופשיות של מועמדים והמרתן למדדים ברורים, כמותיים ומוסברים, המייצגים את מידת התאמת המועמד למשרה. בניגוד לשאלון סגור, שבו התשובות מוגדרות מראש, כאן כל מועמד יכול לענות בניסוח שונה, באורך שונה וברמת פירוט שונה. לכן המערכת אינה יכולה להסתמך רק על מילות מפתח, אלא צריכה להבין את הקשר בין השאלה, תוכן התשובה ודרישות המשרה. המערכת מתמודדת עם כמה בעיות אלגוריתמיות: התאמת שאלות לתיאור משרה חופשי, בדיקת רלוונטיות בין שאלה לתשובה, זיהוי תשובות חלקיות או לא ממוקדות, והפעלת מודלי אבחון שונים על התשובות. לאחר מכן עליה לשלב את כל התוצאות לכדי ציון התאמה סופי, כך שהציון ישקף לא רק תשובה בודדת אלא את כלל הראיות שנאספו לאורך הראיון.

2.4.2 הסיבות לבחירת הנושא:

הנושא נבחר מתוך עניין בתחום הבינה המלאכותית, ובפרט בתחום עיבוד השפה הטבעית והבנת טקסט. תחום זה מציב אתגרים מורכבים, שכן הוא דורש מהמחשב להתמודד עם שפה אנושית, הכוללת הקשרים משמעותיים מרומזות ודקויות לשוניות. בנוסף, נבחר נושא בעל רלוונטיות מעשית גבוהה לעולם האמיתי.

תחום הגיוס והראיונות מהווה חלק מרכזי בהתנהלות ארגונים, והוא כולל תהליכים מורכבים שעדיין נשענים במידה רבה על שיקול דעת אנושי. שילוב טכנולוגיה חכמה בתחום זה מאפשר שיפור משמעותי ביעילות ובאובייקטיביות התהליך.

2.4.3 מוטיבציה לעבודה:

המוטיבציה לפרויקט נובעת מהרצון לפתח מערכת חכמה המסוגלת להבין שפה אנושית ברמה עמוקה, ולא רק לבצע עיבוד טכני של נתונים. הפרויקט מאפשר לשלב בין ידע תיאורטי לבין יישום מעשי, תוך התמודדות עם בעיות אמיתיות בתחום הבנת השפה, זיהוי כוונות והסקת תובנות מתוך טקסט. בנוסף, קיימת מוטיבציה ליצור מערכת בעלת ערך ממשי, היכולה לשפר תהליכים קיימים ולהקל על משתמשים בעולם האמיתי.

2.4.4 על איזה צורך הפרויקט עונה:

הפרויקט נותן מענה לצורך ממשי בתהליך גיוס עובדים, אשר כיום דורש השקעת זמן, משאבים ותיאום רב מצד הארגון. בתהליכי גיוס רבים יש צורך לסנן מספר גדול של מועמדים, לבצע ראיונות ראשוניים, להשוות בין מועמדים ולזהות מי מהם מתאים יותר לדרישות המשרה. תהליך זה עלול להיות ארוך, לא אחיד ולעיתים תלוי מאוד בזמינות ובשיקול הדעת של המראיין. קהל היעד של המערכת כולל מחלקות משאבי אנוש, חברות השמה, מנהלי גיוס וארגונים המבצעים תהליכי גיוס רחבי היקף. עבור משתמשים אלה, המערכת מאפשרת ליצור משרה פתוחה, לקבל שאלות ראיון מותאמות למשרה, לאפשר למועמדים לענות באופן עצמאי, ולקבל בסיום הראיון ציון התאמה וסיכום מוסבר. המערכת עונה גם על צורך של מועמדים, בכך שהיא מאפשרת להם לבצע ראיון מקוון בזמן שנוח להם, לענות בכתב או בקול, ולקבל הזדמנות לשפר תשובה כאשר היא אינה עונה על השאלה בצורה מספקת. הפתרון שהפרויקט מציע הוא תהליך ראיון מובנה, אחיד ומבוסס נתונים, אשר מסייע לחברה להעריך מועמדים בצורה עקבית יותר. המערכת אינה מסתפקת בבדיקת מילים או בציון פשוט, אלא מנתחת את משמעות התשובות, את הקשר שלהן לשאלות, את מאפייני ההתאמה של המועמד ואת הדפוסים החוזרים לאורך הראיון.

2.4.5 הצגת פתרונות לבעיה:

במסגרת המחקר המקדים נבחנו מספר גישות לפתרון בעיית הערכת מועמדים בתהליך גיוס. הגישה הראשונה היא ראיונות עבודה מסורתיים, שבהם מראיין אנושי שואל שאלות ומבצע את ההערכה. גישה זו מאפשרת התרשמות אישית והבנה רחבה של המועמד, אך היא תלויה בזמינות המראיין, בניסיון שלו ובשיקול דעת סובייקטיבי. בנוסף, כאשר יש מספר רב של מועמדים, קשה לשמור על תהליך אחיד ועקבי לכולם. גישה נוספת היא שימוש במערכות אוטומטיות לסינון קורות חיים, המבוססות בעיקר על התאמה בין מילות מפתח לבין דרישות המשרה. מערכות אלו יעילות בשלב הסינון הראשוני, אך הן אינן בודקות כיצד המועמד חושב, מסביר, פותר בעיות או מתמודד עם שאלות פתוחות. נבחנו גם גישה של ניתוח טקסט בסיסי, כגון זיהוי רגשות, איתור מילים מסוימות או חישוב ציונים לפי מאפיינים לשוניים פשוטים. גישה זו יכולה לספק מידע ראשוני על התשובה, אך היא אינה מספיקה כדי לקבוע האם התשובה עונה על השאלה בהקשר הנכון, ואינה מאפשרת להפיק ציון התאמה מלא למשרה. מסקנת המחקר הייתה כי נדרש פתרון המשלב בין ראיון מובנה, ניתוח סמנטי של תשובות, בדיקת רלוונטיות וחישוב ציון התאמה מבוסס ראיות.

לכן בפרויקט נבחרה גישה המשלבת התאמת שאלות למשרה, ניתוח תשובות באמצעות מספר מודלים, ומתן ציון סופי באמצעות גרף ראיות משוקלל.

3. מטרות/ יעדים:

מטרת הפרויקט היא לתכנן ולממש מערכת חכמה לניהול ראיונות עבודה מקוונים, המשלבת צד לקוח, צד שרת, מסד נתונים ומודלי ניתוח מבוססי Python. מטרתי הייתה לבנות מערכת מלאה, שבה חברה יכולה ליצור משרה פתוחה, לקבל שאלות ראיון מותאמות, ומועמד יכול להתחיל ראיון, לענות על שאלות ולקבל ניתוח אוטומטי של תשובותיו. היעדים המרכזיים בפרויקט היו: פיתוח ממשק משתמש ברור ונוח לחברה ולמועמד. מימוש מערכת הרשמה והתחברות עם הרשאות נפרדות לחברה ולמועמד. פיתוח מנגנון ליצירת משרה פתוחה ולהתאמת שאלות ראיון לפי תיאור המשרה. מימוש מנגנון לקבלת תשובות מועמדים בטקסט או בקול. חיבור רכיב תמלול קולי הממיר תשובה מוקלטת לטקסט. פיתוח מנגנון לבדיקת רלוונטיות בין שאלה לתשובה ומתן הערה במקרה של תשובה לא מספקת. שילוב מודלי Python לניתוח מאפייני התאמה כגון ניסיון, יכולת חשיבה ויכולות מעשיות. פיתוח אלגוריתם לחישוב ציון התאמה סופי באמצעות גרף ראיות משוקלל. שמירת הנתונים במסד נתונים והצגת ציון וסיכום ברור לחברה.

4. אתגרים:

במהלך העבודה על הפרויקט התמודדתי עם מספר קשיים מסוגים שונים. אחד האתגרים המרכזיים היה להבין כיצד לשלב בין מערכת Web רגילה לבין מודלים של עיבוד שפה טבעית ולמידת מכונה. נדרשתי ללמוד כיצד להעביר מידע מהשרת למודלי Python, לקבל מהם תוצאות, לשמור את התוצאות במסד הנתונים ולהציג אותן למשתמש בצורה ברורה. אתגר נוסף היה התאמת האלגוריתמים לבעיה הספציפית של ראיונות עבודה. לא היה מספיק לבדוק אם קיימות מילים מסוימות בתשובה, אלא היה צורך לבדוק האם התשובה עונה על השאלה מבחינה רעיונית, האם היא מפורטת מספיק, והאם היא מספקת ראיות אמיתיות להתאמת המועמד למשרה. אתגר טכנולוגי נוסף היה שילוב קלט קולי במערכת. בתחילה היה קובץ Python שהאזין ישירות למיקרופון, אך פתרון זה לא התאים לאתר. לכן נדרשתי להבין כיצד להקליט אודיו בדפדפן, לשלוח אותו לשרת, להריץ תמלול באמצעות Python ולהחזיר את הטקסט לתיבת התשובה. כמו כן, התמודדתי עם קשיים הקשורים לשמירת נתונים והרשאות. היה צורך לוודא שכל ראיון משויך למשרה, למועמד ולחברה הנכונים, ושחברה לא תוכל לצפות בציונים של ראיונות שאינם שייכים לה. בנוסף, נדרשתי לשמור מידע רגיש בצורה מאובטחת ולשלב מנגנוני התחברות והרשאות. קשיים אלו דרשו למידה עצמאית, ניסוי וטעייה, בדיקות חוזרות ושיפור הדרגתי של המערכת עד ליצירת זרימה מלאה: יצירת משרה, התאמת שאלות, התחלת ראיון על ידי מועמד, ניתוח תשובות וחשוב ציון התאמה סופי.

5. מדדי הצלחה למערכת:

הצלחת המערכת תימדד לפי היכולת שלה לבצע תהליך ראיון מלא מקצה לקצה: יצירת משרה על ידי חברה, התאמת שאלות למשרה, התחלת ראיון על ידי מועמד, קבלת תשובות טקסטואליות או קוליות, בדיקת רלוונטיות, הרצת מודלי אבחון וחישוב ציון התאמה סופי.

מדדי ההצלחה המרכזיים שהוגדרו למערכת הם:

- 100% הצלחה בביצוע זרימת מערכת מלאה ללא שגיאות: יצירת משרה, שמירת שאלות, התחלת ראיון, שליחת תשובות וחישוב ציון.
- לפחות 80% הצלחה בזיהוי תשובות רלוונטיות לעומת תשובות שאינן עונות על השאלה, לפי בדיקות ידניות שבוצעו על תשובות לדוגמה.
- יכולת להבחין בין תשובה רלוונטית או לא, ולהציג למועמד הערה מתאימה במקרה הצורך.
- שמירת כל הנתונים המרכזיים במסד הנתונים: משרות, שאלות, ראיונות, תשובות, תוצאות רלוונטיות, תוצאות אבחון וציון סופי.
- הפקת ציון התאמה סופי הכולל לא רק מספר, אלא גם הסבר מילולי, חוזקות, חולשות ודפוסים מרכזיים שעלו לאורך הראיון.
- יציבות המערכת בעבודה עם קלטים שונים, כולל תשובות קצרות, תשובות ארוכות, ניסוחים שונים ותשובות קוליות שעוברות תמלול.
- שמירה על הרשאות תקינות, כך שחברה תוכל לצפות רק בראיונות ובציונים השייכים לה.
- בנוסף, הצלחה תוגדר בכך שהמערכת אינה מציגה למשתמש רק נתונים גולמיים, אלא מספקת תובנות ברורות ומובנות המסייעות לחברה להבין את מידת התאמת המועמד למשרה.

6. רקע תיאורטי:

הפרויקט עוסק בתחום של ניתוח תשובות מועמדים בתהליך גיוס, ולכן הוא משלב בין ידע מתחום משאבי האנוש, ראיונות עבודה, הבנת שפה טבעית ולמידת מכונה.

בתהליך גיוס עובדים, ראיון עבודה משמש כלי מרכזי להערכת התאמת מועמד לתפקיד, אך כאשר הראיון אינו מובנה, ההערכה עלולה להיות מושפעת מהבדלים בין מראיינים, מניסוח שאלות שונה ומפרשנות סובייקטיבית.

מחקרים בתחום המיון התעסוקתי מצביעים על כך שראיונות מובנים, שבהם מועמדים נשאלים שאלות דומות ומוערכים לפי קריטריונים מוגדרים, מסייעים לשפר עקביות והוגנות בתהליך ההערכה.

המערכת שפותחה בפרויקט מתבססת גם על עיבוד שפה טבעית, תחום במדעי המחשב ובבינה מלאכותית העוסק ביכולת של מחשבים להבין, לעבד ולנתח שפה אנושית כתובה או מדוברת.

NLP מאפשר למערכות ממוחשבות לעבוד עם טקסט חופשי, לזהות משמעות, להפיק תובנות ולבצע פעולות על בסיס שפה טבעית ולא רק על בסיס נתונים מובנים.

בפרויקט זה, עיבוד השפה אינו משמש רק לזיהוי מילים, אלא לצורך הבנת הקשר בין שאלה לתשובה. המערכת נדרשת לבדוק האם תשובת מועמד עונה על השאלה מבחינה רעיונית, האם היא מפורטת מספיק, והאם ניתן להסיק ממנה מאפיינים הרלוונטיים למשרה.

לכן יש צורך לשלב בין ניתוח לשוני, ניתוח סמנטי, בדיקת רלוונטיות והפקת מדדים כמותיים.

בנוסף, הפרויקט נשען על מושגים מתחום הערכת מועמדים, כגון ניסיון, יכולת חשיבה, יכולות מעשיות, אחריות, בהירות ותקשורת.

אחד המודלים המוכרים בתחום האישיות הוא מודל Big Five, המתאר חמישה ממדים מרכזיים של אישיות: פתיחות להתנסויות, מצפוניות, מוחצנות, נעימות ונורטיות.

מודל זה משמש כרקע תיאורטי להבנה כיצד ניתן לתאר מאפייני אדם באמצעות ממדים מוגדרים, אם כי בפרויקט הנוכחי המערכת אינה מסתפקת באישיות בלבד, אלא בוחנת גם היבטים מקצועיים והתנהגותיים נוספים.

6.1 מושגים עיקריים בתחום הדעת של הבעיה:

אבחון מועמדים בתהליכי גיוס:

אבחון מועמדים הוא תהליך שמטרתו לבדוק עד כמה אדם מתאים לתפקיד מסוים ולסביבת העבודה. האבחון אינו מתבסס רק על ידע טכני, אלא גם על יכולת חשיבה, ניסיון קודם, דרך התמודדות עם בעיות, אחריות, תקשורת וסגנון עבודה.

ראיון מובנה:

ראיון מובנה הוא ראיון שבו מועמדים נשאלים שאלות מוגדרות מראש, והערכת התשובות נעשית לפי קריטריונים קבועים. גישה זו מאפשרת להשוות בין מועמדים בצורה עקבית יותר, מכיוון שכל מועמד נבחן מול אותן דרישות בסיסיות.

עיבוד שפה טבעית:

עיבוד שפה טבעית הוא תחום העוסק ביכולת של מחשב להבין ולעבד שפה אנושית. בפרויקט זה נעשה שימוש בעיבוד שפה טבעית כדי לנתח תשובות חופשיות של מועמדים, לבדוק את הקשר שלהן לשאלה ולהפיק מהן מדדים רלוונטיים.

רלוונטיות תשובה:

רלוונטיות תשובה מתארת את מידת ההתאמה בין השאלה שנשאלה לבין התוכן שהמועמד סיפק. תשובה יכולה להיות רלוונטית גם אם אינה משתמשת באותן מילים בדיוק כמו השאלה, כל עוד היא עונה על הרעיון המרכזי שלה.

איכות חשיבה:

איכות חשיבה מתייחסת ליכולת של המועמד לנתח מצב, להסביר תהליך, להציג נימוקים, להסיק מסקנות ולתאר פתרון בצורה מסודרת. בפרויקט זה מאפיין זה חשוב במיוחד בשאלות המתייחסות לפתרון בעיות, חקירת תקלות וקבלת החלטות.

יכולות מעשיות:

יכולות מעשיות מתארות את מה שהמועמד יודע לבצע בפועל. לדוגמה, בתפקיד Backend Developer נבדקות יכולות כמו עבודה עם APIs, מסדי נתונים, לוגיקה בצד שרת, פתרון תקלות וכתובת קוד מאובטח.

ניסיון:

ניסיון מתייחס לחשיפה של המועמד למצבים מקצועיים קודמים, לרמת האחריות שנשא, וליכולת שלו להביא דוגמאות ממשיות מתוך עבודה או פרויקטים שביצע.

מאפייני אישיות:

מאפייני אישיות מתארים נטיות יחסית יציבות של האדם, כגון אחריות, פתיחות, שיתוף פעולה ויכולת התמדה. מודל Big Five הוא אחד המודלים המרכזיים בתחום זה, והוא מתאר חמישה ממדים רחבים של אישיות.

מוטיבציה והתאמה לסביבה:

מוטיבציה בוחנת מה מניע את המועמד ומה מידת הרצון שלו להשתלב בתפקיד. התאמה לסביבה מתייחסת ליכולת של המועמד להשתלב בצוות, לתקשר בצורה ברורה ולעבוד לפי ציפיות הארגון.

גרף ראיות משוקלל:

גרף ראיות משוקלל הוא מבנה לוגי שבו המערכת מחברת בין דרישות המשרה, שאלות הראיון, תשובות המועמד, תוצאות המודלים והדפוסים שהתגלו לאורך הראיון. באמצעות מבנה זה ניתן לחשב ציון התאמה סופי שאינו מבוסס על תשובה אחת בלבד, אלא על כלל הראיות שנאספו.

6.2 נושאים רלוונטיים בתחום הדעת של הבעיה:

במסגרת המחקר המקדים נבחנו מספר נושאים ומחקרים הקשורים להבנת תשובות חופשיות, הערכת מועמדים וניתוח שפה טבעית.

מחקר 1: דמיון סמנטי בין תשובות

אחד הנושאים המרכזיים בפרויקט הוא היכולת להשוות בין שאלה לתשובה לא רק לפי מילים זהות, אלא לפי משמעות.

במאמר *Semantic Answer Similarity for Evaluating Question Answering Models* הוצג מדד בשם SAS, שמטרתו להעריך דמיון סמנטי בין תשובות באמצעות מודלים מבוססי Transformer. המחקר מצביע על כך שמדדים סמנטיים מתקדמים מתאימים יותר לשיפוט אנושי לעומת מדדים מילוליים פשוטים, מכיוון שהם מסוגלים לזהות תשובות בעלות משמעות דומה גם כאשר אין חפיפה מילולית מלאה. נושא זה רלוונטי לפרויקט, משום שגם במערכת יש צורך לבדוק האם תשובת מועמד עונה על השאלה מבחינה רעיונית ולא רק מבחינת מילים זהות.

מחקר 2: ראיונות מובנים והערכת מועמדים

בתחום הגיוס, אחת הדרכים לשפר את איכות ההערכה היא שימוש בראיונות מובנים, שבהם מועמדים נשאלים שאלות דומות ונבדקים לפי קריטריונים מוגדרים. מחקרים בתחום מיון עובדים מצביעים על כך שראיונות מובנים וכלי הערכה עקביים יכולים לסייע בהשוואה מסודרת יותר בין מועמדים.

נושא זה רלוונטי לפרויקט, מכיוון שהמערכת שואפת ליצור תהליך ראיון אחיד יותר: החברה יוצרת משרה, המערכת מתאימה שאלות, וכל מועמד נבחן לפי שאלות וקריטריונים שנשמרים במערכת.

מחקר 3: עיבוד שפה טבעית והבנת טקסט חופשי

עיבוד שפה טבעית הוא תחום במדעי המחשב ובבינה מלאכותית העוסק ביכולת של מחשבים להבין, לעבד ולנתח שפה אנושית.

התחום כולל פעולות כמו עיבוד טקסט, זיהוי משמעות, ניתוח הקשר והפקת תובנות מתוך שפה טבעית. בפרויקט זה NLP משמש לניתוח תשובות מועמדים, בדיקת רלוונטיות בין שאלה לתשובה, והפקת מדדים המשמשים בהמשך לחישוב ציון התאמה.

מחקר 4: מודלים של אישיות ומאפייני אדם

בתחום הערכת מועמדים נעשה שימוש גם במושגים מעולם האישיות וההתנהגות. אחד המודלים המוכרים הוא מודל Big Five, המתאר חמישה ממדים מרכזיים של אישיות: פתיחות, מצפוניות, מוחצנות, נעימות ונירוטיות. בפרויקט זה המודל משמש כרקע תאורטי להבנת האפשרות לתאר אדם באמצעות מאפיינים מדידים, אך המערכת אינה מסתפקת באישיות בלבד, אלא בוחנת גם ניסיון, יכולות מעשיות, איכות חשיבה, אחריות ותקשורת.

6.3 סקירה מקצועית של הבעיה בעולם:

בעולם הגיוס, הערכת מועמדים מתבצעת לרוב באמצעות שילוב של קורות חיים, ראיונות עבודה, שאלונים, מבחני אישיות, מבחני יכולת ולעיתים גם מרכזי הערכה. לכל אחד מהכלים הללו יש יתרונות, אך גם מגבלות. קורות חיים מאפשרים לקבל מידע ראשוני על ניסיון והשכלה, אך הם אינם מציגים בהכרח כיצד המועמד חושב, מסביר, פותר בעיות או מתמודד עם שאלות פתוחות. ראיונות אנושיים מאפשרים להתרשם מהמועמד בצורה רחבה יותר, אך הם תלויים במראיין, בניסוח השאלות, בזמן הזמין ובשיקול דעת אנושי. כאשר הראיון אינו מובנה, מועמדים שונים עשויים לקבל שאלות שונות ולהיות מוערכים לפי קריטריונים שאינם זהים. לכן בעולם הגיוס קיימת חשיבות לשימוש בראיונות מובנים ובקריטריונים אחידים, כדי לאפשר השוואה עקבית יותר בין מועמדים. בנוסף, ארגונים רבים משתמשים בכלים אוטומטיים לסינון ראשוני, בעיקר על בסיס קורות חיים ומילות מפתח. כלים אלו יכולים לחסוך זמן, אך הם אינם מספיקים כדי להבין תשובות חופשיות של מועמד או להעריך את דרך החשיבה שלו. הבעיה המרכזית שנותרת היא היכולת להבין לא רק האם מועמד השתמש במילים מתאימות, אלא האם הוא ענה על השאלה, הביא דוגמה רלוונטית, הראה יכולת ניתוח, והציג ראיות ממשיות להתאמתו לתפקיד.

הפרויקט מתמודד עם בעיה זו באמצעות שילוב בין ראיון מובנה, ניתוח תשובות חופשיות, בדיקת רלוונטיות וחישוב ציון התאמה מבוסס ראיות. בכך הוא מנסה לתת מענה לצורך הקיים בעולם הגיוס: הערכה עקבית וברורה יותר של מועמדים, תוך התייחסות למשמעות התשובות ולא רק לנתונים חיצוניים או למילות מפתח.

6.4 נושאים רלוונטיים במדעי המחשב בהקשר לבעיה:

בתחום מדעי המחשב קיימות מספר משפחות של שיטות ואלגוריתמים הרלוונטיות לבעיה שבה עוסק הפרויקט.

עיבוד שפה טבעית- NLP

עיבוד שפה טבעית הוא תחום העוסק ביכולת של מחשבים לעבוד עם שפה אנושית. התחום כולל ניתוח טקסט, הבנת משמעות, זיהוי קשרים בין משפטים, סיווג טקסט והפקת תובנות מתוך מידע לא מובנה. בפרויקט זה התחום רלוונטי משום שתשובות המועמדים הן טקסט חופשי, ולכן לא ניתן לנתח אותן כמו נתונים מספריים רגילים.

ייצוג טקסט באמצעות Embeddings

אחת הגישות המרכזיות בניתוח טקסט היא המרת מילים, משפטים או מסמכים לזוקטורים מספריים. ייצוג זה מאפשר למחשב לחשב קרבה בין טקסטים לפי משמעות. לדוגמה, שתי תשובות יכולות להיות שונות במילים שלהן אך קרובות מבחינה סמנטית. גישה זו רלוונטית לבדיקת התאמה בין שאלת ראיון לבין תשובת מועמד.

מודלי דמיון סמנטי

מודלי דמיון סמנטי משמשים להשוואה בין טקסטים לפי משמעותם. במקום לבדוק רק מילים זהות, המודל בוחן האם שני טקסטים מבטאים רעיון דומה. בפרויקט זה עיקרון זה חשוב במיוחד, מכיוון שמועמד יכול לענות תשובה נכונה ורלוונטית גם אם לא השתמש בדיוק במילים שהופיעו בשאלה.

מודלי Transformer

מודלי Transformer הם משפחת מודלים מתקדמת בתחום עיבוד השפה הטבעית. הם מבוססים על מנגנון Attention, שמאפשר למודל להבין קשרים בין מילים שונות במשפט ובהקשר רחב יותר.

מודלים אלו נמצאים בשימוש במשימות רבות של הבנת טקסט, סיווג, דמיון סמנטי ושאלות-תשובות. מאמר SAS, לדוגמה, משתמש בגישה מבוססת Transformer לצורך חישוב דמיון סמנטי בין תשובות.

סיווג טקסט

סיווג טקסט הוא משפחת אלגוריתמים שמטרתה לשייך טקסט לקטגוריה מסוימת. בפרויקט זה רעיון הסיווג בא לידי ביטוי בזיהוי האם תשובה רלוונטית או לא רלוונטית, וכן בזיהוי מאפיינים כמו איכות חשיבה, ניסיון או יכולות מעשיות.

שילוב מודלים וכללים לוגיים

במערכות מורכבות לא תמיד מספיק להשתמש במודל אחד. לעיתים יש צורך לשלב בין תוצאות של כמה מודלים לבין כללים לוגיים, כדי לקבל החלטה יציבה ומוסברת יותר.

בפרויקט זה עיקרון זה בא לידי ביטוי בכך שתשובות המועמד מנותחות על ידי כמה מודלים, ולאחר מכן משולבות בציון סופי באמצעות גרף ראיות משוקלל.

7. תיאור פתרונות קיימים לבעיה:

בעיית ניתוח תשובות מועמדים והערכת התאמתן לשאלות נבחנה בעולם במספר גישות אלגוריתמיות מרכזיות.

הגישות נבדלות זו מזו ברמת ההבנה הסמנטית, ביכולת ההסבר, במורכבות החישובית וביכולת להתמודד עם טקסט חופשי.

הגישה הראשונה היא התאמת מילות מפתח.

בגישה זו הטקסט מיוצג כאוסף מילים או ביטויים, והמערכת בודקת האם קיימת חפיפה בין מילות השאלה לבין מילות התשובה.

לעיתים נעשה שימוש בשיטות כמו TF-IDF, המעניקות משקל שונה למילים לפי שכיחותן וחשיבותן. יתרונה של שיטה זו הוא פשטות ומהירות, אך חסרונה המרכזי הוא שהיא אינה מבינה משמעות. מועמד יכול לענות תשובה נכונה במילים שונות לחלוטין, והמערכת עלולה שלא לזהות זאת.

גישה נוספת היא דמיון סמנטי מבוסס Embeddings.

בגישה זו משפטים או טקסטים מומרים לוקטורים מספריים במרחב מתמטי, כך שטקסטים בעלי משמעות דומה נמצאים קרוב זה לזה.

לאחר מכן ניתן לחשב דמיון בין השאלה לתשובה באמצעות מדדים כגון Cosine Similarity. גישה זו מאפשרת הבנה טובה יותר של משמעות לעומת התאמת מילות מפתח, אך לרוב היא מספקת ציון דמיון בלבד, ואינה מסבירה באופן ברור מדוע תשובה נחשבת טובה או לא מספקת.

גישה שלישית היא שימוש במודלי סיווג.

בגישה זו מאמנים מודל למידת מכונה לדוגמא על זוגות של שאלה ותשובה, כאשר לכל זוג קיימת תווית כגון "רלוונטי" או "לא רלוונטי".

המודל לומד לזהות דפוסים בטקסט ומחזיר החלטה ברורה.

יתרון הגישה הוא שהיא מאפשרת סיווג ישיר של תשובות, אך היא תלויה באיכות הדאטה ובכמות הדוגמאות שעליהן אומן המודל.

בנוסף, במקרים רבים קשה להבין מדוע המודל קיבל החלטה מסוימת.

גישה נוספת היא מודלי הסקה בשפה טבעית, Natural Language Inference.

מודלים אלו בוחנים את הקשר הלוגי בין שני טקסטים, ומסווגים אותו לקטגוריות כגון נביעה, סתירה או ניטרלי. גישה זו מאפשרת ניתוח עמוק יותר של הקשר בין טקסטים, ולכן היא רלוונטית למשימות שבהן יש לבדוק אם תשובה באמת עונה על שאלה.

עם זאת, גם גישה זו רגישה לניסוחים שונים ולעיתים אינה מספקת פירוק מפורט של סיבת הכשל.

גישה חמישית היא מערכות מבוססות כללים.

במערכות אלו מוגדרים חוקים מראש, לדוגמה: תשובה קצרה מדי תיחשב לא מספקת, תשובה ללא דוגמה תיחשב חלקית, או תשובה שאינה כוללת מונחים מקצועיים מסוימים תיחשב חלשה.

גישה זו מאפשרת שליטה גבוהה והסבר ברור, אך היא מתקשה להתמודד עם שפה טבעית מורכבת, ניסוחים מגוונים ומקרים שלא נצפו מראש.

מהסקירה עולה כי לכל גישה יש יתרונות וחסרונות.

לכן, לצורך מערכת לניתוח ראיונות עבודה, אין די בשימוש בשיטה אחת בלבד.

נדרש שילוב בין יכולת סמנטית, מודלי סיווג, כללים לוגיים ומנגנון הסבר, כדי לקבל מערכת יציבה ומובנת יותר.

8. ניתוח חלופות מערכתי:

גישת התאמת מילות המפתח חזקה מבחינת פשטות, מהירות וקלות מימוש.

היא אינה דורשת מודלים מורכבים או משאבי חישוב רבים, ולכן מתאימה למערכות בסיסיות או לסינון ראשוני. עם זאת, חולשתה המרכזית היא היעדר הבנה סמנטית.

היא עלולה לפסול תשובה טובה רק מפני שנוסחה במילים אחרות, או לקבל תשובה חלשה מפני שהכילה מילים מתאימות.

ההזדמנות בגישה זו היא להשתמש בה כרכיב עזר פשוט, אך האיום הוא קבלת החלטות שטחיות ולא מדויקות.

גישת ה-Embedding חזקה בכך שהיא מאפשרת לזהות דמיון רעיוני בין טקסטים, גם כאשר אין חפיפה מילולית מלאה.

היא מתאימה יותר לניתוח תשובות חופשיות, משום שהיא מייצגת משמעות במרחב מתמטי. חולשתה היא בכך שהיא מחזירה בדרך כלל ציון דמיון בלבד, ללא הסבר מפורט. בנוסף, במקרים מורכבים היא עלולה לזהות קרבה כללית בין טקסטים גם כאשר התשובה אינה באמת עונה על השאלה. ההזדמנות בגישה זו היא שיפור משמעותי לעומת בדיקת מילים, אך האיום הוא קושי להסביר למשתמש את סיבת ההחלטה. מודלי סיווג מאפשרים לקבל החלטה ברורה וממוקדת, לדוגמה האם התשובה רלוונטית או לא. חוזק נוסף שלהם הוא יכולת ללמוד דפוסים מתוך נתונים. עם זאת, הם תלויים מאוד באיכות הדאטה שעליו אומנו, ועלולים להתקשות במקרים חדשים או ניסוחים חריגים. בנוסף, הם עלולים לפעול כ"קופסה שחורה" בלי להסביר באופן מלא את ההחלטה. ההזדמנות היא התאמה טובה למשימות מוגדרות, והאיום הוא תלות גבוהה בדאטה ובאיכות האימון. מודלי NLI מספקים יכולת מתקדמת יותר לניתוח הקשר בין שני טקסטים, ולכן הם מתאימים למשימות שבהן נדרש להבין אם תשובה באמת נובעת מהשאלה או קשורה אליה. חוזקם הוא בהבנה לוגית וסמנטית עמוקה יותר. חולשתם היא רגישות לניסוח ולעיתים קושי בהתמודדות עם תשובות ארוכות, כלליות או לא מסודרות. ההזדמנות היא שימוש בהם כבסיס לבדיקת רלוונטיות מתקדמת, והאיום הוא חוסר יציבות במקרי קצה. מערכות מבוססות כללים חזקות בכך שהן שקופות, ניתנות לשליטה ומאפשרות הסבר ברור. לדוגמה, ניתן להגדיר חוקים לזיהוי תשובה קצרה מדי, היעדר דוגמה או ניסיון חוזר. עם זאת, חולשתן היא שהן אינן גמישות מספיק לשפה טבעית מורכבת. קשה להגדיר מראש חוקים שיתאימו לכל ניסוח אפשרי. ההזדמנות היא לשלב כללים עם מודלים חכמים כדי לשפר יציבות, והאיום הוא שמערכת המבוססת רק על כללים תהיה מוגבלת מדי. מניתוח החלופות עולה כי הפתרון המתאים ביותר אינו מבוסס על גישה אחת בלבד, אלא על שילוב בין מספר שיטות. שילוב כזה מאפשר ליהנות מהבנה סמנטית של מודלים, מהחלטות ברורות של סיווג, ומהיציבות וההסבר של כללים לוגיים.

9. תיאור החלופה הנבחרת ונימוקים:

החלופה שנבחרה בפרויקט היא פתרון משולב, המשלב בין ניתוח סמנטי של טקסט, מודלי סיווג, מודלי אבחון, כללים לוגיים וגרף ראיות משוקלל. מטרת השילוב היא ליצור מערכת שאינה מסתפקת בבדיקת מילים או בציון נקודתי, אלא מנתחת את משמעות התשובה ואת התאמת המועמד למשרה לאורך כל הראיון. האלגוריתם הנבחר מתחיל כבר בשלב יצירת המשרה. החברה מזינה תיאור חופשי של התפקיד, והמערכת משתמשת במידע זה כדי לבחור שאלות ראיון מתאימות. לאחר מכן, כאשר מועמד עונה על שאלה, המערכת בודקת האם התשובה רלוונטית לשאלה מבחינה רעיונית. אם התשובה אינה מספקת, המערכת מציגה למועמד הערה לשיפור ומאפשרת ניסיון נוסף. לאחר שתשובה נמצאה רלוונטית, המערכת מפעילה מנגנון ניתוב הקובע אילו מודלי אבחון מתאימים להפעלה על התשובה. המודלים בוחנים מאפיינים שונים של המועמד, כגון ניסיון, יכולת חשיבה, יכולות מעשיות, בהירות, אחריות ותקשורת. בנוסף, מחושבים מאפיינים נגזרים כגון מוטיבציה והתאמה לסביבה ארגונית. בשלב הסופי, המערכת אינה מחשבת ציון לפי תשובה אחת בלבד, אלא משלבת את כלל הראיות שנאספו לאורך הראיון באמצעות גרף ראיות משוקלל. בגרף זה מיוצגים הקשרים בין המשרה, המאפיינים החשובים לתפקיד, שאלות הראיון, תשובות המועמד, ציוני המודלים ודפוסים חוזרים.

לכל מאפיין ניתן משקל בהתאם לחשיבותו למשרה, והמערכת מתחשבת גם באמינות התשובות ובכיסוי הראיות.

שלבי האלגוריתם בפועל:

1. קבלת תיאור משרה חופשי מהחברה.
 2. התאמת שאלות ראיון למשרה.
 3. קבלת תשובת מועמד בטקסט או בקול.
 4. במקרה של קול, המרת האודיו לטקסט.
 5. בדיקת רלוונטיות בין השאלה לתשובה.
 6. במקרה של תשובה לא מספקת, הצגת הסבר וניסיון חוזר.
 7. הפעלת מנגנון ניתוב לבחירת מודלי האבחון המתאימים.
 8. הפעלת מודלי אבחון על התשובה.
 9. חישוב מאפיינים נגזרים כגון מוטיבציה והתאמה תרבותית.
 10. איסוף כלל הראיות מכל תשובות הראיון.
 11. בניית גרף ראיות משוקלל.
 12. חישוב ציון התאמה סופי והפקת סיכום מוסבר לחברה.
- הבחירה בפתרון משולב נעשתה מכמה סיבות.
- ראשית, טקסט חופשי דורש הבנה סמנטית ולא ניתן להסתפק בבדיקת מילות מפתח.
- שנית, ראיונות עבודה דורשים הסבר ולא רק תוצאה מספרית, ולכן היה צורך במנגנון שמסוגל להציג למשתמש חזקות, חולשות ודפוסים.
- ובנוסף, התאמה למשרה אינה נמדדת על פי תשובה אחת, אלא על פי מכלול של ראיות לאורך הראיון.
- לכן נבחר גרף ראיות משוקלל, המאפשר לשלב בין תוצאות המודלים לבין חשיבות המאפיינים של המשרה.
- פתרון זה מתאים לפרויקט משום שהוא משלב בין גמישות של מודלים חכמים, יציבות של כללים לוגיים, יכולת הסבר למשתמש, ושמירה על תהליך הערכה אחיד ומובנה.

10. אפיון המערכת:

- המערכת כוללת שני סוגי משתמשים עיקריים: חברה ומועמד.
- החברה יוצרת משרה פתוחה, מזינה תיאור חופשי של התפקיד והדרישות, מקבלת שאלות ראיון מותאמות למשרה ושומרת אותן.
- המועמד צופה במשרות פתוחות, מצטרף לראיון ועונה על שאלות בטקסט או בקול.
- רכיבי המערכת המרכזיים הם:
1. **ממשק משתמש לחברה:**
ממשק זה מאפשר הרשמה והתחברות של חברה, יצירת משרה פתוחה, הזנת תיאור משרה, צפייה בשאלות ראיון מותאמות, שמירת שאלות למשרה וצפייה בציוני מועמדים.
 2. **ממשק משתמש למועמד:**
ממשק זה מאפשר הרשמה והתחברות של מועמד, צפייה במשרות פתוחות, התחלת ראיון, מענה על שאלות, הקלטת תשובה קולית וקבלת הערה לשיפור במקרה שהתשובה אינה רלוונטית.
 3. **שרת Backend:**
השרת אחראי על ניהול הבקשות מהלקוח, בדיקת הרשאות, יצירת משרות, שמירת שאלות, יצירת ראיונות, שמירת תשובות, הפעלת מודלים וחישוב ציון סופי.
 4. **מוד נתונים:**
מוד הנתונים שומר את המשתמשים, החברות, המועמדים, המשרות, שאלות הראיון, תשובות המועמדים, בדיקות הרלוונטיות, תוצאות האבחון והציונים הסופיים.
 5. **מודול התאמת שאלות למשרה:**
מודול זה מקבל את תיאור המשרה שהוזן על ידי החברה, מנתח את הדרישות המרכזיות של התפקיד, ובוחר מתוך מאגר שאלות את השאלות המתאימות ביותר למשרה.

מטרת המודול היא לוודא שהראיון אינו כללי בלבד, אלא בודק מאפיינים החשובים לתפקיד הספציפי, כגון ניסיון, יכולת חשיבה, יכולות מעשיות, אחריות ותקשורת.

6. מודול תמלול קולי:

מודול זה מקבל קובץ אודיו שהוקלט בדפדפן, ממיר אותו לטקסט ומחזיר את הטקסט לתיבת התשובה של המועמד.

7. מודול בדיקת רלוונטיות:

מודול זה בודק האם תשובת המועמד עונה על השאלה מבחינה רעיונית. במקרה שהתשובה אינה מספקת, המערכת מעבירה למודול הסיבה ומציגה הודעת הכוונה ומאפשרת למועמד לנסות שוב.

8. מודול ניתוב מודלים:

מודול זה קובע אילו מודלי אבחון יש להפעיל על כל תשובה, בהתאם לשאלה, לתוכן התשובה ולמאפיינים שהשאלה נועדה לבדוק.

9. מודולי אבחון:

מודולים אלו מנתחים מאפיינים שונים של המועמד, כגון ניסיון, יכולת חשיבה, יכולות מעשיות, בהירות, אחריות, תקשורת, מוטיבציה והתאמה לסביבה ארגונית.

10. מודול חישוב ציון סופי:

מודול זה אוסף את כלל תוצאות האבחון מכל תשובות הראיון, בונה גרף ראיות משוקלל ומחשב ציון התאמה סופי למועמד.

בנוסף הוא מפק סיוכום הכולל חוזקות, חולשות ודפוסים מרכזיים.

זרימת המידע במערכת:

חברה יוצרת משרה פתוחה < החברה מזינה תיאור משרה < מודול התאמת השאלות בוחר שאלות רלוונטיות למשרה < החברה שומרת את השאלות < מועמד בוחר משרה ומתחיל ראיון < המועמד עונה בטקסט או בקול < המערכת בודקת רלוונטיות < במקרה הצורך מופעל מודול הבנת הכשל < המערכת מפעילה מודלי אבחון < התוצאות נשמרות במסד הנתונים < בסיום הראיון מחושב ציון התאמה סופי < החברה צופה בציון ובסיוכום המועמד.

10.1 ניתוח דרישות המערכת:

המערכת נדרשת לאפשר תהליך ראיון מלא מקצה לקצה, החל מיצירת משרה על ידי חברה ועד להצגת ציון התאמה וסיוכום מועמד.

כדי שהפרייקט יעבוד בצורה תקינה, נדרש שכל רכיבי המערכת יעבדו יחד: ממשק משתמש, שרת, מסד נתונים ומודלי Python.

הדרישות המרכזיות של המערכת הן:

1. ניהול משתמשים והרשאות:

המערכת צריכה לאפשר הרשמה והתחברות לשני סוגי משתמשים: חברה ומועמד.

לכל סוג משתמש יש הרשאות שונות.

חברה יכולה ליצור משרות, לשמור שאלות ולצפות בציונים של ראיונות השייכים לה בלבד.

מועמד יכול לצפות במשרות פתוחות, להתחיל ראיון ולענות על שאלות.

2. יצירת משרה והתאמת שאלות:

המערכת נדרשת לאפשר לחברה להזין תיאור חופשי של משרה, לנתח את דרישות התפקיד, ולהציע שאלות ראיון מתאימות.

השאלות צריכות להיות רלוונטיות לתפקיד, ללא כפילויות, ולהישמר במסד הנתונים עבור אותה משרה.

3. ביצוע ראיון על ידי מועמד:

המערכת צריכה לאפשר למועמד לבחור משרה פתוחה, להתחיל ראיון אישי, ולקבל את השאלות שנשמרו עבור אותה משרה.

המועמד יכול לענות באמצעות טקסט או באמצעות הקלטה קולית.

4. תמיכה בקלט קולי:

המערכת נדרשת לאפשר הקלטת תשובה קולית בדפדפן, שליחת קובץ האודיו לשרת, תמלול לטקסט באמצעות Python, והחזרת הטקסט לתיבת התשובה של המועמד.

5. בדיקת רלוונטיות:

עבור כל תשובה, המערכת צריכה לבדוק האם התשובה עונה על השאלה מבחינה רעיונית. במקרה שהתשובה אינה מספקת, המערכת צריכה להציג למועמד הערה ברורה ולאפשר ניסיון נוסף.

6. ניתוח תשובות והרצת מודלים ואת ראטר הניתוב למודלים:

המערכת נדרשת להפעיל מודלי אבחון שונים לפי תשובות המועמד, כגון מודלים לניתוח ניסיון, יכולות מעשיות, איכות חשיבה ומאפיינים נוספים.

7. חישוב ציון סופי:

בסיום הראיון, המערכת צריכה לאחד את כלל התוצאות שנאספו לאורך הראיון ולחשב ציון התאמה סופי. הציון צריך להתבסס על חשיבות מאפייני המשרה, איכות התשובות, אמינותן, כיסוי הראיות ודפוסים חוזרים לאורך הראיון.

8. הצגת תוצאות:

המערכת צריכה להציג לחברה ציון התאמה ברור ומעוצב, הכולל ציון מספרי, חוזקות, חולשות יחסיות, פירוט מאפיינים וסיכום מילולי.

9. שמירת נתונים:

כל הנתונים המרכזיים צריכים להישמר במסד הנתונים: משתמשים, חברות, מועמדים, משרות, שאלות, ראיונות, תשובות, תוצאות רלוונטיות, תוצאות אבחון וציונים סופיים.

10. ביצועים וחווית משתמש:

המערכת צריכה להגיב בזמן סביר, להציג למשתמש סימן טעינה בזמן פעולות ארוכות כמו התאמת שאלות, בדיקת תשובה וחישוב ציון, ולספק הודעות ברורות במקרה של שגיאה או הצלחה.

10.2 מודול המערכת:

המערכת בנויה במודל שרת-לקוח ומורכבת ממספר שכבות ומודולים הפועלים יחד.

1. צד לקוח - React:

צד הלקוח אחראי על ממשק המשתמש. הוא כולל מסכים להרשמה והתחברות, אזור חברה, יצירת משרה, בחירת שאלות, צפייה במשרות פתוחות, מענה לראיון וצפייה בציוני מועמדים. בנוסף, צד הלקוח אחראי על הקלטת אודיו בדפדפן ושליחתו לשרת לצורך תמלול.

2. צד שרת - ASP.NET Core:

צד השרת אחראי על קבלת הבקשות מהלקוח, בדיקת הרשאות, ניהול משתמשים, יצירת משרות, שמירת שאלות, יצירת ראיונות, שמירת תשובות, הפעלת מודלי Python וחישוב הציון הסופי.

3. מסד נתונים - PostgreSQL:

מסד הנתונים שומר את כל המידע של המערכת, כולל משתמשים, חברות, מועמדים, משרות, שאלות, ראיונות, תשובות, בדיקות רלוונטיות, תוצאות אבחון וציונים סופיים.

4. מודולי Python:

מודולי Python אחראים על פעולות ניתוח מתקדמות, כגון תמלול אודיו, בדיקת רלוונטיות, ניתוב תשובות למודלים מתאימים, אבחון יכולות וניסיון, וחישוב מאפיינים שונים מתוך הטקסט.

5. מודול התאמת שאלות למשרה:

מודול זה מנתח את תיאור המשרה שהחברה הזינה ובוחר שאלות ראיון רלוונטיות מתוך מאגר שאלות. מטרתו לוודא שהראיון מותאם לתפקיד הספציפי ולא מורכב משאלות כלליות בלבד.

6. מודול בדיקת רלוונטיות:

מודול זה בודק האם תשובת המועמד עונה על השאלה. אם התשובה אינה רלוונטית או אינה מספקת, המערכת מחזירה הודעת הכוונה ומאפשרת ניסיון נוסף.

7. מודול ניתוב למודלי אבחון:

מודול זה קובע אילו מודלים יש להפעיל על כל תשובה, בהתאם לסוג השאלה ולמאפיינים שהשאלה אמורה לבדוק.

8. מודולי אבחון:

מודולים אלו מנתחים היבטים שונים בתשובות המועמד, כגון ניסיון, יכולות מעשיות, איכות חשיבה, אחריות, בהירות ותקשורת.

9. מודול חישוב ציון סופי:

מודול זה אוסף את כלל הראיות מכל הראיון, בונה גרף ראיות משוקלל, ומחשב ציון התאמה סופי למועמד. הציון כולל גם סיכום מילולי שמציג חוזקות, חולשות ודפוסים מרכזיים.

10.3 אפיון פונקציונלי:

המערכת כוללת מספר פעולות מרכזיות המאפשרות לבצע את תהליך הראיון המלא. תחילה, החברה נרשמת למערכת באמצעות שם חברה, שם איש קשר, אימייל וסיסמה. לאחר ההרשמה המערכת יוצרת משתמש מסוג חברה, שומרת את פרטי החברה במסד הנתונים ומחזירה הרשאות מתאימות לעבודה במערכת. באופן דומה, מועמד יכול להירשם באמצעות שם מלא, אימייל, טלפון וסיסמה. המערכת שומרת את פרטי המועמד ומאפשרת לו להתחבר ולצפות במשרות פתוחות. לאחר התחברות החברה, ניתן ליצור משרה פתוחה. החברה מזינה שם משרה ותיאור חופשי של דרישות התפקיד. המערכת בודקת שהמשתמש המחובר הוא חברה, יוצרת משרה חדשה, משייכת אותה לחברה המחוברת ושומרת את המשרה במסד הנתונים. לאחר יצירת המשרה, המערכת מבצעת התאמת שאלות למשרה. היא מקבלת את מזהה המשרה ואת מספר השאלות הרצוי, מנתחת את תיאור המשרה, מזהה מאפיינים חשובים לתפקיד ובוחרת מתוך מאגר שאלות את השאלות הרלוונטיות ביותר. לאחר שהשאלות מוצגות לחברה, החברה יכולה לשמור אותן למשרה. הקלט בשלב זה הוא שאלות שנבחרו מתוך המאגר או תוספת שאלות שהחברה הוסיפה באופן ידני. המערכת שומרת את השאלות תחת המשרה המתאימה, לפי סדר הופעתן, ומוודאת שלא נשמרות שאלות כפולות. לאחר שמירת השאלות, המשרה זמינה למועמדים. בצד המועמד, המערכת מאפשרת לצפות במשרות פתוחות. כאשר מועמד מבקש לראות משרות, המערכת שולפת ממסד הנתונים את המשרות שניתן להצטרף אליהן ומציגה אותן עם שם המשרה ותיאור התפקיד. לאחר שהמועמד בוחר משרה, הוא יכול להתחיל ראיון. המערכת מקבלת את מזהה המשרה, יוצרת ראיון חדש, משייכת אותו למועמד ולמשרה, ושולפת את השאלות שנשמרו עבור אותה משרה. הפלט הוא מזהה ראיון ורשימת שאלות שעליהן המועמד צריך לענות. במהלך הראיון המועמד יכול לענות על השאלות בטקסט או בקול. אם המועמד בוחר לענות בקול, הדפדפן מקליט את האודיו ושולח את קובץ ההקלטה לשרת. השרת שומר את הקובץ באופן זמני, מעביר אותו לסקריפט Python לצורך תמלול, ומחזיר את הטקסט המתומלל לתיבת התשובה של המועמד. לאחר מכן התשובה המתומללת עוברת את אותו תהליך ניתוח כמו תשובה שהוקלדה ידנית. כאשר המועמד שולח תשובה, המערכת מקבלת את מזהה הראיון, מזהה השאלה, נוסח השאלה, טקסט התשובה, מספר הניסיון ומזהה תשובה קודמת במקרה של ניסיון חוזר. המערכת שומרת את התשובה במסד הנתונים, מפעילה בדיקת רלוונטיות בין השאלה לתשובה, ושומרת את תוצאת הרלוונטיות. אם התשובה אינה עונה על השאלה בצורה מספקת, המערכת מחזירה הודעת הכוונה למועמד ומאפשרת לו לנסות לענות שוב. אם התשובה רלוונטית, המערכת מאפשרת מעבר לשאלה הבאה. לאחר שתשובה נמצאה רלוונטית, המערכת מפעילה מנגנון ניתוב למודלי אבחון.

מנגנון זה מקבל את השאלה ואת תשובת המועמד, בודק אילו סוגי מידע קיימים בטקסט, ומחליט אילו מודלים יש להפעיל על התשובה.

לאחר מכן מופעלים מודלי אבחון שונים, הבוחנים מאפיינים כגון ניסיון, יכולת חשיבה, יכולות מעשיות, אחריות, בהירות ותקשורת.

כל מודל מחזיר ציון או קבוצת ציונים עבור המאפיין שהוא בודק, והתוצאות נשמרות במסד הנתונים.

בסיום הראיון המערכת מחשבת ציון התאמה סופי.

הקלט לשלב זה הוא מזהה הראיון וכל הנתונים שנאספו לאורכו: תשובות המועמד, תוצאות בדיקת הרלוונטיות ותוצאות מודלי האבחון.

המערכת אוספת את כלל הראיות, בונה גרף ראיות משוקלל, מחשבת ציוני קטגוריות, מזהה חוזקות וחולשות יחסיות, ומפיקה ציון התאמה סופי.

הפלט הוא ציון מספרי, סיכום מילולי, פירוט מאפיינים ודפוסים חוזרים שנשמרים במסד הנתונים.

לבסוף, החברה יכולה לצפות בציוני מועמדים.

המערכת מקבלת בקשה מהחברה המחוברת, שולפת רק את הציונים של ראיונות השייכים לאותה חברה, ומציגה רשימת ציונים הכוללת את פרטי המשרה, מזהה המועמד, הציון הסופי וסיכום ההתאמה.

כך החברה יכולה לראות את תוצאות הראיונות בצורה מסודרת וברורה.

10.4 ביצועים עיקריים:

המערכת שפותחה מסוגלת לבצע תהליך ראיון עבודה אוטומטי מקצה לקצה, החל מיצירת משרה על ידי חברה ועד להצגת ציון התאמה סופי למועמד.

המערכת מסוגלת ליצור משרה פתוחה על בסיס תיאור חופשי של החברה, להתאים שאלות ראיון רלוונטיות למשרה, למנוע שמירת שאלות כפולות, ולאפשר למועמדים להצטרף לראיון באופן עצמאי.

במהלך הראיון המערכת מאפשרת למועמד לענות על שאלות באמצעות טקסט או קול.

במקרה של תשובה קולית, המערכת מסוגלת להקליט את האודיו בדפדפן, לשלוח אותו לשרת, לתמלל אותו לטקסט ולהכניס את הטקסט לתיבת התשובה.

המערכת מסוגלת לבדוק האם תשובת המועמד עונה על השאלה מבחינה רעיונית ולא רק מילולית.

במקרה שהתשובה אינה מספקת, המערכת מציגה למועמד הערה לשיפור ומאפשרת ניסיון נוסף.

בנוסף, המערכת מפעילה מודלי אבחון שונים על תשובות המועמד ומחשבת מאפיינים כגון ניסיון, יכולת חשיבה, יכולות מעשיות, אחריות, בהירות ותקשורת.

בסיום הראיון המערכת משלבת את כלל התוצאות באמצעות גרף ראיות משוקלל, ומפיקה ציון התאמה סופי הכולל הסבר מילולי, חוזקות, חולשות יחסיות ודפוסים חוזרים.

המערכת גם שומרת את כל הנתונים המרכזיים במסד הנתונים, כולל משרות, שאלות, ראיונות, תשובות, תוצאות רלוונטיות, תוצאות אבחון וציונים סופיים. בנוסף, היא מאפשרת לחברה לצפות בציונים שמורים של ראיונות השייכים לה בלבד.

מבחינת חוויית משתמש, המערכת מציגה סימני טעינה בזמן פעולות ארוכות, כגון התאמת שאלות, בדיקת תשובה וחישוב ציון, ומציגה הודעות ברורות במקרה של הצלחה או שגיאה.

10.5 אילוצים:

במערכת קיימים מספר אילוצים והנחות יסוד, הנובעים מאופי הפרויקט ומהשימוש בעיבוד שפה טבעית.

המערכת תלויה באיכות הטקסט שמתקבל מהמועמד.

תשובות קצרות מאוד, כלליות מדי, לא ברורות או חסרות דוגמה עלולות לפגוע באיכות הניתוח.

בנוסף, המערכת מוגבלת בעיקר לעבודה עם תשובות באנגלית, ולכן תשובות בשפות אחרות עלולות לפגוע בדיוק המודלים.

קיים גם אילוץ הקשור לאיכות המודלים ולדאטה שעליו הם אומנו.

מאחר שמדובר במודלי למידת מכונה ועיבוד שפה טבעית, ייתכנו מקרים שבהם המערכת תטעה בסיווג תשובה או תבחר פרשנות אחת מתוך כמה פרשנויות אפשריות.

אילוץ נוסף הוא זמן עיבוד.

פעולות כמו תמלול אודיו, בדיקת רלוונטיות, הרצת מודלי אבחון וחישוב ציון סופי עשויות לקחת זמן, ולכן המערכת נדרשת להציג למשתמש חיווי טעינה בזמן פעולות אלו. המערכת מניחה שהחברה מזינה תיאור משרה ברור ומספיק מפורט, שהשאלות שנבחרו אכן מתאימות למשרה, ושהמועמד עונה בצורה רצינית וכנה. בנוסף, המערכת מניחה שהקלט הקולי ברור מספיק כדי לאפשר תמלול תקין. גבולות הפרויקט הם שהמערכת אינה מחליפה לחלוטין שיקול דעת אנושי, אלא משמשת כלי תומך החלטה. היא אינה מבצעת זיהוי שקר, אינה מנתחת וידאו, הבעות פנים, שפת גוף או טון דיבור, ואינה משתמשת במידע חיצוני על המועמד מעבר לתשובות שניתנו בתהליך הראיון.

11. תיאור הארכיטקטורה

11.1 הארכיטקטורה של הפתרון המוצע

המערכת בנויה בארכיטקטורת שרת-לקוח ובחלוקה לשכבות. הארכיטקטורה כוללת שכבת תצוגה, שכבת שרת יישום, שכבת מודלי למידת מכונה ושכבת נתונים. שכבת התצוגה היא צד הלקוח, שנבנה באמצעות React. שכבה זו אחראית על ממשקי המשתמש של החברה והמועמד. החברה יכולה להירשם, להתחבר, ליצור משרה, לקבל שאלות מותאמות, לשמור שאלות ולצפות בציוני מועמדים. המועמד יכול להירשם, להתחבר, לצפות במשרות פתוחות, להתחיל ראיון ולענות על שאלות בטקסט או בקול. שכבת שרת היישום נבנתה באמצעות ASP.NET Core. שכבה זו מקבלת את הבקשות מהלקוח, מבצעת בדיקת הרשאות, מנהלת את תהליך הראיון, שומרת ושולפת נתונים ממסד הנתונים, מפעילה את רכיבי ה-Python המתאימים, מעבדת את תוצאותיהם ומחזירה תשובה מסודרת ללקוח. שכבת מודלי למידת המכונה. המודלים פועלים באמצעות שרת Python מקומי המבוסס על FastAPI. שרת זה נטען פעם אחת לזיכרון, טוען את מודלי למידת המכונה, ומאזין לבקשות HTTP מהשרת הראשי ב-#C. כל מודל אחראי למשימה אחרת: בדיקת רלוונטיות, זיהוי סיבת כשל, ניתוב למודלי אבחון, Big Five, איכות חשיבה, יכולות בפועל, ניסיון, התאמת שאלות למשרה והמרת אודיו לטקסט. בנוסף, בחלק מהמקרים קיים מנגנון גיבוי באמצעות PythonRunner, שמאפשר להריץ סקריפט Python כתהליך חיצוני אם שרת המודלים אינו זמין. שכבת הנתונים היא מסד הנתונים PostgreSQL, המאוחסן בענן Neon. שכבה זו שומרת את המידע הקבוע של המערכת: משתמשים, חברות, מועמדים, משרות, שאלות, ראיונות, תשובות, תוצאות רלוונטיות, תוצאות אבחון וציונים סופיים.

11.2 תיאור הרכיבים בפתרון

מערכת כוללת מספר רכיבים מרכזיים הפועלים יחד. רכיב הלקוח הוא ממשק המשתמש. הוא אחראי להצגת המסכים, קבלת קלט מהחברה ומהמועמד, הקלטת אודיו בדפדפן ושליחת בקשות לשרת. רכיב זה אינו מבצע את הניתוח האלגוריתמי בעצמו, אלא מציג את הנתונים והתוצאות שמתקבלות מהשרת. שרת היישום הוא הרכיב המרכזי במערכת. הוא אחראי על ניהול הלוגיקה העסקית, בדיקת הרשאות, יצירת משרות, שמירת שאלות, יצירת ראיונות, שמירת תשובות, הפעלת מודלי Python, חישוב ציון סופי והחזרת תוצאות לממשק המשתמש. מסד הנתונים אחראי לשמירת המידע הקבוע של המערכת.

הוא שומר משתמשים, חברות, מועמדים, משרות, שאלות, ראיונות, תשובות, תוצאות בדיקת רלוונטיות, תוצאות אבחון וציונים סופיים.

רכיבי ה-Python אחראים על עיבוד מתקדם של טקסט ואודיו.

בין הרכיבים קיימים מודול תמלול אודיו לטקסט, מודול התאמת שאלות למשרה, מודול בדיקת רלוונטיות בין שאלה לתשובה, מודול זיהוי סיבת כשל, מודול ניתוב למודלי אבחון, מודלי אבחון עבור ניסיון, יכולות מעשיות ואיכות חשיבה, ומודול לחישוב ציון סופי באמצעות גרף ראיות משוקלל.

רכיבים אלו פועלים דרך שרת Python מקומי המבוסס על FastAPI.

השרת טוען את המודלים לזיכרון פעם אחת בעת ההפעלה, ומקבל מהשרת הראשי בקשות HTTP הכוללות שאלה, תשובה, תיאור משרה או קובץ אודיו.

הפלט מוחזר לשרת הראשי בפורמט JSON.

11.3 תיאור תהליכים של מערכת ההפעלה

בגרסה הראשונית של הפרויקט נעשה שימוש בהרצת תהליכים חיצוניים של מערכת ההפעלה לצורך הפעלת רכיבי Python מתוך שרת היישום.

כאשר השרת נדרש לבצע פעולה אלגוריתמית, כגון תמלול אודיו, בדיקת רלוונטיות, התאמת שאלות או אבחון תשובה, הוא היה יוצר תהליך חדש ומפעיל באמצעותו את קובץ ה-Python המתאים.

אל התהליך הועברו נתוני הקלט הדרושים, לדוגמה תיאור משרה, שאלה, תשובת מועמד או קובץ אודיו.

לאחר סיום העיבוד, רכיב ה-Python החזיר לשרת פלט מובנה בפורמט JSON.

השרת קרא את הפלט, עיבד אותו, שמר את התוצאות במסד הנתונים והחזיר תשובה מסודרת ללקוח.

במהלך הפיתוח שופרה דרך העבודה, והמערכת עברה לשימוש ב-FastAPI Warm Model Server.

בגישה זו שרת Python רץ באופן קבוע, טוען את מודלי למידת המכונה לזיכרון פעם אחת, ומאזין לבקשות מהשרת הראשי ב-C#.

כך אין צורך לפתוח תהליך Python חדש ולטעון מחדש את המודלים בכל בקשה, וזמן התגובה של המערכת מתקצר.

עם זאת, מנגנון הרצת התהליכים החיצוניים עדיין קיים כפתרון גיבוי.

אם שרת המודלים החם אינו זמין או שמתרחשת שגיאה, השרת יכול להפעיל סקריפט Python כתהליך חיצוני באמצעות PythonRunner.

כך נשמרת הפרדה בין שרת היישום לבין רכיבי למידת המכונה, וגם נשמרת יציבות במקרה של תקלה.

בפרויקט לא נעשה שימוש במנגנוני סנכרון מתקדמים כגון סמפורים או מיוטקסים.

עם זאת, כן נעשה שימוש בהרצה מקבילית של מודלי אבחון באמצעות משימות מקביליות ב-C#, כדי להריץ כמה מודלים במקביל ולקצר את זמן העיבוד.

11.4 ארכיטקטורת רשת

המערכת פועלת בארכיטקטורת Client-Server.

צד הלקוח נבנה ב-React ופועל בדפדפן, וצד השרת נבנה ב-ASP.NET Core.

הלקוח שולח בקשות לשרת היישום באמצעות HTTP או HTTPS, והשרת מחזיר תשובות בהתאם לתוצאות העיבוד.

שרת היישום אחראי על ניהול הלוגיקה העסקית, הרשאות המשתמשים, שליפת נתונים ממסד הנתונים, שמירת תוצאות והחזרת מידע לממשק המשתמש.

מסד הנתונים PostgreSQL משמש לשמירת משתמשים, חברות, מועמדים, משרות, ראיונות, תשובות, תוצאות אבחון וציונים סופיים.

בנוסף לתקשורת בין הלקוח לשרת היישום, קיימת תקשורת פנימית בין שרת ה-C# לבין שרת Python מקומי המבוסס על FastAPI.

שרת Python זה פועל ברשת המקומית, ומקבל בקשות מהשרת הראשי לצורך הפעלת מודלי למידת המכונה.

רכיבי למידת המכונה אינם מתקשרים ישירות עם הלקוח ואינם חשופים למשתמשי הקצה.

הלקוח שולח בקשה רק לשרת היישום, ושרת היישום הוא זה שפונה לשרת המודלים, מקבל ממנו פלט בפורמט JSON, מעבד את התוצאה ושומר אותה במסד הנתונים. ארכיטקטורה זו מאפשרת הפרדה ברורה בין ממשק המשתמש, הלוגיקה העסקית, מסד הנתונים ומודלי למידת המכונה. בנוסף, היא מאפשרת לשפר או להחליף מודל Python מסוים מבלי לשנות את כל מערכת השרת או את ממשק המשתמש.

11.5 תיאור API בארכיטקטורה

במערכת זו לא קיימת התממשקות ל-API חיצוני של שירות צד שלישי.

11.6 תיאור פרוטוקולי התקשורת

התקשורת בין הלקוח לשרת היישום מתבצעת באמצעות פרוטוקול HTTP או HTTPS. הלקוח שולח בקשות לשרת, והשרת מחזיר תשובות בפורמט JSON. המערכת משתמשת בבקשות GET לצורך שליפת מידע קיים, כגון טעינת משרות פתוחות, טעינת שאלות ראיון, שליפת ציונים שמורים והצגת נתונים קיימים. המערכת משתמשת בבקשות POST לצורך שליחת מידע חדש לשרת, כגון הרשמה, התחברות, יצירת משרה, שמירת שאלות, התחלת ראיון, שליחת תשובה, תמלול אודיו וחישוב ציון סופי. בבקשות הדורשות הרשאה, הלקוח שולח JWT Token בותרת Authorization. השרת מאמת את הטוקן ומוודא שהמשתמש רשאי לבצע את הפעולה המבוקשת. המידע בין הלקוח לשרת מועבר בדרך כלל בפורמט JSON. במקרה של תמלול אודיו, קובץ ההקלטה נשלח כ-FormData, ולאחר עיבודו השרת מחזיר טקסט מתומלל בפורמט JSON. גם התקשורת הפנימית בין שרת היישום לבין רכיבי Python מבוססת על פלט מובנה בפורמט JSON. השרת מפעיל תהליך Python, מקבל ממנו פלט דרך stdout, מפענח את ה-JSON ומשלב את התוצאה בתהליך העבודה.

11.7 שרת-לקוח

המערכת פועלת בארכיטקטורת שרת-לקוח. הלקוח אחראי להצגת הממשק המשתמש, קבלת קלט מהחברה ומהמועמד, הצגת הודעות טעינה, הצגת שגיאות והצגת תוצאות. הלקוח אינו מבצע את הניתוח האלגוריתמי בעצמו, אלא שולח את הנתונים לשרת. שרת היישום אחראי על הלוגיקה המרכזית של המערכת. הוא מקבל בקשות מהלקוח, בודק הרשאות, שומר ושולף נתונים ממסד הנתונים, מפעיל רכיבי Python, מעבד את תוצאות המודלים ומחזיר תשובות מסודרות ללקוח. זרימת העבודה הכללית היא: לקוח שולח בקשת HTTP לשרת < השרת בודק הרשאות < השרת מבצע עיבוד או שולף נתונים מהמסד < במקרה הצורך השרת מפעיל רכיב Python מתאים < רכיב ה-Python מחזיר JSON < השרת שומר תוצאות במסד הנתונים < השרת מחזיר תשובה ללקוח < הלקוח מציג את התוצאה למשתמש. בנוסף לתקשורת בין הלקוח לשרת, קיימת תקשורת פנימית בין שרת היישום לבין רכיבי Python. תקשורת זו אינה מתבצעת באמצעות Socket או שרת נוסף, אלא באמצעות יצירת תהליך והרצת קובץ Python חיצוני. הנתונים מועברים כקלט לתהליך, והפלט מתקבל דרך stdout בפורמט JSON.

12. תיאור תהליכי אבטחת מידע במערכת

במערכת MindMatch AI קיימים מספר מנגנוני אבטחת מידע שמטרתם להגן על מידע אישי, למנוע גישה לא מורשית לנתונים, ולהבטיח שכל משתמש יוכל לבצע רק פעולות המתאימות לתפקידו במערכת. המערכת שומרת מידע רגיש כגון פרטי חברה, פרטי מועמד, כתובות אימייל, מספרי טלפון, תשובות מועמד, תוצאות אבחון וציוני התאמה. לכן נדרש להגן הן על תהליך ההתחברות וההרשאה והן על המידע הנשמר במסד הנתונים.

12.1 תיאור התקיפה או ההגנה

במערכת קיימת מערכת משתמשים מלאה הכוללת הרשמה והתחברות עבור שני סוגי משתמשים: חברה ומועמד. בעת ההרשמה המשתמש מוסר פרטים אישיים וסיסמה. הסיסמה אינה נשמרת במסד הנתונים כטקסט גלוי, אלא נשמרת כ-Hash מאובטח הכולל Salt. בעת התחברות מוצלחת המשתמש מקבל JWT Token. הטוקן נשלח בכל בקשה רגישה לשרת בכותרת Authorization. השרת בודק את תקינות הטוקן, את תוקפו ואת סוג המשתמש. אם הטוקן חסר, שגוי או פג תוקף, הבקשה נחסמת. המערכת מבצעת הרשאה לפי תפקיד המשתמש. משתמש מסוג Company יכול ליצור משרות, לשמור שאלות, לצפות בראיונות ובציונים השייכים לחברה שלו בלבד. משתמש מסוג Candidate יכול לצפות במשרות פתוחות, להתחיל ראיון ולשלוח תשובות. הפרדה זו מונעת ממועמד לבצע פעולות של חברה, ומונעת מחברה לצפות בנתונים שאינם שייכים לה. בנוסף, המערכת בודקת קשרים בין נתונים רגישים. לדוגמה, כאשר חברה מבקשת לצפות בציון של מועמד, השרת בודק שהראיון שייך למשרה של אותה חברה. כך נמנעת גישה לציונים של חברות אחרות. המערכת מתייחסת למספר מקרי תקיפה אפשריים: ניסיון גישה ל-API ללא התחברות, שימוש בטוקן לא תקין, ניסיון לבצע פעולה שאינה מתאימה לתפקיד המשתמש, ניסיון של חברה לצפות בראיון של חברה אחרת, שליחת נתונים ריקים או לא תקינים, וחשיפה של מידע אישי מתוך מסד הנתונים. בנוסף, קיימת הפרדה בין קוד המערכת לבין סודות מערכת כמו JWT Secret ו-Connection String, כך שסודות אלו אינם אמורים להופיע ישירות בקוד המקור.

12.2 תיאור הצפנות

במערכת קיימים מספר מנגנוני הצפנה והגנה על מידע רגיש. סיסמאות המשתמשים אינן נשמרות במסד הנתונים כטקסט גלוי. בעת ההרשמה נוצרת עבור הסיסמה תבנית Hash מאובטחת הכוללת Salt. בעת התחברות, המערכת אינה משווה את הסיסמה ישירות לטקסט שמור, אלא מחשבת מחדש את ה-Hash ומשווה אותו ל-PasswordHash השמור במסד הנתונים. בנוסף, נתונים אישיים כגון אימייל וטלפון נשמרים בצורה מוצפנת באמצעות מנגנון Data Protection של ASP.NET Core. בצורה זו, גם אם יש גישה ישירה למסד הנתונים, הנתונים האישיים אינם מופיעים כטקסט גלוי. עבור שדות שבהם נדרש חיפוש, כגון אימייל, נשמר גם Hash ייעודי. לדוגמה, EmailHash מאפשר לאתר משתמש לפי אימייל בלי לשמור את כתובת האימייל המקורית בצורה גלויה. לצורך כך נעשה שימוש ב-256HMACSHA. בנוסף, מנגנון JWT משמש לאימות משתמשים לאחר התחברות.

הטוקן כולל מידע על זהות המשתמש ותפקידו, והשרת משתמש בו כדי לוודא שהמשתמש רשאי לבצע את הפעולה המבוקשת.

13. למידת מכונה

13.1 הליך איסוף הנתונים

בפרויקט נעשה שימוש במספר רכיבי למידת מכונה ועיבוד שפה טבעית, ולכן תהליך איסוף הנתונים הותאם לכל משימה בנפרד.

ברכיב **המרת אודיו לטקסט** לא בוצע איסוף נתונים לצורך אימון. נעשה שימוש במודל Whisper קיים, שמטרתו להמיר דיבור באנגלית לטקסט. רכיב זה משמש כרכיב תומך בלבד, המספק למערכת קלט טקסטואלי מתוך תשובה קולית של המועמד. עבור מודול **התאמת שאלות למשרה** נאספו תיאורי משרות באנגלית, שמות תפקידים ומאגר שאלות ראיון כלליות ומקצועיות. מטרת המודול היא לקבל תיאור חופשי של משרה, ולתת לכל שאלה במאגר ציון התאמה למשרה. ובסוף התהליך לבחור את מס' השאלות שהחברה רוצה שיהיו בראיון לפי השאלות עם הציון הגבוה ביותר. עבור כל שאלה הוגדרו מאפיינים שהיא נועדה לבדוק, כגון ניסיון, יכולת חשיבה, יכולות מעשיות, אחריות, בהירות, תקשורת והתמודדות עם בעיות.

ולפי השאלות שנבחרו נדע מהם המאפיינים שחשובים שיהיו במועמד. עבור מודל **בדיקת רלוונטיות תשובה לשאלה** נאספו זוגות של שאלות ראיון ותשובות מועמדים באנגלית. הנתונים כללו שאלות כלליות ומקצועיות, תשובות רלוונטיות העונות על השאלה, ותשובות שאינן רלוונטיות, כגון תשובות כלליות מדי, תשובות חלקיות, תשובות שסוטות מהנושא או תשובות שאינן מתייחסות לשאלה ועוד.

כל דוגמה תוגה באופן בינארי: 1 עבור תשובה רלוונטית ו-0 עבור תשובה לא רלוונטית. עבור מודל **זיהוי סיבת הכשל** נאספו דוגמאות שבהן התשובה אינה עונה על השאלה בצורה מספקת. מטרת הנתונים הייתה לאפשר למערכת לזהות לא רק שהתשובה אינה מתאימה, אלא גם מהי סיבת הכשל. התיגים כללו קטגוריות כגון סטייה מהנושא, מענה חלקי, חוסר ידע, התחמקות מתשובה ועוד. עבור מודל **הניתוב** נאספו תשובות מועמדים המכילות סוגי מידע שונים, ותיוג לפי המודלים שאליהם יש לשלוח את התשובה. התיג הוא רב-תויתי, מכיוון שתשובה אחת יכולה לכלול מידע הרלוונטי ליותר ממודל אחד, לדוגמה גם ניסיון וגם יכולת חשיבה.

עבור מודל **Big Five** נאספו טקסטים באנגלית שניתן להסיק מהם מאפייני אישיות. הנתונים תויגו לפי חמשת ממדי האישיות: Openness, Conscientiousness, Extraversion, Agreeableness ו-Neuroticism. לכל ממד ניתן ציון בטווח 1-5.

עבור מודל **איכות חשיבה** נאספו תשובות מועמדים המבטאות צורות חשיבה שונות. הנתונים תויגו לפי מדדים כגון לוגיקה, בהירות, עומק, מבנה, תהליך חשיבה שלו עצמו ויישום מעשי. לכל מדד ניתן ציון בטווח 1-5.

עבור מודל **יכולות בפועל** נאספו תשובות+ השאלות שעליהן הן נענו, שבהן מועמדים מתארים פעולות שביצעו בפועל, פרויקטים, משימות, פתרון בעיות או תרומה מעשית. הנתונים תויגו לפי שלושה מדדים: Impact, Specificity, Ownership ו-Impact. לכל מדד ניתן ציון בטווח 1-5.

עבור מודל **רמת ניסיון** נאספו תשובות מועמדים המתארות ניסיון קודם, חשיפה מקצועית, מורכבות משימות ורמת אחריות.

הנתונים תויגו לפי שלושה מדדים: exposure, complexity ו-responsibility. לכל מדד ניתן ציון בטווח 1-5.

13.2 טיוב ואיזון נתונים

לפני אימון או הפעלת המודלים בוצעו מספר פעולות טיוב כדי לשפר את איכות הנתונים ואת יציבות התוצאות. ברכיב **המרת אודיו לטקסט** לא בוצע טיוב נתונים לצורך אימון, משום שנעשה שימוש במודל Whisper קיים. עם זאת, בצד המערכת בוצעה התאמה של תהליך העבודה כך שהאודיו מוקלט בדפדפן, נשלח לשרת, מתומלל, ורק לאחר מכן עובר לניתוח טקסטואלי. במודל **התאמת השאלות למשרה** בוצע ניקוי של תיאורי משרה קצרים מדי או כלליים מדי, כדי לוודא שהמערכת מקבלת מספיק מידע לצורך התאמה. בנוסף, בוצעה אחידות בפורמט הקלט כך שכל דוגמה תכלול שם משרה, תיאור משרה ושאלות אפשריות. במאגר **השאלות** השאלות קוטלגו לפי המאפיינים שהן בודקות, כגון ניסיון, יכולת חשיבה, יכולות מעשיות, ותכונות אופי. במודל **בדיקת הרלוונטיות** הוסרו תשובות ריקות, תשובות קצרות מדי או דוגמאות לא תקינות. בנוסף, בוצעה אחידות בפורמט הקלט כך שכל דוגמה תכלול זוג של שאלה ותשובה. בוצעה הסרת כפילויות ואיזון בין תשובות רלוונטיות לבין תשובות לא רלוונטיות, כדי למנוע הטיה של המודל לכיוון מחלקה אחת. במודל **זיהוי סיבת הכשל** בוצע טיוב של רשימת הקטגוריות, כדי לצמצם חפיפה בין סיבות שונות. בנוסף, הוגדרו דוגמאות ותבניות לכל סוג כשל, כדי לסייע למודל לזהות את הסיבה המתאימה. במודל **הניתוב** בוצעה בדיקה של תשובות השייכות ליותר מקטגוריה אחת, מכיון שתשובה יכולה לכלול גם ניסיון וגם יכולת חשיבה או יכולות מעשיות. לכן נעשה שימוש בתיגוב רב-תויתי ולא בתיגוב חד-מחלקתי. בנוסף, נקבע סף החלטה שמטרתו להפחית מצב שבו תשובה לא תישלח למודל רלוונטי. במודל **Big Five** בוצע ניקוי של טקסטים לא תקינים, הסרת כפילויות ואחידות בטווח הציונים 1–5. בנוסף נבדקה התפלגות הציונים כדי לצמצם הטיה חזקה למדד מסוים. במודל **איכות החשיבה** בוצעה חלוקה ברורה של המדדים, בדיקת עקביות בתיגוב בין המדדים, איזון בין רמות הציון השונות והסרת תשובות שאינן מכילות מספיק מידע להערכת איכות החשיבה. במודל **יכולות בפועל** הוסרו תשובות כלליות מדי, וחוזקו דוגמאות הכוללות תיאור פעולה, אחריות ותוצאה. בנוסף, בוצע איזון בין ציונים נמוכים, בינוניים וגבוהים, ונשמר פורמט אחיד של שאלה ותשובה. במודל **רמת ניסיון** בוצע ניקוי של תשובות לא ברורות, איזון בין רמות ניסיון שונות, והפרדה בין ניסיון אמיתי לבין ידע תאורטי בלבד. בנוסף, נבדק שהתשובה כוללת אינדיקציה לחשיפה, מורכבות או אחריות.

13.3 סוג הלמידה הנבחר

בפרויקט נעשה שימוש במספר סוגי למידה, בהתאם למשימה האלגוריתמית. רכיב **המרת אודיו לטקסט** מבוסס על מודל Whisper קיים, ולכן בפרויקט זה לא בוצע אימון מחדש עבורו. מודול **התאמת שאלות למשרה** מבוסס על חישוב התאמה סמנטית ודירוג שאלות לפי מאפייני המשרה, בשילוב כללים למניעת כפילויות. מטרתו לבחור מתוך מאגר שאלות את השאלות המתאימות ביותר לתיאור המשרה. מודל **בדיקת הרלוונטיות** מבוסס על למידה מונחית מסוג סיווג בינארי. הקלט הוא זוג של שאלה ותשובה, והפלט הוא האם התשובה רלוונטית או לא רלוונטית. מודל **זיהוי סיבת הכשל** מבוסס על שילוב של מודל NLI, דמיון סמנטי וכללים לוגיים. מטרתו לסווג את הסיבה לכך שהתשובה אינה עונה על השאלה. מודל **הניתוב** מבוסס על TF-IDF ו-Logistic Regression בלמידה מונחית מסוג סיווג רב-תויתי. מטרתו להחליט לאילו מודלי אבחון יש לשלוח כל תשובה. מודל **Big Five** מבוסס על למידה מונחית מסוג סיווג, הוא אומן בחמישה ראשים נפרדים לכל תכונת אופי. ומחזיר ציונים עבור חמשת ממדי האישיות. מודל **איכות חשיבה** מבוסס על למידה מונחית מסוג Multi-Task Classification, מכיון שהוא מחזיר מספר ציונים עבור מדדים שונים של איכות החשיבה.

מודל יכולות בפועל ומודל רמת ניסיון מבוססים על למידה מונחית, ומחזירים ציונים בטווח מוגדר עבור מספר מדדי אבחון.

הציון הסופי אינו מודל למידה עצמאי, אלא אלגוריתם חישוב מבוסס ראיות. הוא משלב את תוצאות המודלים, משקלל אותן לפי חשיבות מאפייני המשרה, ומתחשב באמינות, בכיסוי הראיות ובדפוסים חוזרים לאורך הראיון.

13.4 נוסחאות ורשתות למידה

בפרויקט נעשה שימוש במספר מודלים ושיטות חישוב, כאשר כל אחת מתאימה לשלב אחר בתהליך הניתוח. במודל התאמת שאלות למשרה, המערכת מחשבת התאמה בין תיאור המשרה לבין שאלות הראיון. עבור כל שאלה q ותיאור משרה j מחושב ציון התאמה:

$$\text{MatchScore}(q,j) = \text{SemanticSimilarity}(q,j) + \text{FeatureOverlap}(q,j)$$

כאשר $\text{SemanticSimilarity}$ מודד את הקרבה הסמנטית בין תיאור המשרה לבין נוסח השאלה, ו- FeatureOverlap מודד את מידת החפיפה בין המאפיינים שהשאלה בודקת לבין המאפיינים הנדרשים במשרה.

לאחר חישוב הציון לכל השאלות, המערכת מדרגת את השאלות ובוחרת את השאלות בעלות הציון הגבוה ביותר, תוך מניעת כפילויות לפי נוסח השאלה.

במודל בדיקת הרלוונטיות, הקלט למודל הוא זוג טקסטים: השאלה ותשובת המועמד.

שני הטקסטים מאוחדים לקלט אחד ומועברים למודל שפה מבוסס Transformer.

המודל מפיק ייצוג מספרי של הקלט, ולאחר מכן שכבת סיווג מחשבת הסתברות לשתי מחלקות: תשובה רלוונטית ותשובה לא רלוונטית.

באופן מתמטי ניתן לתאר את החיזוי כך:

$$P(y | x) = \text{softmax}(W \cdot h + b)$$

כאשר x הוא הקלט הכולל שאלה ותשובה, h הוא הייצוג הווקטורי שהופק על ידי המודל, W היא מטריצת משקלים, b הוא bias, ו- y היא המחלקה החזויה.

המחלקה בעלת ההסתברות הגבוהה ביותר נבחרת כתוצאת הסיווג.

במודל הניתוב נעשה שימוש ב-TF-IDF כדי להמיר את הטקסט לווקטור מספרי.

משקל TF-IDF של מילה מחושב לפי שכיחות המילה במסמך ביחס לשכיחותה בכלל הדאטה:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \cdot \text{IDF}(t)$$

כאשר TF מייצג את שכיחות המונח במסמך, ו-IDF מייצג את נדירות המונח בכלל המסמכים.

לאחר מכן מופעל מודל Logistic Regression, המחזיר הסתברות לכל קטגוריית אבחון.

מכיוון שמדובר בסיווג רב-תויתי, כל קטגוריה נבחנת בנפרד, ואם ההסתברות שלה גבוהה מסף שנקבע מראש, התשובה נשלחת למודל המתאים.

במודלי האבחון מבוססי Transformer, הקלט הוא טקסט המורכב מהשאלה ותשובת המועמד.

המודל מפיק ייצוג סמנטי של הטקסט, ועל גביו מופעלות שכבות פלט שונות בהתאם למשימה.

במודל איכות החשיבה, לדוגמה, קיימים מספר פלטים עבור מדדים שונים כגון לוגיקה, בהירות, עומק, מבנה ורפלקציה.

כל פלט מחזיר ציון עבור הממד הרלוונטי.

ניתן לתאר חישוב ציון עבור מדד מסוים כך:

$$\text{score}_i = f_i(h)$$

כאשר h הוא ייצוג הטקסט שהופק על ידי המודל, ו- f_i היא שכבת הפלט של הממד ה- i .

לאחר חישוב ציוני המדדים, ניתן לחשב ציון כולל באמצעות ממוצע:

$$\text{overall_score} = (\text{score}_1 + \text{score}_2 + \dots + \text{score}_n) / n$$

במודל דמיון סמנטי, כאשר נדרש להשוות בין שני טקסטים, כל טקסט מומר לווקטור, ולאחר מכן מחושב דמיון קוסינוס:

$$\text{cosine_similarity}(A,B) = (A \cdot B) / (||A|| \cdot ||B||)$$

כאשר A הוא וקטור השאלה ו- B הוא וקטור התשובה.

ערך גבוה יותר מצביע על קרבה סמנטית גבוהה יותר בין השאלה לתשובה.

במודל NLI, המערכת בוחנת את הקשר בין שני טקסטים ומחזירה הסתברות לקטגוריות כגון נביעה, סתירה או ניטרלי.

בפרויקט, גישה זו משמשת כדי לסייע בזיהוי סיבת הכשל כאשר תשובה אינה עונה על השאלה.

בשלב חישוב הציון הסופי, המערכת משתמשת בגרף ראיות משוקלל.

לכל מאפיין במשרה מוגדר משקל חשיבות, ולכל מאפיין מחושב ציון על בסיס הראיות שנאספו מתשובות המועמד.

הציון המשוקלל הראשוני מחושב כך:

$$\text{WeightedFeatureScore} = \sum(\text{weight}_i \cdot \text{score}_i)$$

כאשר weight_i הוא משקל החשיבות של המאפיין ה- i , ו- score_i הוא הציון שהתקבל עבור אותו מאפיין. לאחר מכן המערכת משלבת גורמים נוספים כגון אמינות התשובות, כיסוי הראיות, התנהגות רלוונטיות ודפוסים חוזרים.

הציון הסופי מחושב באופן כללי כך:

$$\text{FinalScore} = \text{WeightedFeatureScore} + \text{ReliabilityAdjustment} + \text{CoverageAdjustment} + \text{RelevanceAdjustment} + \text{PatternAdjustment}$$

עד כמה ניתן לסמוך על הציון שחושב $\text{ReliabilityAdjustment}$

האם הראיון סיפק מספיק מידע על כל התחומים החשובים למשרה $\text{CoverageAdjustment}$

אם רוב התשובות של המועמד היו רלוונטיות לשאלות זה מחזק את הציון $\text{RelevanceAdjustment}$

זיהוי התנהגויות שחוזרות על עצמן לפי מהלך הראיון PatternAdjustment

בפועל, התאמות אלו מאפשרות למערכת לא רק לחשב ממוצע ציונים, אלא להתחשב באיכות הראיות,

במספר התשובות שתומכות בכל מאפיין, ובחולשות שחוזרות על עצמן לאורך הראיון.

13.5 כיוונון, ייעול וניטור מדדי ביצוע

במהלך פיתוח המודלים בוצעו מספר פעולות לשיפור איכות החיזוי, יציבות התוצאות ודיוק המערכת.

תחילה חולק סט הנתונים לנתוני אימון ונתוני בדיקה, כדי לאפשר למודל ללמוד על חלק מהנתונים ולבדוק את ביצועיו על דוגמאות שלא נחשף אליהן בזמן האימון.

במהלך האימון והבדיקות נבחנו מספר פרמטרים, כגון גודל אצווה, קצב למידה, אורך טקסט מקסימלי וסף החלטה.

מטרת הכיוונון הייתה למצוא איזון בין דיוק גבוה לבין מניעת התאמת יתר לנתוני האימון.

לאחר כל הרצה נבדקו תוצאות המודל, נבחנו דוגמאות שבהן המודל טעה, ובוצעו שיפורים בהתאם לשגיאות שהתגלו.

תקינות המודלים נבדקה באמצעות השוואה בין הפלט שהמודל החזיר לבין התייגים האמיתיים בסט הבדיקה. בנוסף, בוצעה בדיקה ידנית של דוגמאות שונות כדי לוודא שהמודלים אינם מחזירים תוצאות מקריות או לא הגיוניות, ושהם מתמודדים גם עם ניסוחים שונים של אותה כוונה.

במערכת קיימים מספר מודלים ורכיבי ניתוח, וכל אחד מהם נבדק לפי מטרתו.

מודל התאמת השאלות למשרה נבדק לפי היכולת לבחור שאלות רלוונטיות לתיאור המשרה.

מודל הרלוונטיות נבדק לפי היכולת לזהות האם תשובת מועמד עונה על השאלה.

מודל סיבת הכשל נבדק לפי היכולת להסביר מדוע תשובה אינה מספקת.

מודל הניתוב נבדק לפי היכולת לבחור אילו מודלי אבחון יש להפעיל על תשובה.

מודלי האבחון נבדקו לפי היכולת להחזיר ציונים המתאימים לקריטריונים שהוגדרו מראש.

לאחר בדיקת המודלים בנפרד, הם שולבו במערכת ונבדקו כחלק מתהליך מלא: יצירת משרה, התאמת

שאלות, שמירת שאלות, התחלת ראיון על ידי מועמד, שליחת תשובות, בדיקת רלוונטיות, הפעלת מודלי

אבחון, חישוב ציון סופי והצגת התוצאה לחברה.

בדיקה זו נועדה לוודא שהמודלים אינם עובדים רק בנפרד, אלא משתלבים בצורה תקינה בתהליך העבודה המלא של המערכת.

אחוזי הדיוק שהתקבלו במודלים:

מודל התאמת שאלות למשרה: 71.43%

מודל רלוונטיות: לפי הדוגמאות שנבדקו (50) 100%

מודל סיבת הכשל: 64%
 מודל ניתוב: 70%
 מודל Big Five / מאפייני אישיות: 90.12%
 מודל ניסיון: 84%
 מודל יכולות מעשיות: 75%
 מודל איכות חשיבה: 57% הסטייה הממוצעת (MAE) היא 0.43
 בנוסף לאחוזי הדיוק של המודלים, נבדקה גם הצלחת המערכת כולה בביצוע זרימה מלאה מקצה לקצה.
 כלומר, נבדק שהמערכת מצליחה ליצור משרה, להתאים שאלות, לאפשר למועמד לענות, לשמור את
 הנתונים, להפעיל את המודלים ולחשב ציון סופי בצורה תקינה.

14. תיאור התוכנה

14.1 פירוט API

ה-API המרכזי במערכת הוא API פנימי שפותח ב-ASP.NET Core, והוא משמש לתקשורת בין צד הלקוח שנבנה ב-React לבין צד השרת.

שרת ASP.NET Core

1. Auth API

רכיב זה אחראי על הרשמה, התחברות וזיהוי המשתמש המחובר.
הפונקציות המרכזיות:

POST /api/auth/company/register

הרשמת חברה חדשה למערכת, יצירת משתמש מסוג Company, הצפנת מידע רגיש, שמירת סיסמה כ-Hash והחזרת JWT Token.

POST /api/auth/candidate/register

הרשמת מועמד חדש למערכת, יצירת משתמש מסוג Candidate, שמירת פרטי מועמד, הצפנת מידע רגיש, שמירת סיסמה כ-Hash והחזרת JWT Token.

POST /api/auth/login

התחברות משתמש קיים למערכת. הפעולה בודקת את פרטי ההתחברות ומחזירה JWT Token.

GET /api/auth/me

שליפת פרטי המשתמש המחובר לפי הטוקן, כגון סוג המשתמש והמזהים הרלוונטיים שלו.

2. Jobs API

רכיב זה אחראי על יצירת משרות, שליפת משרות פתוחות, הצטרפות מועמד למשרה והתאמת שאלות למשרה.
הפונקציות המרכזיות:

POST /api/jobs/open

יצירת משרה פתוחה על ידי חברה.

GET /api/jobs/open

שליפת רשימת משרות פתוחות עבור מועמד.

POST /api/jobs/{jobId}/join

הצטרפות מועמד למשרה פתוחה ויצירת ראיון חדש עבור המועמד והמשרה.

GET /api/jobs/{jobId}/suggest-questions

קבלת שאלות מומלצות למשרה באמצעות שירות התאמת שאלות.

POST /api/jobs/{jobId}/questions

הוספת שאלה ידנית למשרה.

POST /api/jobs/{jobId}/questions/from-bank

הוספת שאלה מתוך מאגר השאלות למשרה.

3. Interviews API

רכיב זה אחראי על ניהול תהליך הראיון, שליפת שאלות, שליחת תשובות, הוספת שאלות לראיון וסיום הראיון. הפונקציות המרכזיות:

GET /api/interviews/{interviewId}/questions

שליפת שאלות הראיון עבור המועמד.

POST /api/interviews/{interviewId}/questions

הוספת שאלה ידנית לראיון קיים.

POST /api/interviews/{interviewId}/questions/from-bank

הוספת שאלה מתוך מאגר השאלות לראיון קיים.

POST /api/interviews/{interviewId}/answers

שליחת תשובת מועמד לשאלה. הפעולה שומרת את התשובה, בודקת רלוונטיות, מחזירה משוב במקרה הצורך, ומפעילה מודלי אבחון כאשר התשובה רלוונטית.

POST /api/interviews/{interviewId}/finish

סיום ראיון וחישוב ציון סופי באמצעות גרף ראיות משוקלל.

GET /api/interviews/{interviewId}/score

שליפת הציון הסופי של ראיון מסוים.

4. Scores and Ranking API

רכיב זה אחראי על שליפת ציונים ודירוג מועמדים עבור חברה. הפונקציות המרכזיות:

GET /api/company/scores

שליפת ציונים שמורים של החברה המחוברת.

GET /api/company/jobs/{jobId}/ranking

שליפת דירוג מועמדים עבור משרה מסוימת לפי הציון הסופי.

5. Speech API

רכיב זה אחראי על תמלול תשובות קוליות.

הפונקציות המרכזיות:

POST /api/speech/transcribe
קבלת קובץ אודיו מהלקוח, שליחתו לרכיב התמלול והחזרת הטקסט המתומלל ללקוח.

6. Health API
רכיב זה אחראי על בדיקת תקינות בסיסית של המערכת.
הפונקציות המרכזיות:

GET /api/health
בדיקת תקינות בסיסית של השרת ושל החיבור למסד הנתונים.

שרת מודלים פנימי Python Model Server

בנוסף ל-API מול צד הלקוח, שרת ה-C# מתממשק לשרת מודלים פנימי שנכתב ב-Python. שרת זה אינו API חיצוני של צד שלישי, אלא רכיב פנימי במערכת, והוא משמש להרצת מודלי למידת המכונה. הפונקציות המרכזיות:

POST /relevance
בדיקת רלוונטיות של תשובת מועמד ביחס לשאלה.

POST /diagnosis
אבחון הסיבה לכך שתשובה אינה מספקת והחזרת משוב למועמד.

POST /routing
ניתוב תשובה למודלי האבחון המתאימים.

POST /diagnostic
הרצת מודל אבחון מסוים לפי סוג האבחון הנדרש.

POST /job-question-match
התאמת שאלות מתוך מאגר השאלות לתיאור משרה.

POST /transcribe-file
תמלול קובץ אודיו לטקסט.

POST /warmup
חימום וטעינה מוקדמת של המודלים.

GET /health
בדיקת תקינות שרת המודלים.

14.2 סביבת עבודה

המערכת בנויה ממספר רכיבים, כאשר לכל רכיב קיימת סביבת עבודה משלו.
רכיב הלקוח פותח בסביבת React.
רכיב זה אחראי להצגת ממשק המשתמש לחברה ולמועמד, שליחת בקשות לשרת, הצגת הודעות טעינה, הצגת שגיאות והצגת תוצאות.
בנוסף, רכיב הלקוח אחראי על הקלטת אודיו בדפדפן ושליחת קובץ ההקלטה לשרת.

רכיב ה-API פותח בסביבת .NET / ASP.NET Core. רכיב זה אחראי לקבלת בקשות HTTP, בדיקת הרשאות, הפעלת שירותי המערכת, שמירת נתונים במסד הנתונים, הפעלת רכיבי Python והחזרת תוצאות למשתמש. רכיב מסד הנתונים מבוסס PostgreSQL בענן Neon. במסד הנתונים נשמרים נתונים קבועים כגון משתמשים, חברות, מועמדים, משרות, שאלות שנבחרו, ראיונות, תשובות, תוצאות רלוונטיות, החלטות ניתוב, תוצאות אבחון וציונים סופיים. רכיב המודלים פועל בסביבת Python. המודלים נשמרים כסקריפטים נפרדים ומופעלים מתוך מערכת ה-API לפי הצורך. כל מודל אחראי על פעולה מסוימת, לדוגמה התאמת שאלות למשרה, בדיקת רלוונטיות, אבחון סיבת תשובה לא רלוונטית, ניתוב למודלי אבחון, תמלול אודיו ומתן ציוני אבחון. רכיב האבטחה נמצא בצד השרת. הוא כולל אימות באמצעות JWT Token, הרשאות לפי סוג משתמש, הצפנת מידע אישי, שמירת Hash לסיסמאות ושדות חיפוש, והגבלת גישה לנתונים לפי החברה או המועמד המחובר. סביבת הפיתוח כללה עבודה עם Visual Studio / Rider או סביבת פיתוח מתאימה ל-ASP.NET Core, סביבת React עבור צד הלקוח, סביבת Python עבור המודלים, ו-DBaver לצפייה ובדיקה של מסד הנתונים PostgreSQL.

14.3 שפות תכנות

המערכת פותחה באמצעות מספר שפות וטכנולוגיות, בהתאם לתפקיד של כל רכיב בארכיטקטורה. רכיב הלקוח נכתב ב-JavaScript באמצעות React. רכיב זה כולל את מסכי המשתמש, ניהול מצב הלקוח, שליחת בקשות ל-API, הצגת שאלות, שליחת תשובות, הקלטת אודיו והצגת הציונים. עיצוב ממשק המשתמש נכתב באמצעות CSS. קובצי העיצוב אחראים על מבנה המסכים, כפתורים, כרטיסים, הודעות טעינה, מסך הציון הסופי וחווית המשתמש הכללית. רכיב ה-API נכתב בשפת #C באמצעות ASP.NET Core. רכיב זה כולל את נקודות הקצה של המערכת, שירותי הראיון, בדיקת הרשאות, חישוב הציונים, עבודה מול מסד הנתונים והפעלת מודלי Python. רכיב המודלים נכתב בשפת Python. בשפה זו פותחו והורצו רכיבי למידת המכונה ועיבוד השפה הטבעית, ביניהם מודול התאמת שאלות למשרה, מודל בדיקת רלוונטיות, מודל Diagnosis להסבר תשובה לא רלוונטית, מודל ניתוב, מודלי אבחון ותמלול אודיו. מסד הנתונים מבוסס PostgreSQL, ולכן העבודה מול הנתונים מתבצעת באמצעות SQL. בפועל, רוב הגישה למסד נעשית מתוך #C באמצעות Entity Framework Core, אך הנתונים נשמרים בטבלאות PostgreSQL וניתן לבדוק אותם גם באמצעות שאלות SQL. פורמט JSON משמש להעברת מידע בין רכיבים שונים במערכת. לדוגמה, הלקוח והשרת מעבירים נתונים בפורמט JSON, ורכיבי Python מחזירים תוצאות בפורמט JSON לשרת. PowerShell שימש להפעלת המערכת, בדיקות API, הרצת פקודות build ושליחת בקשות בדיקה לשרת המקומי.

15. ניתוח ותרשים Use cases/UML של המערכת המוצעת.

15.1 תיאור ה- UC העיקריים של המערכת

המערכת נועדה לאפשר לחברה לבצע תהליך מיון ראשוני למועמדים באמצעות ראיון ממוחשב חכם. התהליך המרכזי מתחיל בכך שחברה יוצרת משרה פתוחה ומזינה תיאור חופשי של התפקיד. לאחר מכן המערכת מתאימה שאלות ראיון למשרה, החברה שומרת את השאלות, והמועמד יכול לבחור את המשרה ולהתחיל ראיון אישי.

ה- Use Cases העיקריים במערכת הם:

הרשמת חברה והתחברות:

החברה נרשמת למערכת באמצעות פרטי חברה, אימייל וסיסמה. לאחר התחברות החברה מקבלת הרשאות המאפשרות לה ליצור משרות, לשמור שאלות ולצפות בציוני מועמדים השייכים אליה.

הרשמת מועמד והתחברות:

המועמד נרשם למערכת באמצעות שם, אימייל, טלפון וסיסמה. לאחר התחברות הוא יכול לצפות במשרות פתוחות, לבחור משרה ולהתחיל ראיון.

יצירת משרה פתוחה:

החברה מזינה שם משרה ותיאור חופשי של דרישות התפקיד. המערכת שומרת את המשרה במסד הנתונים ומשייכת אותה לחברה המחוברת.

התאמת שאלות למשרה:

המערכת מקבלת את תיאור המשרה ומספר שאלות רצוי, מנתחת את דרישות התפקיד, ובוחרת שאלות רלוונטיות מתוך מאגר השאלות.

השאלות נבחרות לפי התאמתן למאפיינים הנדרשים במשרה, כגון ניסיון, יכולת חשיבה, יכולות מעשיות, אחריות ותקשורת.

שמירת שאלות למשרה:

החברה בוחרת את השאלות המתאימות ושומרת אותן למשרה. המערכת מונעת שמירת שאלות כפולות ושומרת את סדר השאלות עבור הראיון.

הצגת משרות פתוחות למועמד:

המועמד יכול לצפות במשרות פתוחות במערכת. עבור כל משרה מוצגים שם המשרה ותיאור התפקיד.

הצטרפות מועמד למשרה והתחלת ראיון:

כאשר מועמד בוחר משרה, המערכת יוצרת עבורו ראיון חדש המשוך למשרה ולמועמד, ושולפת את השאלות שנשמרו עבור אותה משרה.

שליחת תשובת מועמד:

המועמד עונה על שאלות הראיון בטקסט או בקול. אם הוא עונה בקול, המערכת מתמללת את האודיו לטקסט. לאחר שליחת התשובה, המערכת שומרת אותה ובודקת את מידת הרלוונטיות שלה לשאלה.

טיפול בתשובה לא רלוונטית:

אם תשובת המועמד אינה עונה על השאלה בצורה מספקת, המערכת מפעילה מנגנון אבחון, מחזירה למועמד הערה לשיפור ומאפשרת ניסיון נוסף.

ניתוח תשובה רלוונטית:

כאשר תשובה נמצאת רלוונטית, המערכת מפעילה מודלי אבחון שונים הבודקים מאפיינים כגון ניסיון, איכות חשיבה ויכולות מעשיות. תוצאות האבחון נשמרות במסד הנתונים.

חישוב ציון סופי:

בסיום הראיון, המערכת אוספת את כל התשובות, תוצאות הרלוונטיות ותוצאות האבחון, ובונה גרף ראיות משוקלל.

על בסיסו מחושב ציון התאמה סופי הכולל גם הסבר מילולי, חוזקות, חולשות ודפוסים מרכזיים.

צפייה בציוני מועמדים:

החברה יכולה לצפות בציונים הסופיים של ראיונות השייכים למשרות שלה בלבד.

המערכת מציגה את הציון, שם המשרה, מזהה המועמד וסיכום ההתאמה.

15.2 הצגת מקרה (case use) עבור כל הפונקציות העיקריות בפרויקט.

1UC- הרשמת חברה

שם הפעולה: הרשמת חברה.

משתמש ראשי: חברה.

תנאים מוקדמים: השרת פעיל וקיים חיבור למסד הנתונים.

קלט: שם חברה, שם איש קשר, אימייל וסיסמה.

תהליך תקין: החברה מזינה את הפרטים בטופס ההרשמה. המערכת בודקת שהאימייל אינו קיים, מצפינה

מידע רגיש, שומרת את הסיסמה כ-Hash מאובטח, יוצרת רשומת חברה ומשתמש מסוג Company, ומחזירה JWT Token.

פלט: משתמש חברה מחובר, מזהה חברה וטוקן הרשאה.

מקרי שגיאה: אימייל שכבר קיים, שדות חסרים, סיסמה לא תקינה או תקלה במסד הנתונים.

2UC- הרשמת מועמד

שם הפעולה: הרשמת מועמד.

משתמש ראשי: מועמד.

תנאים מוקדמים: השרת פעיל וקיים חיבור למסד הנתונים.

קלט: שם מלא, אימייל, טלפון וסיסמה.

תהליך תקין: המועמד מזין את פרטיו. המערכת בודקת תקינות, שומרת מידע רגיש בצורה מוצפנת, שומרת

סיסמה כ-Hash, יוצרת רשומת מועמד ומשתמש מסוג Candidate, ומחזירה JWT Token.

פלט: משתמש מועמד מחובר, מזהה מועמד וטוקן הרשאה.

מקרי שגיאה: אימייל קיים, נתונים חסרים, סיסמה לא תקינה או תקלה במסד הנתונים.

3UC- יצירת משרה פתוחה

שם הפעולה: יצירת משרה פתוחה.

משתמש ראשי: חברה.

תנאים מוקדמים: החברה מחוברת למערכת עם JWT Token תקין.

קלט: שם משרה ותיאור משרה.

תהליך תקין: החברה מזינה את פרטי המשרה.

המערכת יוצרת משרה חדשה, משייכת אותה לחברה המחוברת ושומרת אותה במסד הנתונים.

פלט: מזהה משרה, שם משרה ותיאור משרה.

מקרי שגיאה: משתמש לא מחובר, משתמש שאינו חברה, תיאור קצר מדי, שם משרה חסר או תקלה במסד

הנתונים.

4UC- התאמת שאלות למשרה

שם הפעולה: קבלת שאלות מתאימות למשרה.

משתמש ראשי: חברה.

תנאים מוקדמים: קיימת משרה השייכת לחברה המחוברת וקיים מאגר שאלות פעיל.

קלט: מזהה משרה ומספר שאלות רצוי.

תהליך תקין: המערכת שולפת את תיאור המשרה, מנתחת את הדרישות המרכזיות, משווה אותן למאפייני

השאלות במאגר, מדרגת שאלות לפי התאמה ומחזירה רשימת שאלות ללא כפילויות.

פלט: רשימת שאלות מומלצות למשרה.

מקרי שגיאה: משרה לא קיימת, אין הרשאה למשרה, אין שאלות במאגר, מספר שאלות לא תקין או כשל

במודל התאמת השאלות.

5UC- שמירת שאלות למשרה

שם הפעולה: שמירת שאלות למשרה.
 משתמש ראשי: חברה.
 תנאים מוקדמים: קיימת משרה, והמערכת הציגה שאלות מתאימות או שהחברה הוסיפה שאלה ידנית.
 קלט: שאלות מתוך המאגר או שאלות שהחברה הוסיפה ידנית.
 תהליך תקין: המערכת בודקת שהשאלות אינן כפולות, שומרת אותן תחת המשרה לפי סדר הופעתן, ומאפשרת למועמדים לקבל אותן בראיון.
 פלט: רשימת שאלות שמורות למשרה.
 מקרי שגיאה: ניסיון לשמור ללא משרה, שאלות כפולות, שאלה ריקה או תקלה במסד הנתונים.

6UC- הצגת משרות פתוחות

שם הפעולה: הצגת משרות פתוחות.
 משתמש ראשי: מועמד.
 תנאים מוקדמים: המועמד מחובר למערכת.
 קלט: בקשה לטעינת משרות פתוחות.
 תהליך תקין: המערכת שולפת משרות פתוחות ממסד הנתונים ומציגה אותן למועמד.
 פלט: רשימת משרות הכוללת שם משרה, תיאור ומזהה משרה.
 מקרי שגיאה: משתמש לא מחובר, משתמש שאינו מועמד או תקלה בשליפת הנתונים.

7UC- הצטרפות למשרה והתחלת ראיון

שם הפעולה: התחלת ראיון.
 משתמש ראשי: מועמד.
 תנאים מוקדמים: קיימת משרה פתוחה עם שאלות שמורות.
 קלט: מזהה משרה.
 תהליך תקין: המועמד בוחר משרה. המערכת יוצרת ראיון חדש, משייכת אותו למועמד ולמשרה, ושולפת את השאלות השמורות למשרה.
 פלט: מזהה ראיון ורשימת שאלות.
 מקרי שגיאה: משרה לא קיימת, אין שאלות למשרה, מועמד כבר התחיל או סיים ראיון למשרה זו, או תקלה במסד הנתונים.

8UC- שליחת תשובת מועמד

שם הפעולה: שליחת תשובה לשאלה.
 משתמש ראשי: מועמד.
 תנאים מוקדמים: קיים ראיון פעיל וקיימת שאלה פעילה.
 קלט: מזהה ראיון, מזהה שאלה, נוסח השאלה, טקסט התשובה, מספר ניסיון ומזהה תשובה קודמת אם קיימת.
 תהליך תקין: המערכת שומרת את התשובה, מפעילה מודל בדיקת רלוונטיות, ושומרת את תוצאת הרלוונטיות. אם התשובה רלוונטית, המערכת ממשיכה לניתוב ולאבחון.
 אם אינה רלוונטית, מוחזרת הערה למועמד.
 פלט: מזהה תשובה, סטטוס רלוונטיות, הודעת הכוונה במקרה הצורך.
 מקרי שגיאה: תשובה ריקה, שאלה לא קיימת, ראיון לא קיים, משתמש לא מורשה או כשל במודל הרלוונטיות.

9UC- תמלול תשובה קולית

שם הפעולה: תמלול אודיו לטקסט.
 משתמש ראשי: מועמד.
 תנאים מוקדמים: הדפדפן קיבל הרשאה למיקרופון והמשתמש מחובר.
 קלט: קובץ אודיו שהוקלט בדפדפן.
 תהליך תקין: הדפדפן שולח את קובץ האודיו לשרת.
 השרת שומר אותו זמנית, מפעיל סקריפט Python לתמלול, מקבל טקסט מתומלל ומחזיר אותו ללקוח.
 פלט: טקסט מתומלל שמוכנס לתיבת התשובה.
 מקרי שגיאה: אין הרשאת מיקרופון, קובץ אודיו לא תקין, כשל בתמלול או תקלה בשרת.

10UC- אבחון תשובה רלוונטית

שם הפעולה: אבחון תשובת מועמד.

משתמש ראשי: מערכת.

תנאים מוקדמים: התשובה סווגה כרלוונטית.

קלט: שאלה ותשובת מועמד.

תהליך תקין: המערכת מפעילה מנגנון ניתוב כדי לקבוע אילו מודלי אבחון רלוונטיים לתשובה.

לאחר מכן היא מפעילה את המודלים המתאימים, כגון ניסיון, יכולות מעשיות ואיכות חשיבה, ושומרת את

התוצאות במסד הנתונים.

פלט: תוצאות אבחון וציונים לפי מדדים.

מקרי שגיאה: כשל במודל הניתוב, כשל במודל אבחון או פלט לא תקין מהמודלים.

11UC- חישוב ציון סופי

שם הפעולה: חישוב ציון התאמה סופי.

משתמש ראשי: חברה.

תנאים מוקדמים: קיים ראיון עם תשובות ותוצאות אבחון.

קלט: מזהה ראיון.

תהליך תקין: המערכת שולפת את תשובות הראיון, תוצאות הרלוונטיות ותוצאות האבחון.

לאחר מכן היא בונה גרף ראיות משוקלל, מחשבת ציון לפי חשיבות מאפייני המשרה, מזהה חוזקות וחולשות,

ושומרת ציון סופי וסיכום מילולי.

פלט: ציון סופי, סיכום, פירוט מאפיינים, RawJson וגרסת מודל.

מקרי שגיאה: ראיון לא נמצא, אין הרשאה לראיון, אין תשובות או כשל בחישוב הציון.

12UC- צפייה בציוני מועמדים

שם הפעולה: צפייה בציונים שמורים.

משתמש ראשי: חברה.

תנאים מוקדמים: החברה מחוברת וקיימים ציונים שמורים עבור משרות שלה.

קלט: בקשה לטעינת ציונים.

תהליך תקין: המערכת שולפת את הציונים של ראיונות השייכים למשרות של החברה המחוברת בלבד,

ומציגה אותם ברשימה.

החברה יכולה לבחור ציון ולצפות בפירוט שלו.

פלט: רשימת ציונים, שם משרה, מזהה מועמד, ציון סופי וסיכום.

מקרי שגיאה: אין ציונים שמורים, אין הרשאה, או תקלה בשליפה מהמסד.

15.3 מבנה נתונים בהם השתמש

במערכת נעשה שימוש במספר מבני נתונים לניהול מידע בזמן ריצה ובמהלך עיבוד הנתונים.

List:

נעשה שימוש ב-List לשמירת אוספים שיש חשיבות לסדר שלהם, לדוגמה רשימת שאלות בראיון, רשימת

תשובות, רשימת ציוני אבחון ורשימת ציונים שמורים.

הבחירה ב-List מתאימה משום שבמהלך ראיון יש משמעות לסדר השאלות והתשובות.

יתרונות המבנה הם פשטות, שמירה על סדר ונוחות במעבר סדרתי.

החיסרון הוא שחיפוש לפי מזהה דורש מעבר על הרשימה ועלול להיות $O(n)$.

מבנה זה נבחר משום שבחלקים רבים במערכת נדרש להציג או לעבד מידע לפי סדר.

Dictionary / Hash Map:

נעשה שימוש ב-Dictionary כאשר נדרשת שליפה מהירה לפי מפתח, לדוגמה שמירת ניסיונות לפי מזהה

שאלה, שמירת תשובה קודמת לפי מזהה שאלה, או מיפוי מאפיינים לציונים.

מבנה זה מאפשר גישה ממוצעת של $O(1)$ לפי מפתח.

החיסרון הוא צריכת זיכרון גבוהה יותר וחוסר שמירה על סדר.

הוא נבחר משום שבמערכת יש צורך לגשת במהירות לנתונים לפי מזהים כגון questionId או

featureName.

HashSet:

נעשה שימוש ב-HashSet במקומות שבהם חשוב למנוע כפילויות, לדוגמה מניעת שאלות כפולות לפי נוסח השאלה או בדיקת קיום של מזהים שכבר עובדו. יתרון של HashSet הוא בדיקת קיום מהירה ומניעת כפילויות. חסרונו הוא שאינו שומר על סדר. הוא נבחר משום שמניעת כפילויות היא דרישה מרכזית במערכת, במיוחד בשלב התאמת ושמירת שאלות למשרה.

JSON Object:

נעשה שימוש באובייקטי JSON להעברת מידע בין הלקוח לשרת, וכן להעברת פלט בין רכיבי Python לבין שרת ה-C#.

בנוסף, תוצאות גולמיות כמו RawJson של חישוב הציון הסופי נשמרות במסד הנתונים. היתרון הוא שמדובר בפורמט גמיש, קריא ומתאים להעברת מידע מובנה בין רכיבים שונים. החיסרון הוא שיש צורך לוודא תקינות מבנה לפני שימוש בנתונים.

טבלאות במסד הנתונים:

המידע הקבוע של המערכת נשמר בטבלאות PostgreSQL. לדוגמה, משתמשים, חברות, מועמדים, משרות, שאלות, ראיונות, תשובות וציונים סופיים. מבנה זה נבחר משום שהמערכת צריכה לשמור מידע לטווח ארוך, גם לאחר רענון הדפדפן או יציאה מהמערכת.

היתרון הוא עקביות ושמירה קבועה של נתונים.

החיסרון הוא שכל פעולה דורשת גישה למסד הנתונים ולכן איטית יותר מגישה לזיכרון.

Enum:

נעשה שימוש בערכים קבועים לייצוג סוג משתמש, סטטוס ראיון, סטטוס רלוונטיות או סוגי מודלים. היתרון הוא מניעת טעויות כתיב ושיפור קריאות הקוד. החיסרון הוא שיש לעדכן את הקוד כאשר מוסיפים ערך חדש. שילוב של List + Dictionary:

נבחר מפני שהוא מאפשר למערכת ליהנות משני יתרונות מרכזיים:

מצד אחד, List מאפשרת לשמור את הנתונים לפי סדר קבוע ונוח לעיבוד,

ומצד שני, Dictionary מאפשרת שליפה מהירה של נתון מסוים לפי מזהה.

עם זאת, יש לקחת בחשבון ששימוש בשני המבנים יחד עלול לגרום לשימוש כפול בזיכרון,

במיוחד אם אותו מידע נשמר גם ברשימה וגם במפוי.

במערכת הנוכחית הדבר אינו מהווה בעיה משמעותית, אך במקרה של גדילה בכמות הראיונות הפעילים או הנתונים הזמניים, ייתכן שיהיה צורך לנהל את הזיכרון בצורה יעילה יותר.

בעתיד ניתן יהיה לשפר את המימוש באמצעות שימוש במנגנון Cache (מטמון) מתקדם, שאילתות יעילות יותר למסד הנתונים, או מנגנון ניקוי אוטומטי לנתונים זמניים שאינם נדרשים עוד.

15.4 חישוב יעילות האלגוריתם

חישוב היעילות מתייחס לפעולות המרכזיות במערכת, כאשר n מייצג את מספר השאלות במאגר, q מייצג את מספר השאלות בראיון, a מייצג את מספר התשובות בראיון, d מייצג את מספר תוצאות האבחון, m מייצג את אורך הטקסט, ו- k מייצג את מספר מודלי האבחון שמופעלים.

הרשמת משתמש:

המערכת יוצרת רשומת משתמש ורשומת חברה או מועמד.

מבחינת האלגוריתם, הפעולה כוללת מספר קבוע של בדיקות ושמירות.

סיבוכיות זמן: $O(1)$, לא כולל זמן גישה למסד הנתונים.

סיבוכיות מקום: $O(1)$.

יצירת משרה:

הפעולה יוצרת רשומת משרה אחת ומשייכת אותה לחברה המחוברת.

סיבוכיות זמן: $O(1)$, לא כולל זמן DB.

סיבוכיות מקום: $O(1)$.

התאמת שאלות למשרה:

המערכת עוברת על מאגר השאלות הפעילות, משווה כל שאלה לתיאור המשרה ומדרגת את השאלות לפי התאמה.

אם קיימות n שאלות במאגר, מעבר על השאלות ובניית רשימת מועמדות הם $O(n)$.

אם מתבצע מיון מלא לפי ציון התאמה, סיבוכיות המיון היא $O(\log \log n)$.

לכן הסיבוכיות הכללית היא:

סיבוכיות זמן: $O(\log \log n)$.

סיבוכיות מקום: $O(n)$, עבור רשימת השאלות והציונים.

מניעת שאלות כפולות:

המערכת משתמשת ב-HashSet כדי לשמור נוסחים שכבר הופיעו.

עבור n שאלות, כל בדיקת קיום היא במוצע $O(1)$, ולכן:

סיבוכיות זמן: $O(n)$.

סיבוכיות מקום: $O(n)$.

שמירת שאלות למשרה:

אם החברה שומרת q שאלות, המערכת עוברת על כל השאלות ושומרת אותן במסד הנתונים.

סיבוכיות זמן: $O(q)$, לא כולל זמן DB.

סיבוכיות מקום: $O(1)$ או $O(q)$, בהתאם לאופן טעינת הרשימה בזיכרון.

התחלת ראיון:

המערכת יוצרת ראיון חדש ושולפת את שאלות המשרה. אם יש q שאלות:

סיבוכיות זמן: $O(q)$.

סיבוכיות מקום: $O(q)$, עבור רשימת השאלות המוחזרת ללקוח.

שליחת תשובות מועמד:

המערכת שומרת את התשובה, בודקת רלוונטיות, ובמידת הצורך מפעילה מודלי אבחון.

אם אורך הטקסט הוא m ומספר המודלים שרצים הוא k :

סיבוכיות זמן: $O(m+k)$, מבחינת זרימת המערכת.

זמן המודלים עצמם תלוי במימוש הפנימי שלהם.

סיבוכיות מקום: $O(k)$, עבור תוצאות המודלים.

תמלול אודיו:

זמן התמלול תלוי באורך קובץ האודיו. אם אורך ההקלטה הוא t :

סיבוכיות זמן: $O(t)$, בקירוב, לא כולל מורכבות פנימית של מודל התמלול.

סיבוכיות מקום: $O(t)$, עבור קובץ האודיו והתוצאה הזמנית.

חישוב ציון סופי:

המערכת עוברת על כל התשובות ותוצאות האבחון של הראיון. אם קיימות a תשובות ו- d תוצאות אבחון:

סיבוכיות זמן: $O(a+d)$.

סיבוכיות מקום: $O(d)$, עבור טעינת תוצאות האבחון וחישוב הציון.

שליפת ציונים שמורים:

אם לחברה קיימים s ציונים שמורים, המערכת שולפת ומציגה אותם.

סיבוכיות זמן: $O(s)$, לא כולל אופטימיזציות DB.

סיבוכיות מקום: $O(s)$.

סיכום יעילות:

האלגוריתם המרכזי של המערכת תלוי בעיקר במספר השאלות במאגר, במספר השאלות בראיון, במספר

התשובות ובמספר תוצאות האבחון.

מכיוון שבפועל מספר השאלות בראיון מוגבל, זמן הריצה מתאים למערכת Web.

החלק הכבד ביותר מבחינת זמן הוא הרצת מודלי למידת המכונה ותמלול האודיו, ולא פעולות ניהול הנתונים עצמן.

15.5 הקשרים בין היחידות השונות

המערכת מחולקת למספר יחידות מרכזיות הפועלות יחד: ממשק המשתמש, שרת ה-API, מסד הנתונים, רכיבי Python ומודולי החישוב.

ממשק המשתמש ב-React אחראי לקבלת קלט מהחברה ומהמועמד.

כאשר חברה יוצרת משרה, שומרת שאלות או מבקשת לראות ציונים, הלקוח שולח בקשת HTTP לשרת. כאשר מועמד בוחר משרה או שולח תשובה, גם פעולה זו נשלחת לשרת דרך API.

שרת ה-ASP.NET Core מקבל את הבקשות מהלקוח, בודק הרשאות לפי JWT Token, ומחליט איזה שירות להפעיל.

לדוגמה, בקשה ליצירת משרה מפעילה את רכיב ניהול המשרות, בקשה להתאמת שאלות מפעילה את מודול התאמת השאלות, ובקשה לשליחת תשובה מפעילה את רכיב ניהול הראיון.

מודול התאמת השאלות מקבל את תיאור המשרה ואת מאגר השאלות, מפעיל את רכיב Python המתאים, מקבל ממנו דירוג שאלות בפורמט JSON, ומחזיר ללקוח רשימת שאלות מומלצות. לאחר שהחברה שומרת את השאלות, הן נשמרות במסד הנתונים ומשמשות בהמשך את המועמד בזמן הראיון.

כאשר מועמד מצטרף למשרה, השרת יוצר ראיון חדש במסד הנתונים ושולף את השאלות שנשמרו למשרה. בזמן שליחת תשובה, השרת שומר את התשובה, מפעיל את מודול בדיקת הרלוונטיות, ומקבל תשובה האם המענה רלוונטי לשאלה.

אם התשובה אינה רלוונטית, מופעל מודול Diagnosis שמחזיר הודעת הכוונה למועמד. אם התשובה רלוונטית, מופעל מודול ניתוב ולאחריו מודלי האבחון המתאימים.

תוצאות המודלים נשמרות במסד הנתונים באמצעות שכבת הגישה לנתונים.

בסיום הראיון, מודול חישוב הציון הסופי שולף את כל תשובות המועמד, תוצאות הרלוונטיות ותוצאות האבחון, ובונה גרף ראיות משוקלל.

לאחר מכן הוא מחשב ציון סופי, יוצר סיכום מילולי ושומר את התוצאה בטבלת הציונים הסופיים.

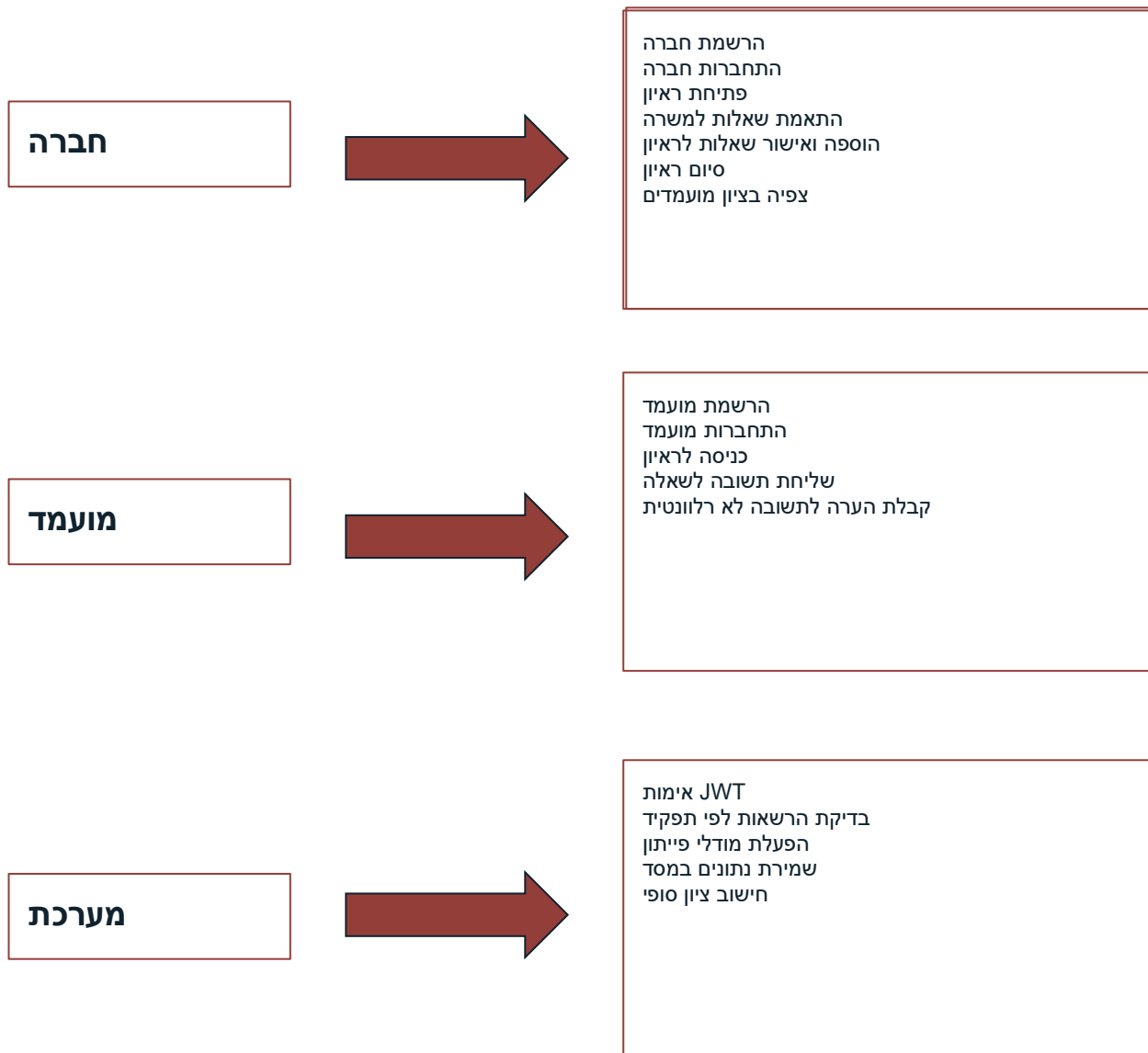
בצד החברה, כאשר החברה נכנסת למסך ציוני מועמדים, הלקוח שולח בקשה לשרת. השרת שולף רק את הציונים של ראיונות השייכים למשרות של אותה חברה, ומחזיר אותם ללקוח להצגה.

כך נשמר הקשר בין כל יחידות המערכת: חברה, משרה, שאלות, מועמד, ראיון, תשובות, אבחונים וציון סופי.

15.6 עץ מודולים



15.7 Use cases Diagram



15.8 רשימת Use cases

1UC - הרשמת חברה:

יצירת חשבון חדש עבור חברה במערכת, כולל שמירת פרטי החברה, הצפנת אימייל ושמירת סיסמה כ-Hash מאובטח.

2UC - הרשמת מועמד:

יצירת חשבון חדש עבור מועמד במערכת, כולל שמירת פרטים אישיים, הצפנת אימייל וטלפון, ושמירת סיסמה בצורה מאובטחת.

3UC - התחברות למערכת:

אימות זהות המשתמש מול מסד הנתונים באמצעות אימייל וסיסמה. לאחר התחברות מוצלחת המערכת מחזירה JWT Token המשמש לזיהוי המשתמש והרשאותיו.

4UC - יצירת משרה פתוחה:

חברה מחוברת יוצרת משרה חדשה במערכת באמצעות שם משרה ותיאור דרישות התפקיד. המערכת שומרת את המשרה במסד הנתונים ומשייכת אותה לחברה המחוברת.

5UC - התאמת שאלות למשרה:

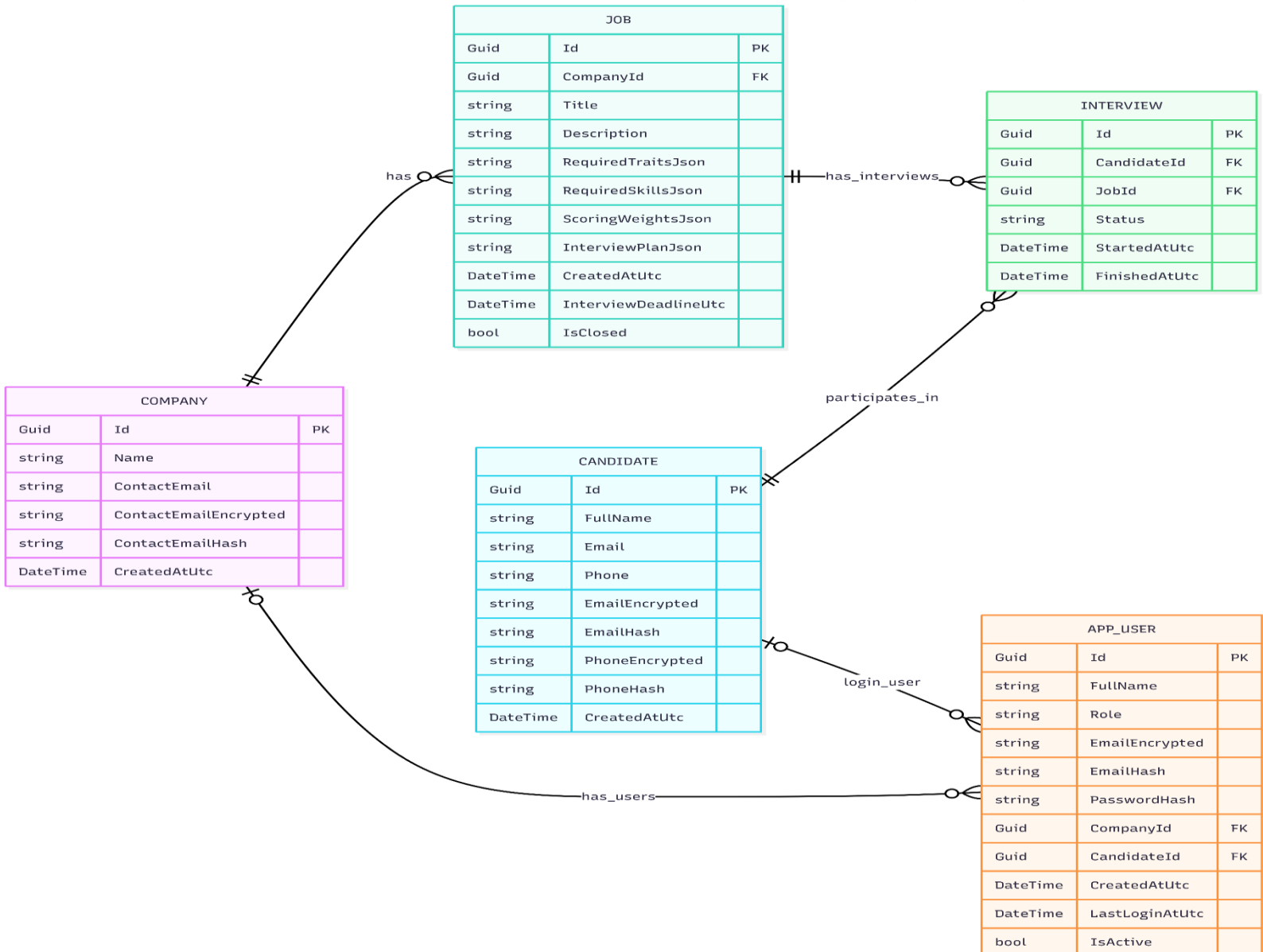
- המערכת מקבלת את תיאור המשרה ומפעילה מודול התאמת שאלות, אשר מדרג שאלות מתוך מאגר השאלות לפי מידת התאמתן למשרה.
- 6UC - שמירת שאלות למשרה:**
 החברה שומרת למשרה את השאלות שנבחרו מתוך רשימת השאלות המומלצות, או מוסיפה שאלות ידניות. המערכת שומרת את השאלות במסד הנתונים ומונעת שמירת שאלות כפולות.
- 7UC - צפייה במשרות פתוחות:**
 מועמד מחובר צופה ברשימת המשרות הפתוחות הקיימות במערכת, כולל שם המשרה ותיאור התפקיד.
- 8UC - הצטרפות למשרה והתחלת ראיון:**
 המועמד בוחר משרה פתוחה ומתחיל ראיון. המערכת יוצרת רשומת ראיון חדשה, משייכת אותה למועמד ולמשרה, ושולפת את השאלות שנשמרו עבור אותה משרה.
- 9UC - תמלול תשובה קולית:**
 המועמד יכול להקליט תשובה קולית. המערכת שולחת את קובץ האודיו לשרת, מפעילה רכיב תמלול ומחזירה את הטקסט המתומלל לתיבת התשובה.
- 10UC - שליחת תשובת מועמד:**
 המועמד שולח תשובה לשאלה בראיון. המערכת שומרת את התשובה במסד הנתונים ומתחילה את תהליך ניתוחה.
- 11UC - בדיקת רלוונטיות:**
 המערכת מפעילה מודל בדיקת רלוונטיות כדי לבדוק האם תשובת המועמד עונה על השאלה מבחינה רעיונית ולא רק מילולית.
- 12UC - יצירת הערה לתשובה לא רלוונטית:**
 אם התשובה אינה רלוונטית או אינה מספקת, מופעל מודל 'Diagnosis' שמזהה את סיבת הכשל ומחזיר למועמד הערה מסייעת לשיפור התשובה.
- 13UC - ניתוב למודלי אבחון:**
 כאשר התשובה רלוונטית, המערכת מפעילה מודל ניתוב שקובע אילו מודלי אבחון יש להריץ על התשובה.
- 14UC - אבחון תשובה:**
 מודלי Python מנתחים את תשובת המועמד ומפיקים ציונים בתחומים כגון ניסיון, יכולות מעשיות, איכות חשיבה, אחריות, בהירות ותקשורת.
- 15UC - סיום ראיון וחישוב ציון:**
 בסיום הראיון החברה מפעילה חישוב ציון סופי. המערכת אוספת את תשובות המועמד, תוצאות הרלוונטיות ותוצאות האבחון, ומשקללת אותן באמצעות גרף ראיות משוקלל.
- 16UC - צפייה בציוני מועמדים:**
 החברה שולפת את הציונים הסופיים של ראיונות השייכים למשרות שלה בלבד, וצופה בציון ההתאמה, סיכום האבחון, חוזקות וחולשות יחסיות של המועמד.

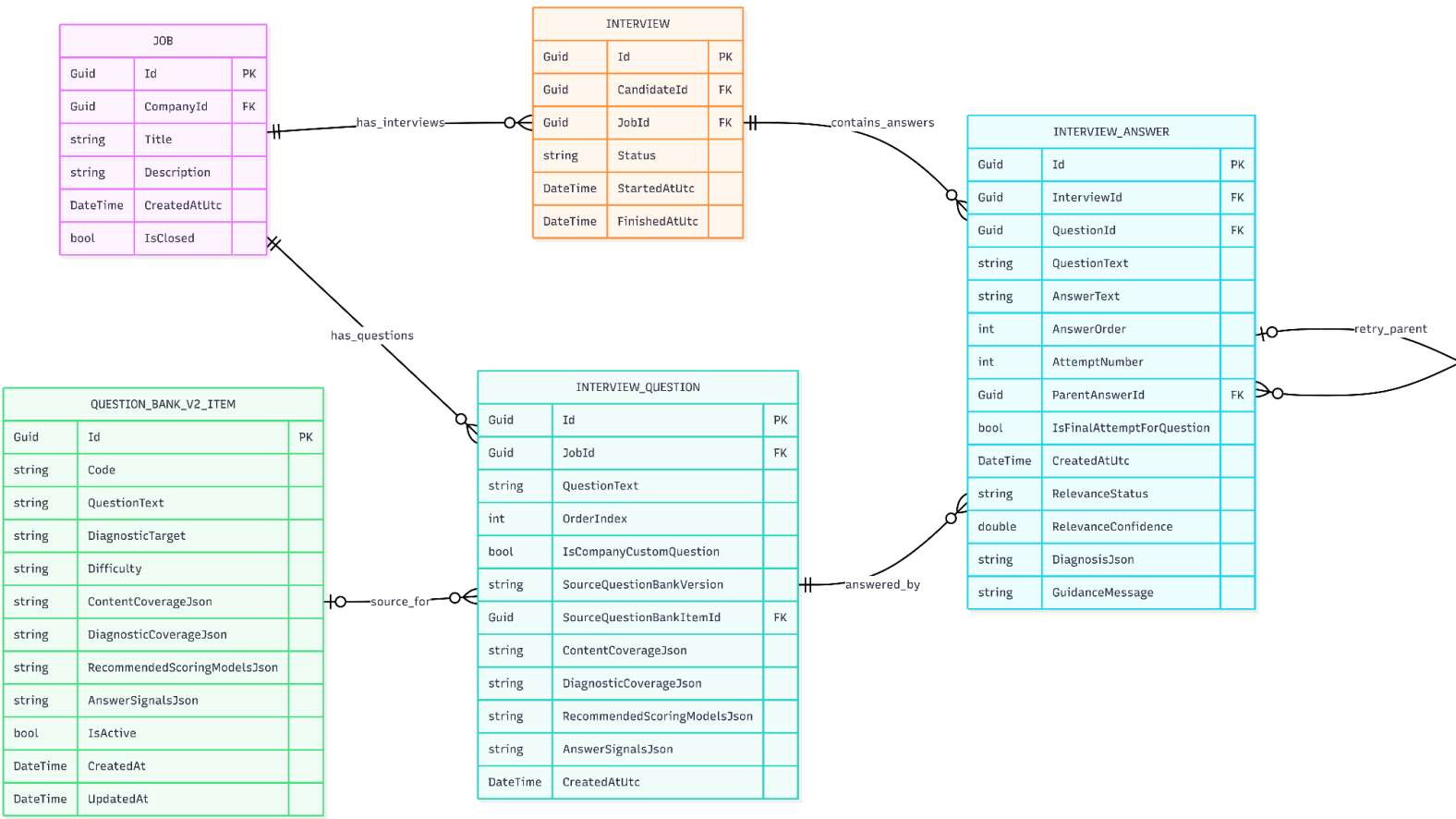
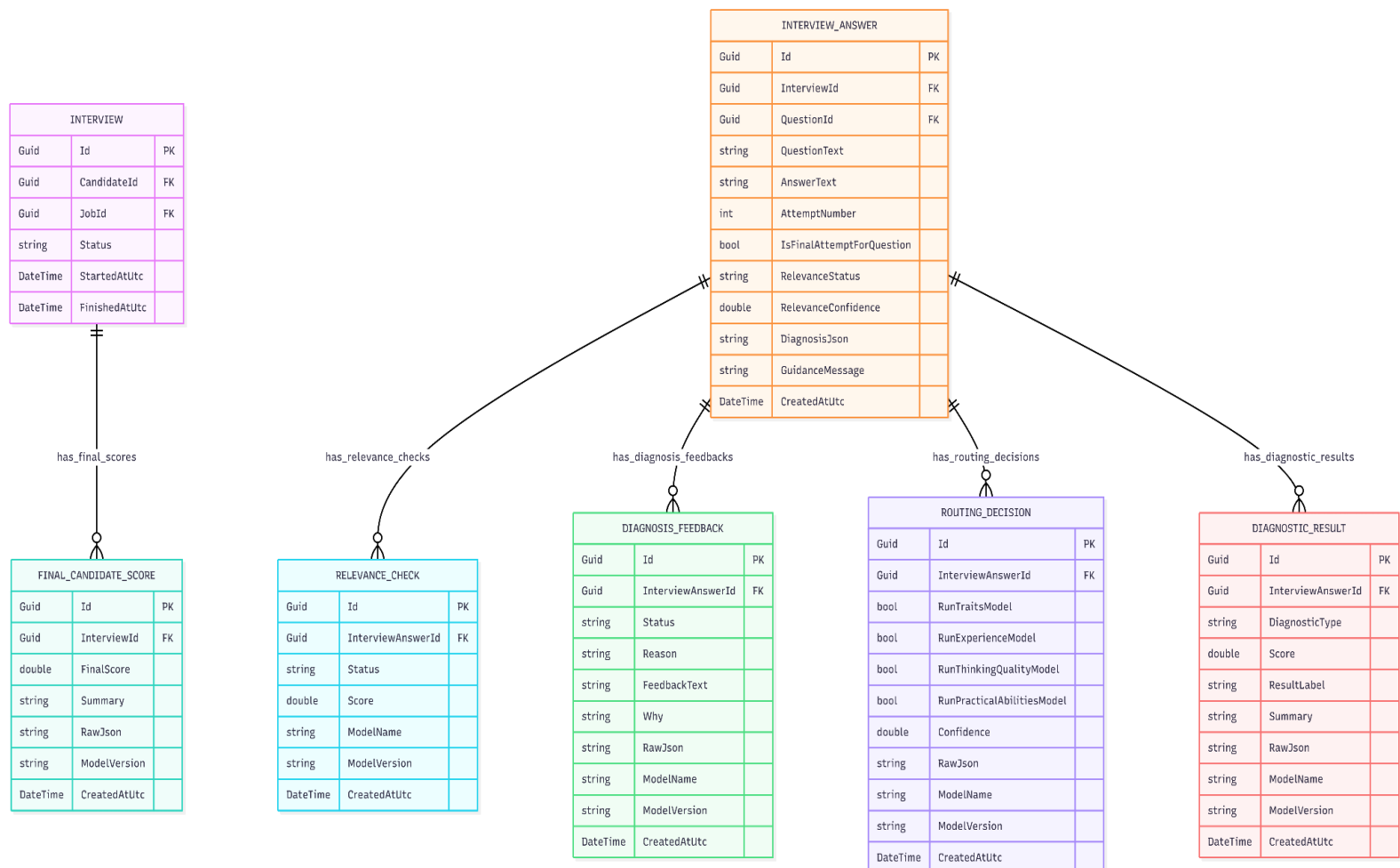
UML תרשים 15.9



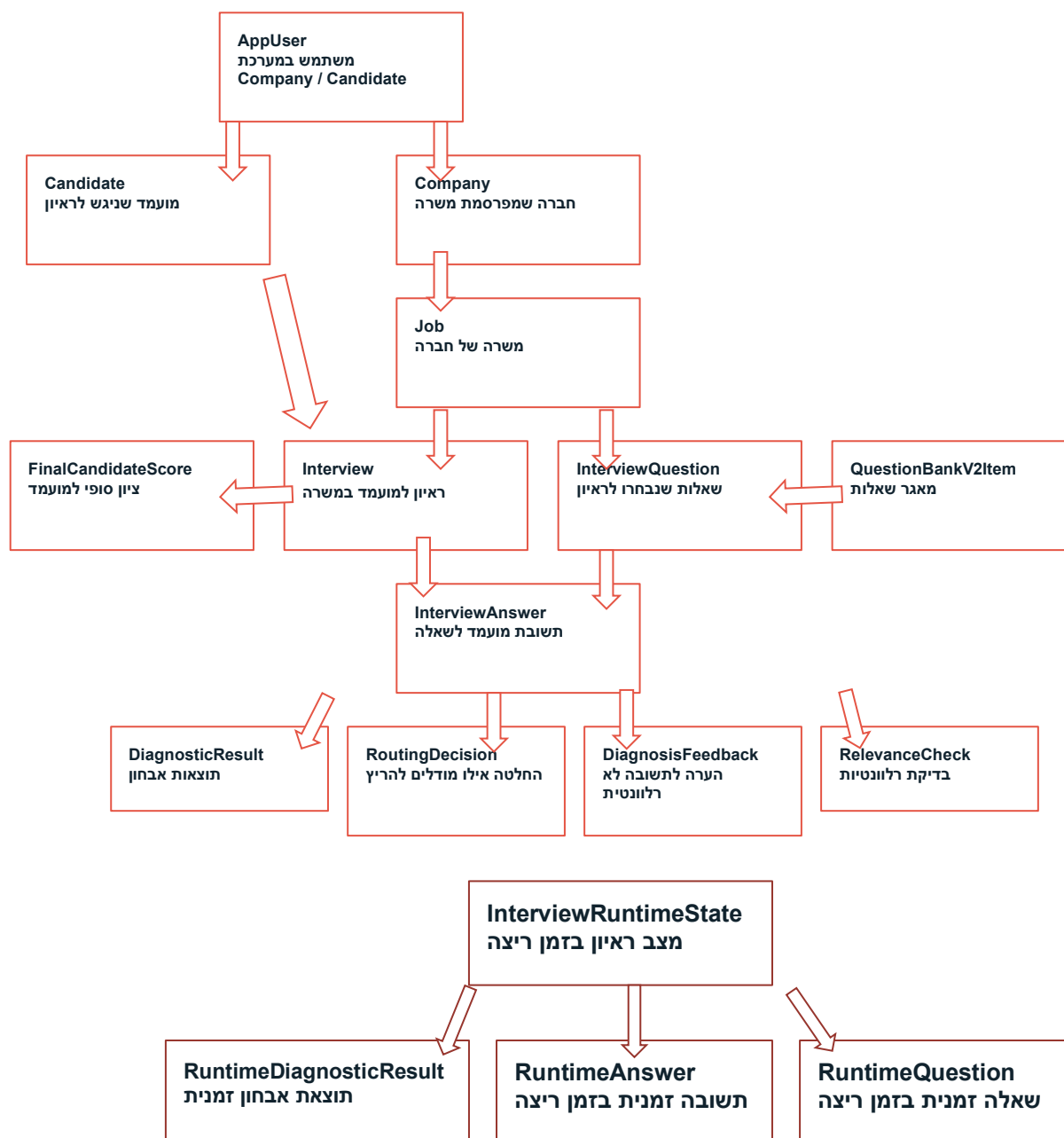
15.10 Design class diagram

תרשים 1, משתמשים, חברות, משרות ומועמדים:



תרשים 2, שאלון ותשובות ראיון:

תרשים 3, עיבוד תשובה, אבחונים וציון סופי:


15.11 תרשים מחלקות.



15.12 תיאור המחלקות המוצעות

המערכת בנויה ממספר יחידות מרכזיות הפועלות יחד. המבנה הכללי כולל צד לקוח, שרת API, שכבת לוגיקה עסקית, שכבת נתונים, רכיבי אבטחה, רכיבי Python ומודלי למידת מכונה. זרימת המידע מתחילה בקלט מהמשתמש בממשק, עוברת לשרת ה-API, ממשיכה לשירותים המתאימים ולמסד הנתונים, ובמידת הצורך מועברת גם למודלי Python. לאחר העיבוד, התוצאה חוזרת לשרת, נשמרת במסד הנתונים ומוצגת למשתמש.

יחידת הלקוח - MindMatchAI.Client

יחידת הלקוח אחראית על ממשק המשתמש של החברה ושל המועמד.

תפקידה הוא להציג מסכים, לקבל קלט מהמשתמש, לשלוח בקשות לשרת ולהציג את התוצאות שמתקבלות ממנו.

הקלט של יחידה זו מגיע מהמשתמשים: פרטי הרשמה והתחברות, תיאור משרה, מספר שאלות רצוי, בחירת משרה, תשובות מועמד בטקסט או בקול, ולחיצה על פעולות כמו שמירת שאלות או חישוב ציון. הפלט של יחידה זו הוא בקשות HTTP שנשלחות לשרת, וכן הצגת מידע למשתמש: שאלות מותאמות, הודעות שגיאה או הצלחה, מצב טעינה, שאלות ראיון, הערות לשיפור תשובה, ציוני מועמדים וסיכום התאמה.

יחידת השרת המרכזי - MindMatchAI.Api / Program.cs

יחידת השרת המרכזי היא נקודת הכניסה של המערכת בצד השרת. תפקידה לקבל בקשות API מהלקוח, להפעיל את השירותים המתאימים ולהחזיר תשובות בפורמט JSON.

הקלט של יחידה זו הוא בקשות HTTP מהלקוח, לדוגמה בקשות להרשמה, התחברות, יצירת משרה, התאמת שאלות, שמירת שאלות, התחלת ראיון, שליחת תשובה, תמלול אודיו, חישוב ציון ושלילת ציונים שמורים.

הפלט של היחידה הוא תשובות JSON ללקוח.

בנוסף, היחידה מפעילה שירותים פנימיים, ניגשת למסד הנתונים, מפעילה רכיבי Python ומבצעת בדיקות הרשאה. Program.cs מגדיר את נקודות הקצה, JWT, Swagger, Dependency Injection, חיבור למסד הנתונים והגדרות מערכת נוספות.

יחידת ניהול המשתמשים והאבטחה - Services / Security

יחידה זו אחראית על ניהול משתמשים, הרשאות והגנה על מידע רגיש.

היא כוללת שירותים ומחלקות כגון AppUser, PasswordService, JwtTokenService ו-PersonalDataProtector.

הקלט של היחידה כולל פרטי משתמש כגון אימייל, סיסמה, טלפון, סוג משתמש ופרטי התחברות.

בנוסף, היא מקבלת בקשות לבדוק הרשאה או ליצור Token.

הפלט של היחידה כולל סיסמה שמורה כ-JWT Token, Hash, לאחר התחברות מוצלחת, נתונים מוצפנים עבור מידע אישי, ו-Hash לשדות חיפוש כמו אימייל.

יחידה זו גם מחזירה לשרת מידע על סוג המשתמש, לדוגמה Company או Candidate, כדי לאפשר הרשאות מתאימות.

יחידת ניהול הנתונים - Data / InterviewDbContext

יחידת ניהול הנתונים אחראית על הקשר בין מחלקות המערכת לבין מסד הנתונים PostgreSQL. המחלקה InterviewDbContext מגדירה את הטבלאות, הקשרים בין הישויות, האינדקסים והגישה לנתונים.

הקלט של היחידה מגיע משירותי המערכת, לדוגמה בקשה לשמור משתמש, ליצור משרה, לשמור תשובה, לשמור תוצאות אבחון או לשלוח ציונים שמורים.

הפלט של היחידה הוא נתונים שנשלפים ממסד הנתונים או אישור שהנתונים נשמרו.

דרך יחידה זו נשמרים ונשלפים משתמשים, חברות, מועמדים, משרות, ראיונות, שאלות, תשובות, תוצאות רלוונטיות, תוצאות אבחון וציונים סופיים.

יחידת הישויות המרכזיות - Models

יחידה זו כוללת את מחלקות הנתונים המרכזיות של המערכת.

המחלקות מייצגות את הישויות שנשמרות במסד הנתונים ומשמשות את תהליך הראיון.

הקלט של יחידה זו הוא הנתונים שהמערכת מקבלת מהמשתמש או יוצרת במהלך התהליך, כגון פרטי חברה, פרטי מועמד, תיאור משרה, שאלה, תשובה, תוצאת אבחון וציון סופי.

הפלט של היחידה הוא אובייקטים מסודרים שנשמרים במסד הנתונים או מועברים בין שירותי המערכת.

המחלקות המרכזיות הן AppUser, Company, Candidate, Job, Interview, InterviewQuestion, DiagnosticResult ו-FinalCandidateScore.

Company מייצגת חברה במערכת. Candidate מייצגת מועמד. Job מייצגת משרה פתוחה.

Interview מייצגת ראיון שנוצר כאשר מועמד מצטרף למשרה.

InterviewQuestion מייצגת שאלה שנשמרה למשרה או מוצגת בראיון.

InterviewAnswer מייצגת תשובת מועמד. FinalCandidateScore

מייצגת את הציון הסופי והסיכום של המועמד.

יחידת התאמת שאלות למשרה - Question Suggestion

יחידה זו אחראית על בחירת שאלות ראיון מתאימות למשרה מתוך מאגר השאלות. היא כוללת שירותים כגון QuestionSuggestionService ו-JobQuestionMatcherService. הקלט של היחידה הוא תיאור המשרה, שם המשרה, מספר השאלות הרצוי ורשימת שאלות מתוך מאגר 2.QuestionBankV.

העיבוד כולל ניתוח תיאור המשרה, השוואה בין דרישות המשרה לבין מאפייני השאלות, דירוג השאלות לפי התאמה ומניעת כפילויות.

הפלט של היחידה הוא רשימת שאלות מומלצות למשרה. רשימה זו מוחזרת לחברה, ולאחר אישור נשמרת במסד הנתונים כשאלות של אותה משרה.

יחידת ניהול תהליך הראיון - InterviewFlow

יחידה זו אחראית על הזרימה המרכזית של הראיון לאחר שמועמד מצטרף למשרה. היא כוללת שירותים כגון InterviewFlowService, RelevanceService, DiagnosisService, RoutingService, DiagnosticModelRunnerService ו-DerivedCategoryService. הקלט של היחידה הוא מזהה ראיון, מזהה שאלה, נוסח השאלה, תשובת המועמד, מספר ניסיון ומידע קודם על השאלה במקרה של ניסיון חוזר.

העיבוד כולל שמירת התשובה, בדיקת רלוונטיות, הפקת הערה במקרה שהתשובה אינה מספקת, ניתוב התשובה למודלי אבחון מתאימים, הפעלת המודלים ושמירת תוצאות האבחון. הפלט של היחידה הוא סטטוס רלוונטיות, הודעת הכוונה למועמד במקרה הצורך, תוצאות אבחון, ועדכון מצב הראיון כך שהמועמד יוכל להמשיך לשאלה הבאה או לנסות שוב.

יחידת תוצאות האבחון - Diagnosis and Results

יחידה זו שומרת את תוצאות העיבוד של תשובות המועמד. היא מאפשרת למערכת לשמור לא רק את התשובה עצמה, אלא גם את כל תהליך הניתוח שעברה. הקלט של היחידה הוא פלטים ממודלי הרלוונטיות, ה-Diagnosis, הניתוב ומודלי האבחון. הפלט של היחידה הוא רשומות מובנות במסד הנתונים, כגון תוצאת רלוונטיות, סיבת כשל, החלטת ניתוב ותוצאות אבחון. מחלקות רלוונטיות כוללות RelevanceCheck או RelevanceResult, DiagnosisFeedback, RoutingDecision ו-DiagnosticResult.

יחידת חישוב הציון הסופי - Scoring

יחידה זו אחראית על חישוב ציון ההתאמה הסופי של המועמד. תפקידה לאסוף את כלל הראיות שנוצרו לאורך הראיון ולשלב אותן לציון אחד מוסבר. הקלט של היחידה הוא מזהה ראיון, תשובות המועמד, תוצאות בדיקת הרלוונטיות, תוצאות האבחון, מאפייני המשרה והמשקלים של כל מאפיין. העיבוד כולל איסוף ראיות, חישוב ציונים לפי מאפיינים, זיהוי חוזקות וחולשות, זיהוי דפוסים חוזרים, והפעלת

גרף ראיות משוקלל - Weighted Candidate Evidence Graph

הפלט של היחידה הוא FinalCandidateScore הכולל ציון מספרי, סיכום מילולי, גרסת מודל, RawJson עם פירוט החישוב, חוזקות, חולשות ודפוסים מרכזיים. התוצאה נשמרת במסד הנתונים ומוצגת לחברה. יחידת Runtime לניהול מצב זמני

יחידת Runtime משמשת לייצוג מצב זמני בזמן ריצה, בעיקר עבור נתונים שנדרשים במהלך תהליך פעיל. היא כוללת מחלקות כגון RuntimeQuestion, RuntimeAnswer ו-RuntimeInterviewState. הקלט של היחידה הוא מידע זמני שנוצר במהלך הרצת הראיון, כגון שאלה נוכחית, תשובה נוכחית, ניסיון חוזר או תוצאה זמנית.

הפלט של היחידה הוא מצב זמני שבו שירותי המערכת יכולים להשתמש בזמן העיבוד. עם זאת, המידע החשוב לטווח ארוך נשמר במסד הנתונים, ולכן יחידה זו אינה מחליפה את שכבת הנתונים אלא משמשת כתמיכה בזמן ריצה.

יחידת הפעלת מודלי Python - Python / Python-Services

יחידה זו אחראית על החיבור בין שרת ה-C# לבין מודלי למידת המכונה שנכתבו ב-Python. היא כוללת מחלקות כגון PythonRunner ו-PythonPaths. הקלט של היחידה הוא שם הסקריפט או הנתביב שלו, וכן נתוני הקלט למודל, כגון תיאור משרה, שאלה, תשובה או קובץ אודיו.

העיבוד כולל יצירת תהליך חיצוני, הרצת סקריפט Python, העברת פרמטרים, קריאת הפלט והמרתו לאובייקט JSON. הפלט של היחידה הוא תוצאה מובנית שמוחזרת לשירות ה-#C, לדוגמה רשימת שאלות מומלצות, סטטוס רלוונטיות, סיבת כשל, ציוני אבחון או טקסט מתומלל.

יחידת מודלי למידת המכונה - Python Models

יחידה זו כוללת את רכיבי למידת המכונה ועיבוד השפה הטבעית של המערכת. כל מודל אחראי למשימה אלגוריתמית אחרת. הקלט של היחידה משתנה לפי המודל. מודל התאמת השאלות מקבל תיאור משרה ושאלות אפשריות. מודל הרלוונטיות מקבל שאלה ותשובה. מודל Diagnosis מקבל תשובה לא מספקת ושאלה מקורית. מודל Routing מקבל שאלה ותשובה. מודלי האבחון מקבלים טקסט של שאלה ותשובה. מודול התמלול מקבל קובץ אודיו. הפלט של היחידה הוא JSON שמוחזר לשרת ה-#C. הפלט יכול לכלול שאלות מדורגות, סטטוס רלוונטיות, סיבת כשל, החלטת ניתוב, ציוני ניסיון, ציוני יכולות מעשיות, ציוני איכות חשיבה, מאפייני אישיות או טקסט מתומלל.

יחידת מסד הנתונים - PostgreSQL / Neon

יחידת מסד הנתונים אחראית לשמירת כל המידע הקבוע של המערכת. היא מאפשרת למערכת לשמור נתונים גם לאחר רענון הדפדפן, יציאה מהמערכת או הפעלה מחדש. הקלט של היחידה הוא נתונים שמגיעים משרת היישום דרך InterviewDbContext, כגון משתמשים, משרות, שאלות, ראיונות, תשובות, תוצאות אבחון וציונים. הפלט של היחידה הוא נתונים שנשלפים לפי צורך, לדוגמה משרות פתוחות למועמד, שאלות של משרה, תשובות של ראיון, תוצאות אבחון וציונים שמורים לחברה.

זרימת המידע בין היחידות:

החברה מזינה פרטי משרה בממשק React. הלקוח שולח את הנתונים ל-MindMatchAI.Api. השרת בודק הרשאות באמצעות יחידת האבטחה, שומר את המשרה דרך InterviewDbContext במסד הנתונים, ולאחר מכן מפעיל את יחידת התאמת השאלות. יחידת התאמת השאלות מפעילה מודל Python דרך PythonRunner, מקבלת שאלות מדורגות ומחזירה אותן ללקוח. לאחר שהחברה שומרת את השאלות, הן נשמרות במסד הנתונים תחת אותה משרה. המועמד נכנס למערכת, צופה במשרות פתוחות ובוחר משרה. הלקוח שולח בקשת הצטרפות לשרת, השרת יוצר ראיון במסד הנתונים ושולף את השאלות שנשמרו למשרה. כאשר המועמד שולח תשובה, השרת שומר אותה, מפעיל את מודל הרלוונטיות דרך PythonRunner, ומקבל תוצאה. אם התשובה אינה רלוונטית, מופעל מודל Diagnosis והערה מוחזרת למועמד. אם התשובה רלוונטית, השרת מפעיל Routing ולאחר מכן את מודלי האבחון המתאימים. תוצאות האבחון נשמרות במסד הנתונים. בסיום הראיון, יחידת Scoring שולפת את כל התשובות, תוצאות הרלוונטיות ותוצאות האבחון של הראיון, מחשבת ציון התאמה סופי באמצעות גרף ראיות משוקלל, ושומרת את התוצאה כ-FinalCandidateScore. כאשר החברה נכנסת למסך ציונים, השרת שולף מהמסד רק ציונים השייכים למשרות של אותה חברה ומחזיר אותם ללקוח להצגה.

16. תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם

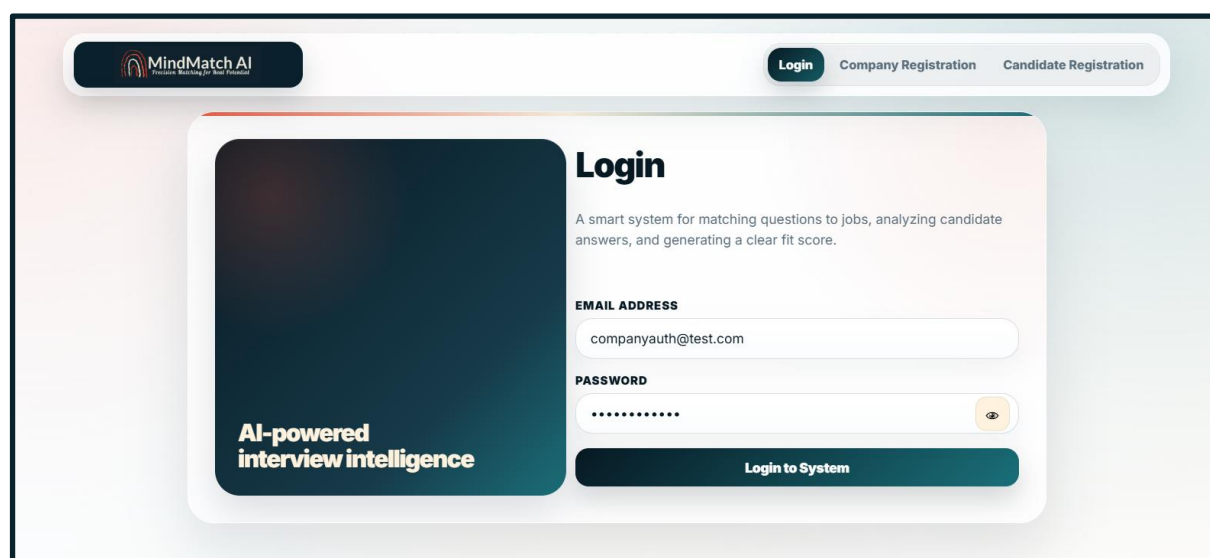


17. עבור כל מסך באפליקציה

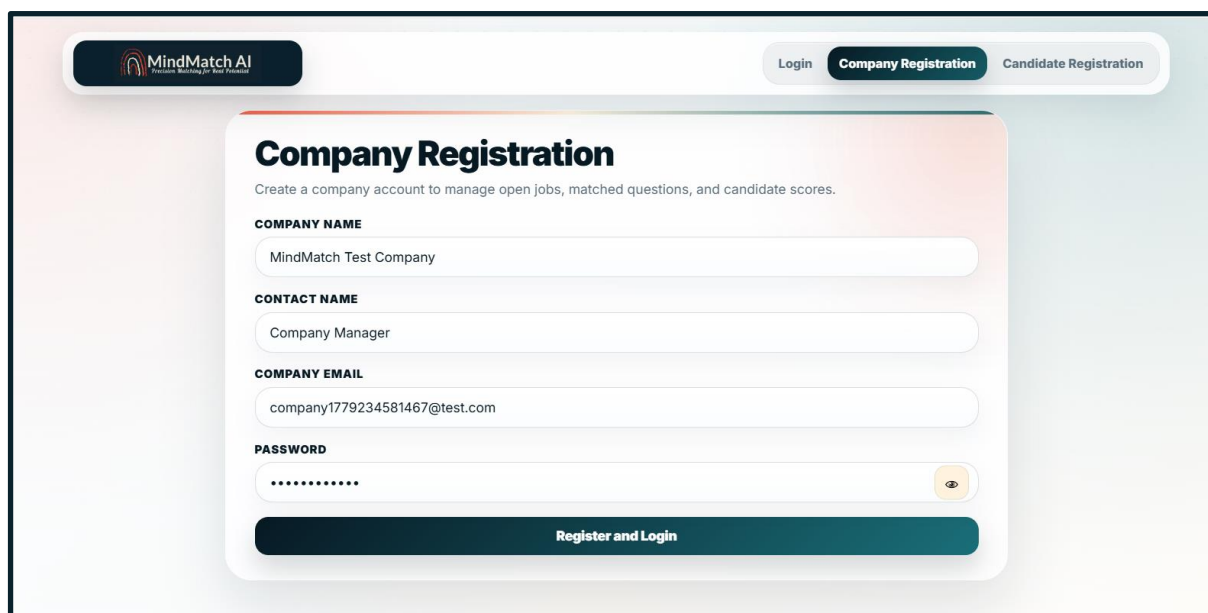
17.1 מה תפקידו של המסך

17.2 תיאור המסך

17.3 צילום מסך



מסך זה מאפשר למשתמש קיים להתחבר למערכת. המסך כולל שדות להזנת אימייל וסיסמה, כפתור התחברות, ותפריט ניווט להרשמת חברה או הרשמת מועמד. לאחר התחברות מוצלחת, המערכת מזהה את סוג המשתמש ומעבירה אותו לאזור המתאים: אזור חברה או אזור מועמד.



Company Registration

Create a company account to manage open jobs, matched questions, and candidate scores.

COMPANY NAME

MindMatch Test Company

CONTACT NAME

Company Manager

COMPANY EMAIL

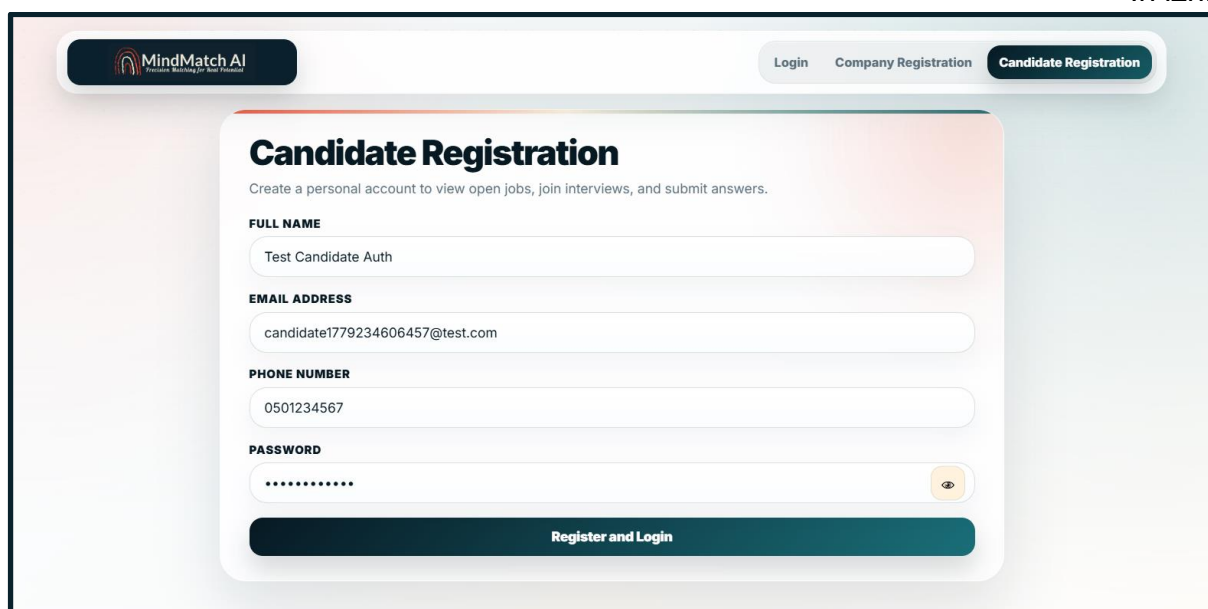
company1779234581467@test.com

PASSWORD

.....

Register and Login

מסך זה מאפשר לחברה ליצור חשבון חדש במערכת. המסך כולל שדות להזנת שם חברה, שם איש קשר, אימייל וסיסמה. לאחר שליחת הטופס, המערכת יוצרת משתמש מסוג Company, שומרת את פרטי החברה בצורה מאובטחת ומעבירה את המשתמש לאזור החברה.



Candidate Registration

Create a personal account to view open jobs, join interviews, and submit answers.

FULL NAME

Test Candidate Auth

EMAIL ADDRESS

candidate1779234606457@test.com

PHONE NUMBER

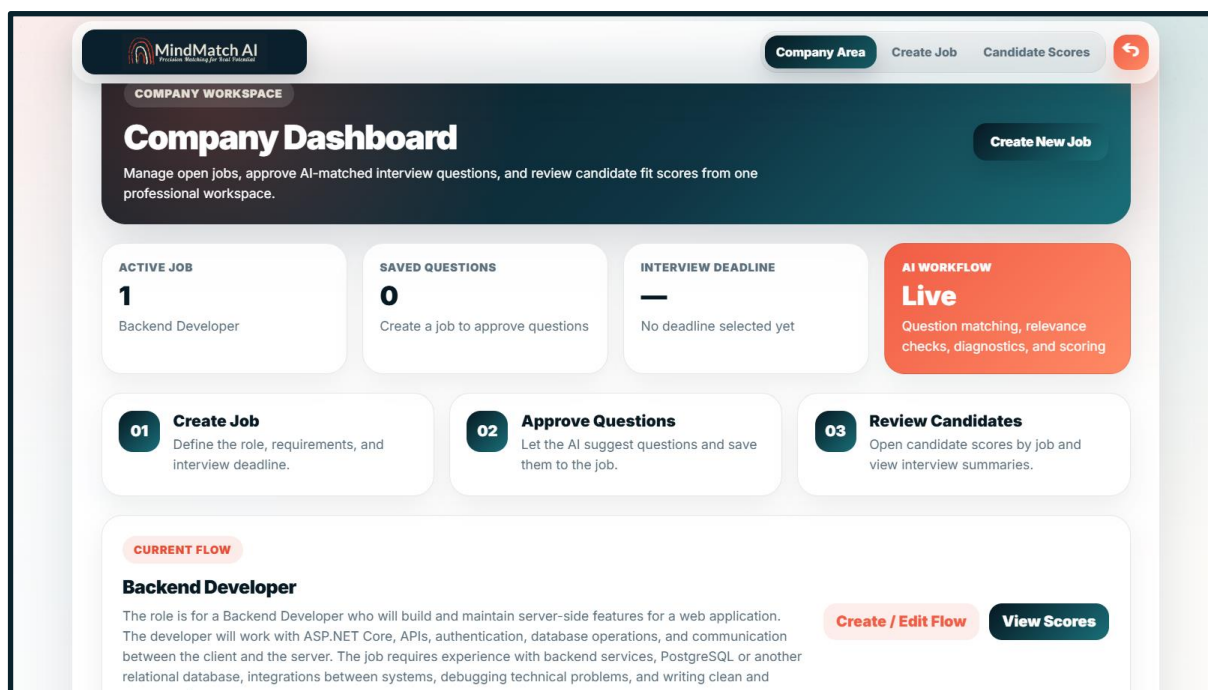
0501234567

PASSWORD

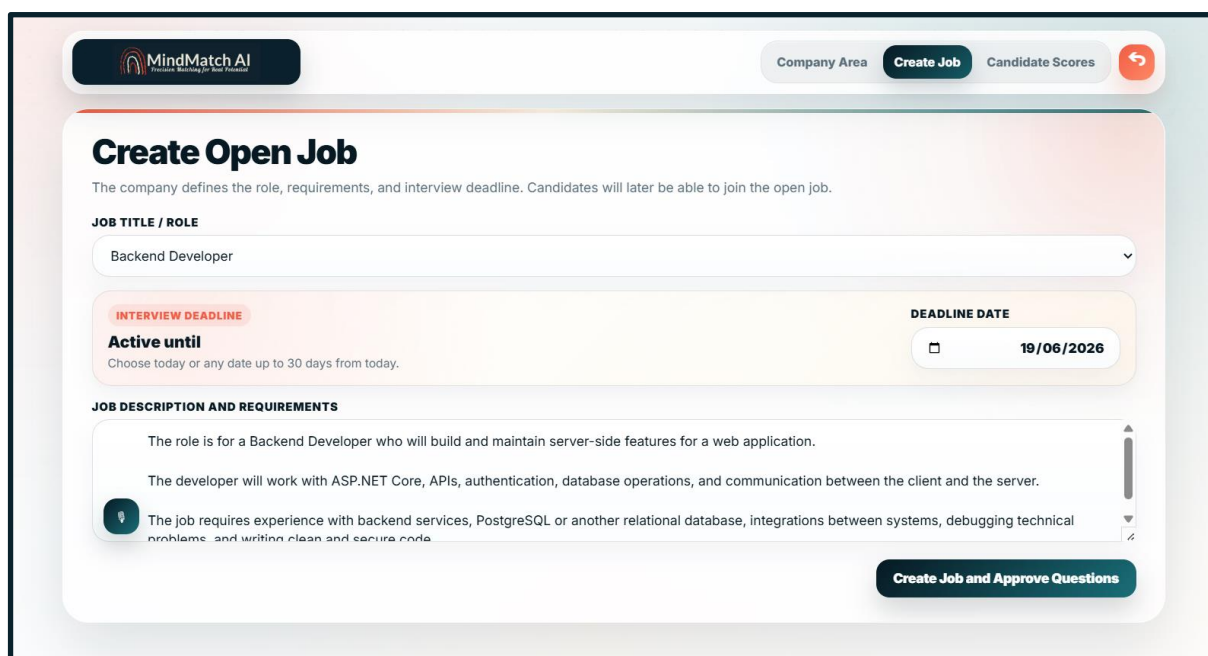
.....

Register and Login

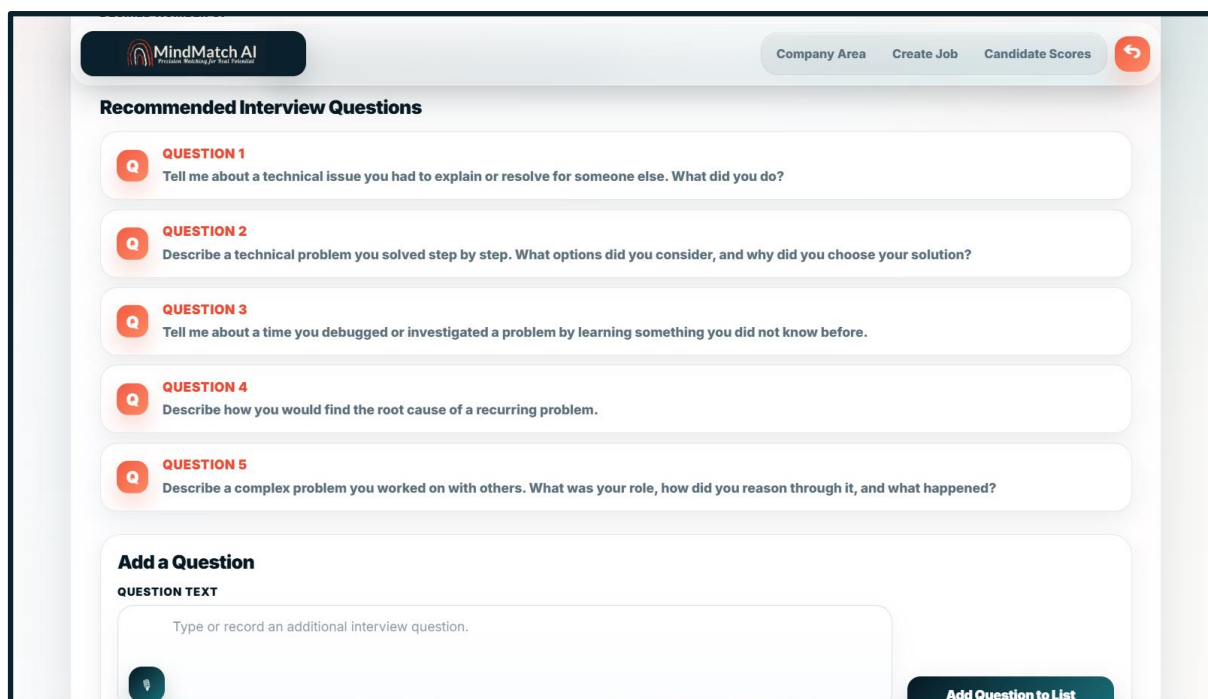
מסך זה מאפשר למועמד ליצור חשבון חדש במערכת. המסך כולל שדות להזנת שם מלא, אימייל, טלפון וסיסמה. לאחר הרשמה מוצלחת, המערכת יוצרת משתמש מסוג Candidate ומעבירה אותו לאזור המועמד.



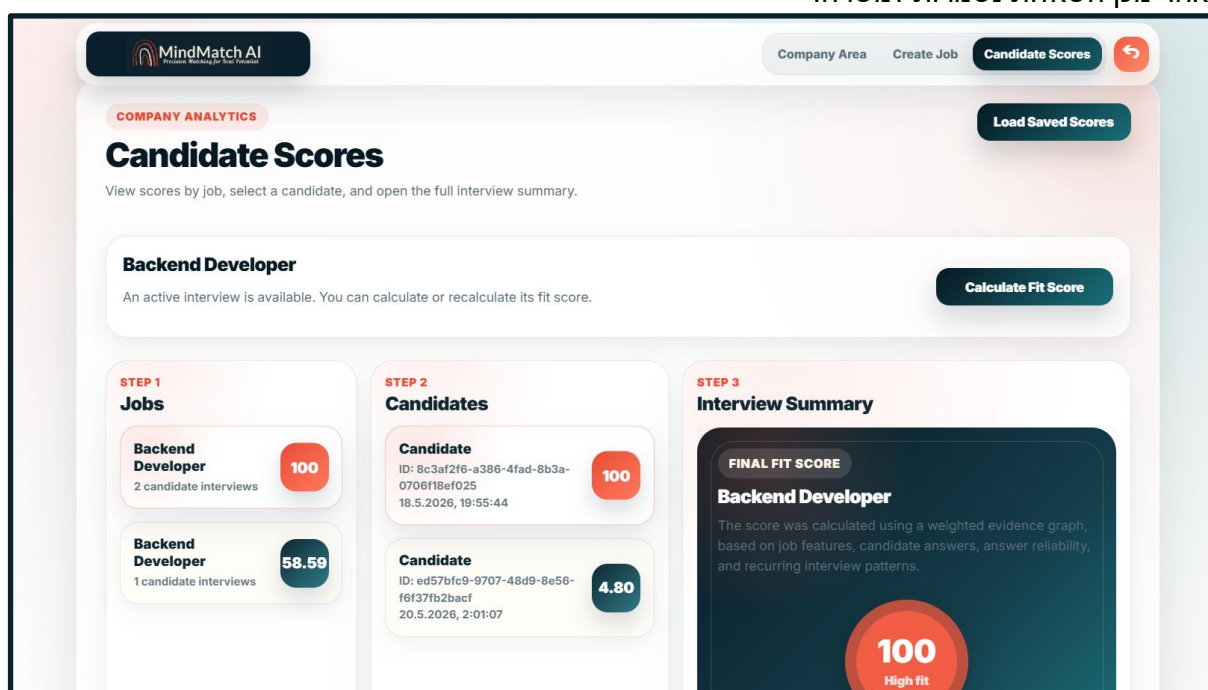
מסך זה משמש כעמוד הבית של החברה לאחר התחברות. המסך מציג לחברה את הפעולות המרכזיות במערכת: יצירת משרה חדשה, צפייה במשרות קיימות וצפייה בציוני מועמדים. בנוסף, מוצג סיכום קצר של מצב העבודה הנוכחי, כגון משרות פעילות, שאלות שנשמרו ותוצאות ראיונות של מועמדים.



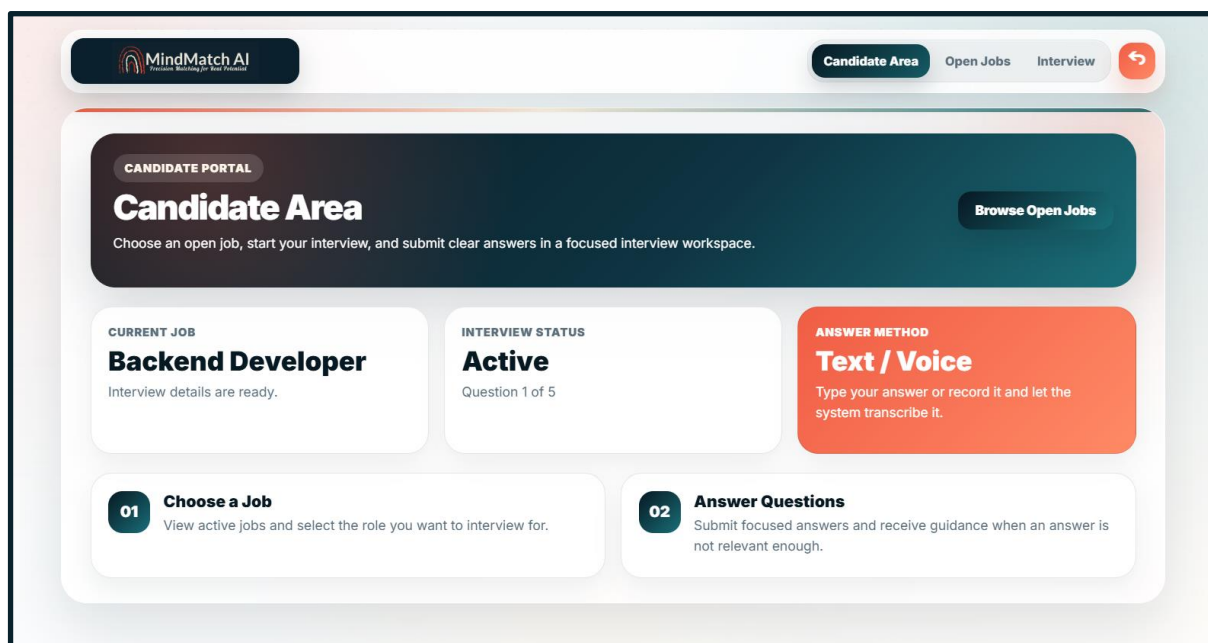
מסך זה מאפשר לחברה ליצור משרה פתוחה חדשה. המסך כולל בחירת שם תפקיד, אפשרות להזנת תפקיד מותאם אישית, ציון תאריך סיום הראיונות, ותיבת טקסט לתיאור המשרה והדרישות. ניתן להזין את התיאור בכתב או באמצעות הקלטה קולית. לאחר יצירת המשרה, המערכת מעבירה את החברה למסך אישור שאלות.



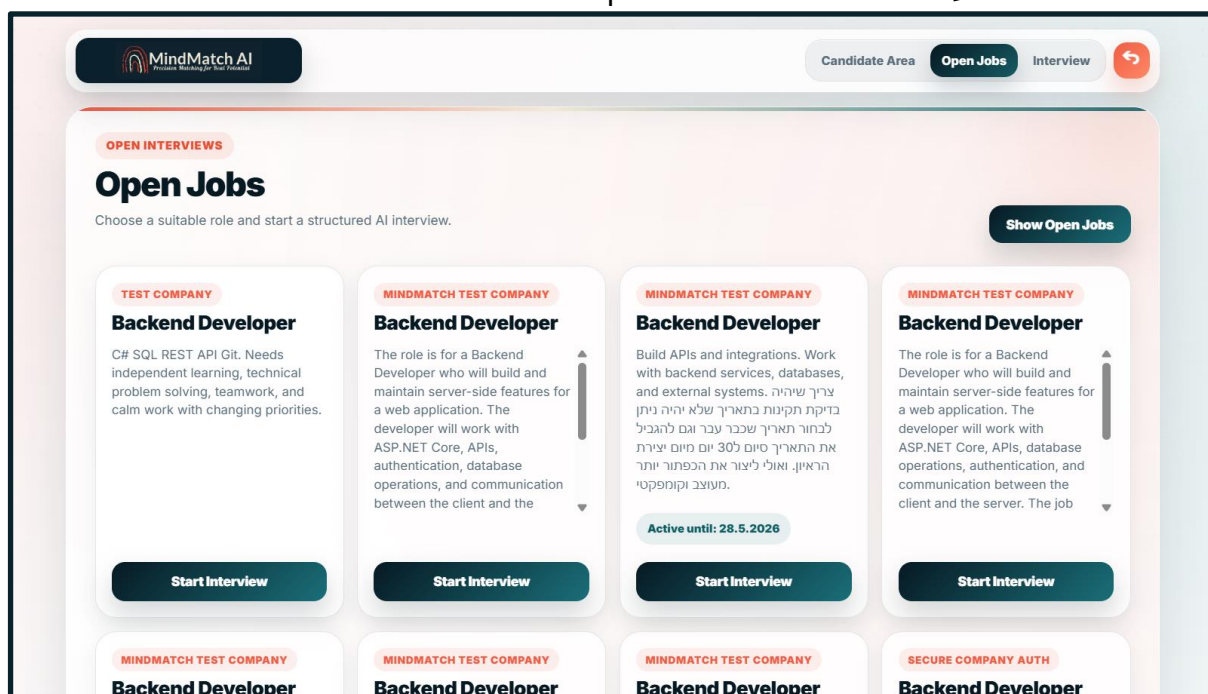
מסך זה מאפשר לחברה לקבל שאלות ראיון מותאמות למשרה ולשמור אותן. המסך מציג את שם המשרה ותיאור המשרה, מאפשר לבחור את מספר השאלות הרצוי, ומציג שאלות מומלצות שנבחרו על ידי מודול התאמת השאלות. החברה יכולה להוסיף גם שאלה ידנית משלה. לאחר מכן השאלות נשמרות למשרה.



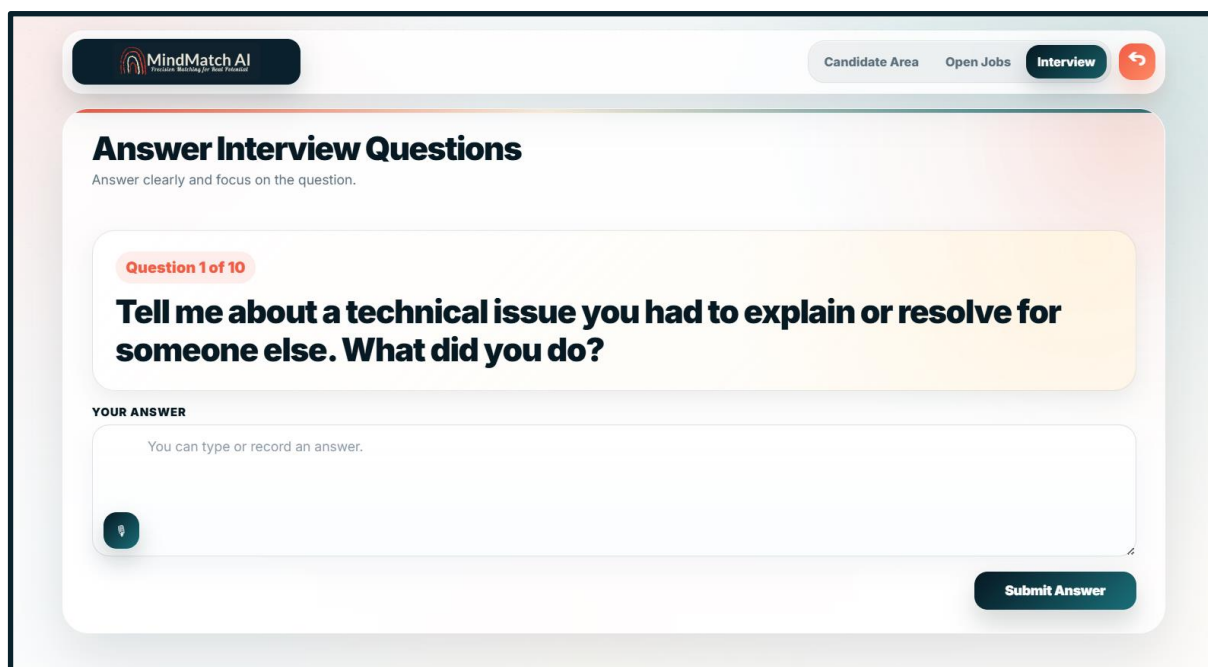
מסך זה מאפשר לחברה לצפות בציוני מועמדים שסיימו ראיונות. המסך כולל אפשרות לטעון ציונים שמורים של ראיונות השייכים לחברה המחוברת בלבד. עבור כל ציון מוצגים פרטי המשרה, מזהה הראיון, מזהה המועמד, תאריך יצירה וציון התאמה סופי.



מסך זה משמש כעמוד הבית של המועמד לאחר התחברות. המסך מציג למועמד אפשרות לצפות במשרות פתוחות ולהתחיל תהליך ראיון. מטרתו להוביל את המועמד בצורה פשוטה וברורה למסך בחירת המשרה.



מסך זה מאפשר למועמד לבחור משרה ולהתחיל ראיון. המסך מציג רשימת משרות פתוחות הכוללת שם משרה ותיאור. המועמד יכול לבחור משרה מתאימה וללחוץ על כפתור התחלת ראיון. לאחר מכן המערכת יוצרת ראיון אישי עבור המועמד.



MindMatch AI Precision Matching for Real Potential

Candidate Area Open Jobs Interview

Answer Interview Questions

Answer clearly and focus on the question.

Question 1 of 10

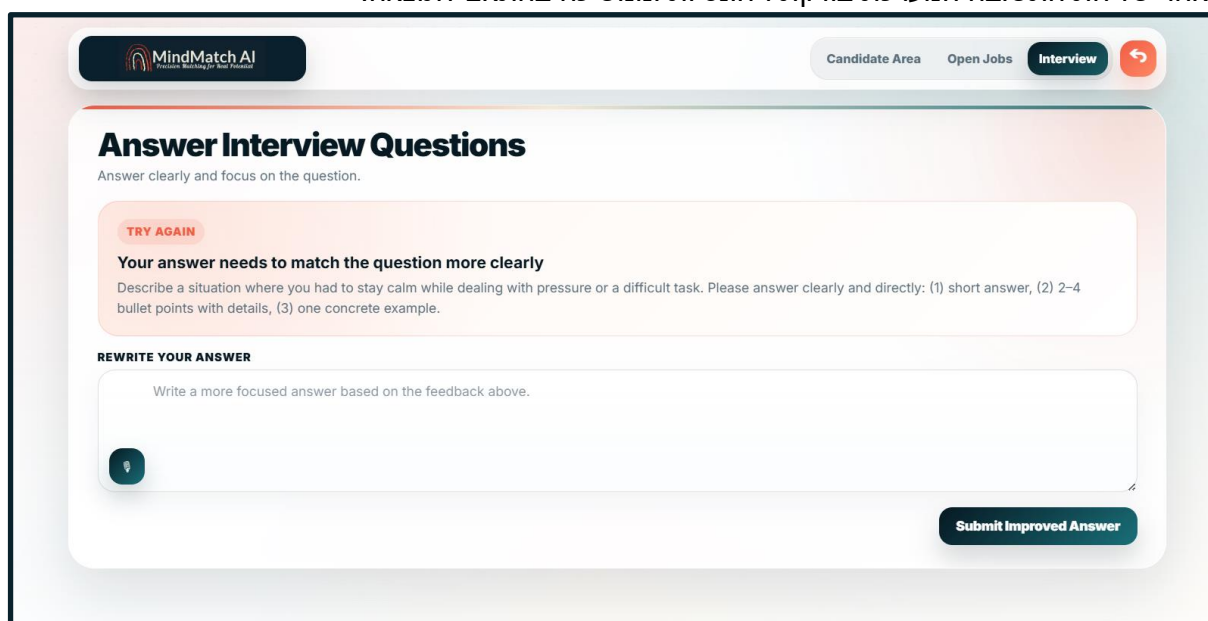
Tell me about a technical issue you had to explain or resolve for someone else. What did you do?

YOUR ANSWER

You can type or record an answer.

Submit Answer

מסך זה מאפשר למועמד לענות על שאלות הראיון. המסך מציג את מספר השאלה הנוכחית מתוך כלל השאלות, את נוסח השאלה ותיבת תשובה. המועמד יכול להקליד תשובה או להקליט תשובה קולית שתעבור תמלול לטקסט. לאחר שליחת התשובה המערכת בודקת רלוונטיות וממשיכה בהתאם לתוצאה.



MindMatch AI Precision Matching for Real Potential

Candidate Area Open Jobs Interview

Answer Interview Questions

Answer clearly and focus on the question.

TRY AGAIN

Your answer needs to match the question more clearly

Describe a situation where you had to stay calm while dealing with pressure or a difficult task. Please answer clearly and directly: (1) short answer, (2) 2-4 bullet points with details, (3) one concrete example.

REWRITE YOUR ANSWER

Write a more focused answer based on the feedback above.

Submit Improved Answer

מסך זה מוצג כאשר תשובת המועמד אינה עונה על השאלה בצורה מספקת. במסך מופיעה הודעת הכוונה שמסבירה למועמד כיצד לשפר את תשובתו. המועמד נשאר באותה שאלה ויכול לנסות לענות שוב. מטרת המסך היא לאפשר למועמד לתקן תשובה לא רלוונטית במקום לפסול אותה מיד.

18. הסבר על תפקידי אלמנטים בתצוגת הפרויקט

במערכת קיימים מספר אלמנטי תצוגה מרכזיים, שלכל אחד מהם תפקיד מוגדר בתהליך העבודה של המשתמש. האלמנטים תוכננו כך שיאפשרו לחברה ולמועמד לבצע את הפעולות הנדרשות בצורה ברורה, פשוטה ונוחה.

כפתורי הפעולה הראשיים משמשים לביצוע פעולות מרכזיות במערכת, כגון התחברות, הרשמה, יצירת משרה, שמירת שאלות, התחלת ראיון, שליחת תשובה וחישוב ציון התאמה. כפתורים אלו מופיעים בדרך כלל בסיום טופס או ליד הנתונים שעליהם מתבצעת הפעולה, כדי שהמשתמש יבין בבירור מהי הפעולה הבאה בתהליך.

כפתורי הניווט משמשים למעבר בין המסכים המרכזיים במערכת. אצל משתמש מסוג חברה קיימים כפתורים למעבר בין אזור החברה, יצירת משרה וצפייה בציוני מועמדים. אצל משתמש מסוג מועמד קיימים כפתורים למעבר בין האזור האישי, משרות פתוחות ומסך הראיון הפעיל. כך כל משתמש רואה רק את אזורי המערכת הרלוונטיים לו.

כפתור החזרה מופיע במסכים פנימיים ומאפשר למשתמש לחזור למסך קודם. מטרתו למנוע מצב שבו המשתמש אינו יודע כיצד לצאת ממסך מסוים, ולאפשר ניווט פשוט וברור לאורך כל תהליך העבודה.

תיבות הטקסט משמשות להזנת מידע חופשי. לדוגמה, חברה מזינה בתיבת טקסט את תיאור המשרה והדרישות, ומועמד מזין בתיבת טקסט את תשובותיו לשאלות הראיון.

תיבות הטקסט עוצבו כך שיהיו רחבות וברורות, עם טקסט עזר שמסביר למשתמש מה עליו להקליד. שדות הקלט הרגילים משמשים להזנת פרטים קצרים כגון שם חברה, שם איש קשר, אימייל, טלפון וסיסמה. לכל שדה קיימת תווית ברורה המופיעה מעליו, כדי שהמשתמש יבין איזה מידע עליו להזין. שדות אלו נבדקים לפני שליחת הטופס כדי לוודא שלא חסר מידע חיוני.

שדה הסיסמה משמש להזנת סיסמה בעת הרשמה והתחברות. הסיסמה מוצגת ככרית מחדל כתווים מוסתרים, כדי לשמור על פרטיות המשתמש. לצד השדה קיים כפתור צפייה בסיסמה, שמאפשר למשתמש להציג או להסתיר את הסיסמה בעת הצורך. שדה התאריך מופיע במסך יצירת משרה, והוא מאפשר לחברה לבחור עד איזה תאריך הראיון יהיה פעיל עבור מועמדים.

המערכת מגבילה את בחירת התאריך כך שלא ניתן לבחור תאריך שכבר עבר, ולא ניתן לבחור תאריך רחוק ביותר מ-30 יום ממועד יצירת המשרה.

שדה זה נועד להבטיח שהראיונות יהיו תחומים בזמן ולא יישארו פתוחים ללא הגבלה. כפתור המיקרופון מופיע ליד תיבות טקסט שבהן ניתן להזין קלט קולי. תפקידו לאפשר למשתמש להקליט תשובה או טקסט במקום להקליד ידנית.

לאחר ההקלטה המערכת ממירה את הקלט הקולי לטקסט ומשלבת אותו בתהליך העבודה. רשימות וכרטיסים משמשים להצגת מידע בצורה מסודרת וברורה.

לדוגמה, משרות פתוחות מוצגות למועמד באמצעות כרטיסים, כאשר כל כרטיס כולל את שם המשרה, תיאור המשרה, תאריך סיום וכפתור התחלת ראיון.

גם ציוני מועמדים מוצגים לפי משרות ומועמדים, כך שהחברה יכולה לבחור משרה, לאחר מכן מועמד, ולצפות בסיכום הראיון שלו.

19. הודעות למשתמש

המערכת מציגה למשתמש הודעות מסוגים שונים בהתאם לפעולה שבוצעה. מטרת ההודעות היא לתת למשתמש משוב ברור, להסביר האם הפעולה הצליחה, ולהנחות אותו במקרה של שגיאה או פעולה לא תקינה.

הודעות הצלחה מופיעות כאשר פעולה הסתיימה כראוי.

לדוגמה, לאחר הרשמה מוצלחת, התחברות, יצירת משרה, שמירת שאלות או שליחת תשובה, המערכת מציגה הודעת אישור שמבהירה למשתמש שהפעולה נקלטה ובוצעה בהצלחה.

הודעות שגיאה מופיעות כאשר חסר מידע או כאשר המשתמש ביצע פעולה שאינה תקינה.

לדוגמה, אם המשתמש מנסה לשלוח טופס ללא אימייל או סיסמה, אם הסיסמה קצרה מדי, אם תיאור המשרה קצר מדי, אם המועמד מנסה לשלוח תשובה ריקה, או אם נבחר תאריך שאינו בטווח התקין- המערכת מציגה הודעת שגיאה מתאימה.

במקרה שמועמד מנסה להיכנס לראיון שכבר התחיל בעבר, המערכת מציגה הודעה ליד כרטיס המשרה שעליו הוא לחץ. ההודעה מסבירה למועמד שהוא כבר התחיל את הראיון ולכן אינו יכול להתחיל אותו שוב. כאשר מועמד שולח תשובה שאינה רלוונטית לשאלה, המערכת מציגה הודעת משוב ייעודית. ההודעה מסבירה למועמד כיצד לשפר את התשובה, לדוגמה לענות ישירות על השאלה, להוסיף דוגמה קונקרטית, לפרט את הפעולות שביצע או להתמקד בנושא השאלה. לאחר מכן מוצגת למועמד תיבת טקסט חדשה לכתיבת תשובה משופרת. כאשר המערכת מבצעת פעולה שדורשת זמן עיבוד, כגון בדיקת תשובת מועמד באמצעות מודלי למידת מכונה, מוצגת הודעת טעינה. לדוגמה, ההודעה "The system is checking your answer" מבהירה למשתמש שהמערכת מעבדת את התשובה, ובכך מונעת תחושה שהמערכת נתקעה. לאחר סיום העיבוד מוצגת תוצאה מתאימה: מעבר לשאלה הבאה, הודעת הצלחה או משוב לשיפור התשובה. המערכת כוללת בדיקות תקינות גם בצד הלקוח וגם בצד השרת. בצד הלקוח מתבצעות בדיקות בסיסיות לפני שליחת הטופס, כדי לתת תגובה מהירה למשתמש. בצד השרת מתבצעות בדיקות נוספות כדי לוודא שהנתונים תקינים גם אם המשתמש עקף את בדיקות הדפדפן. לדוגמה, השרת בודק שהמשתמש מחובר, שיש לו הרשאה מתאימה, שהמשרה קיימת, שהראיון שייך למועמד הנוכחי, ושהתאריך שנבחר אינו תאריך שעבר ואינו מעבר ל-30 יום.

20. ממשק משתמש

ממשק המשתמש של המערכת תוכנן כך שיהיה ברור, אחיד ונוח לשימוש עבור שני סוגי משתמשים עיקריים: חברה ומועמד. מאחר שהמערכת כוללת תהליך רב-שלבי, הכולל הרשמה, התחברות, יצירת משרה, בחירת שאלות, מענה לראיון וצפייה בציונים, הושם דגש על ניווט פשוט, חלוקה ברורה בין אזורי המערכת ושימוש באלמנטים חוזרים לאורך כל המסכים. פלטת הצבעים שנבחרה למערכת מבוססת על צבעי הלוגו של MindMatch AI: כחול כהה, טורקיז עמוק, גווני קורל עדינים ורקע בהיר. הצבע הכחול הכהה משדר אמינות, יציבות וטכנולוגיה, ומתאים למערכת העוסקת בתהליכי גיוס והערכה. צבעי הקורל משמשים להדגשת פעולות חשובות והתראות, כדי למשוך את תשומת לב המשתמש מבלי ליצור עומס חזותי. בכל המסכים נשמרים רכיבים אחידים. לוגו המערכת מופיע בחלק העליון של המסך, תפריט ניווט מאפשר מעבר בין האזורים המרכזיים, וכפתור חזרה מופיע במסכים פנימיים. הכותרות בכל מסך גדולות וברורות, ומתחתיהן מופיע טקסט הסבר קצר המתאר את מטרת המסך. אחדות זו מסייעת למשתמש להבין היכן הוא נמצא ומהי הפעולה הבאה שעליו לבצע. הניווט במערכת מתבצע בהתאם לסוג המשתמש. משתמש מסוג חברה יכול לעבור בין אזור החברה, יצירת משרה וצפייה בציוני מועמדים. משתמש מסוג מועמד יכול לעבור בין האזור האישי, משרות פתוחות ומסך הראיון הפעיל. בצורה זו כל משתמש נחשף רק לפעולות הרלוונטיות אליו, והמערכת נמנעת מהצגת אפשרויות מיותרות. הממשק עושה שימוש בכרטיסים להצגת מידע מורכב בצורה נוחה. לדוגמה, משרות פתוחות, מועמדים, ציונים וסיכומי ראיונות מוצגים ככרטיסים נפרדים. כך ניתן להפריד בין פריטי מידע שונים, לשמור על סדר חזותי, ולהקל על המשתמש להתמצא במערכת גם כאשר קיימות כמה משרות או כמה מועמדים. דרישות החומרה והתוכנה מצד המשתמש הן בסיסיות יחסית. המערכת מיועדת לפעול בדפדפן מודרני, ולכן אין צורך בהתקנת תוכנה מיוחדת בצד הלקוח. נדרש מחשב או מחשב נייד עם חיבור אינטרנט יציב, דפדפן עדכני כגון Google Chrome, Microsoft Edge או Firefox, וזיכרון עבודה בסיסי המאפשר טעינת אתר מודרני. לצורך שימוש בקלט קולי נדרש מיקרופון פעיל והרשאת גישה למיקרופון בדפדפן.

בצד השרת נדרשת סביבת הרצה המסוגלת להריץ שרת ASP.NET Core, מוד נתונים PostgreSQL ושרת Python עבור מודלי למידת המכונה. מאחר שהמודלים רצים על המעבד, מומלץ להשתמש במחשב עם מעבד מודרני וזיכרון RAM מספק. ככל שיותר מודלים טעונים במקביל, נדרש יותר זיכרון. לצורך ביצועים טובים יותר ניתן להשתמש במחשב חזק יותר או בכרטיס גרפי, אך המערכת יכולה לפעול גם על CPU. בממשק בוצעו התאמות נגישות בסיסיות. הצבעים נבחרו כך שתהיה ניגודיות ברורה בין הטקסט לרקע, במיוחד בטקסטים מרכזיים, כפתורים והודעות שגיאה. הכפתורים גדולים וברורים מספיק ללחיצה, והטקסטים בשדות הקלט ובכותרות נכתבו בגודל קריא. בנוסף, שימוש חוזר במבנה קבוע של מסכים מסייע למשתמש להבין היכן הוא נמצא במערכת. בתיבות הקלט מופיע טקסט עזר המסביר למשתמש מה עליו להזין. הודעות השגיאה אינן מסתפקות בסימון כללי של שגיאה, אלא מסבירות מה הבעיה ומה יש לעשות כדי לתקן אותה. במקומות שבהם נעשה שימוש בלוגו או באייקונים, הם אינם הדרך היחידה להבנת הפעולה, משום שלצד רוב הפעולות מופיע גם טקסט ברור. לסיכום, ממשק המשתמש של MindMatch AI נבנה במטרה לאפשר תהליך עבודה ברור, מקצועי ונוח עבור חברות ומועמדים. העיצוב מבוסס על צבעי המותג של המערכת, שימוש בכרטיסים ברורים, כפתורים אחידים, הודעות משתמש ממוקדות וניווט פשוט בין המסכים. בנוסף, המערכת כוללת טיפול בשגיאות, בדיקות תקינות והתאמות נגישות בסיסיות, כדי לאפשר שימוש יעיל וברור גם בתהליך מורכב הכולל הזנת נתונים, ניתוח תשובות, הפקת ציונים וצפייה בתוצאות.

21. קוד התוכנית

21.1 קלט

21.2 פלט

21.3 פונקציות עיקריות

Tokenizer למודל RoBERTa

הקוד הבא אחראי על המרת טקסט לרצף טוקנים מספריים עבור מודלי RoBERTa. הוא טוען את קובצי ה-vocabulary וה-merges, מפעיל אלגוריתם BPE לפירוק מילים לתתי-יחידות, ומחזיר רשימת מזהים מספריים המשמשת כקלט למודל.


```
def __init__(self, vocab_file, merges_file):
    with open(vocab_file, 'r', encoding='utf-8') as f:
        self.vocab = json.load(f)
    self.inverse_vocab = {v: k for k, v in self.vocab.items()}# יצירת מילון הפוך
    with open(merges_file, 'r', encoding='latin-1') as f:
        lines = f.read().splitlines()# מחזיר רשימה של שורות מהקובץ
        merges = lines[1:]# מדלג על השורה הראשונה
    self.bpe_ranks = dict(zip([tuple(m.split()) for m in merges],
                              range(len(merges))))# לפי סדר עדיפות BPE יצירת מילון של חוקי
    self.cache = {}# שומרים תוצאה למילה מסוימת כדי שלא נחשב שוב
    self.byte_encoder = self.bytes_to_unicode()
```

```
def bpe(self, token):
    word = tuple(token)# הופכת את הטקסט לרצף תווים
    pairs = self.get_pairs(word)# מציאת כל זוגות התווים הצמודים
    if not pairs:# אם אין זוגות מחזירה את הטוקן כמו שהוא
        return token
    while True:
        bigram = min(pairs, key=lambda pair: self.bpe_ranks.get(pair, float('inf')))# בחירת הזוג עם הדירוג הכי טוב
        if bigram not in self.bpe_ranks:# BPE מפסיקים אם אף זוג לא נמצא בחוקי
            break
        first, second = bigram# פירוק הזוג לשני חלקים
        new_word = []# MERGE רשימה שתכיל את המילה אחרי
        i = 0# התחלת מעבר על המילה
        while i < len(word):# מעבר על רכיבי המילה
            try:
                j = word.index(first, i)# הבא FIRST חיפוש
                new_word.extend(word[i:j])
                i = j
            except ValueError:
                new_word.extend(word[i:])
                break
            if i < len(word) - 1 and word[i] == first and word[i + 1] == second:
                new_word.append(first + second)# איחוד הקוד ליחידה אחת
                i += 2# דילוג על 2 החלקים שאיחדנו
            else:
                new_word.append(word[i])# אם לא נמצא הזוג המדויק, נוסיף את התו הנוכחי כמו שהוא
                i += 1# התקדמות תו אחד
        word = tuple(new_word)
    if len(word) == 1: break# אם כל המילה הפכה לחלק אחד - מפסיקים
```

הקובץ layers.py מגדיר את שכבות הבסיס של המודל, כגון Multi Head Attention, Feed Forward, Positional Encoding ו-Encoder Layer:

```

class MultiHeadAttention(nn.Module):#מחלקת ריבוי ראשים
    def __init__(self, d_model, num_heads):
        super(MultiHeadAttention, self).__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)
    def scaled_dot_product_attention(self, Q, K, V, mask = None):
        attn_scores = torch.matmul(Q, K.transpose(-1, -2)) / math.sqrt(self.d_k)
        if mask is not None:
            attn_scores = attn_scores.masked_fill(mask == 0, -1e9)
        attn_probs = torch.softmax(attn_scores, dim = -1)
        output = torch.matmul(attn_probs, V)
        return output
    def split_head(self, x):
        batch_size, seq_length, d_model = x.size()
        return x.view(batch_size, seq_length, self.num_heads, self.d_k).transpose(1, 2)
    def combine_heads(self, x):
        batch_size, _, seq_length, d_k = x.size()
        return x.transpose(1, 2).contiguous().view(batch_size, seq_length, self.d_model)
    def forward(self, Q, K, V, mask = None):
        batch_size = Q.size(0)
        q = self.W_q(Q).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        k = self.W_k(K).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        v = self.W_v(V).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        attn_output = self.scaled_dot_product_attention(q, k, v, mask)
        return self.W_o(attn_output)

class PositionWiseFeedForward(nn.Module):#Feed Forward מגדיר את שכבת ה-Transformer
    def __init__(self, d_model, d_ff):
        super(PositionWiseFeedForward, self).__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)
        self.activation = nn.GELU()
    def forward(self, x):
        return self.fc2(self.activation(self.fc1(x)))

class PositionalEncoding(nn.Module):#חוספת מידע על מיקום הטוקנים במשפט
    def __init__(self, d_model, max_seq_length=514):
        super().__init__()
        self.pe = nn.Embedding(max_seq_length, d_model)
    def forward(self, x):
        seq_len = x.size(1)
        positions = torch.arange(2, seq_len + 2, device=x.device).unsqueeze(0)
        return x + self.pe(positions)

class EncoderLayer(nn.Module):#שכבת קידוד
    def __init__(self, d_model, num_heads, d_ff, dropout):
        super(EncoderLayer, self).__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.feed_forward = PositionWiseFeedForward(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)
    def forward(self, x, mask):
        attn_output = self.self_attn(x, x, x, mask)
        x = self.norm1(x + self.dropout(attn_output))
        ff_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ff_output))
        return x

```

הקובץ roberta_model.py מרכיב את השכבות למודל מלא, מוסיף Embedding לטוקנים, שכבות Encoder וראש חיזוי לטוקן חסר:

```

class Transformer(nn.Module):#מחלקת טרנזפורמר
    def __init__(self, vocab_size=50265, d_model=768, num_heads=12, num_layers=12, d_ff=3072, max_len=514, dropout=0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)#המרת טוקן למספרים
        self.pos = PositionalEncoding(d_model, max_len)#הוספת מידע על מיקום הטוקנים
        self.emb_norm = nn.LayerNorm(d_model)#נרמול
        self.layers = nn.ModuleList([#רשימת שכבות
            EncoderLayer(d_model, num_heads, d_ff, dropout)#שכבת קידוד
            for _ in range(num_layers)])#12 שכבות
    def generate_mask(self, src):#יצירת מסכה
        mask = (src != 1).unsqueeze(1).unsqueeze(2)
        return mask
    def forward(self, src):#הגדרת המודל בזמן הרצה
        mask = self.generate_mask(src)#יצירת מסכה
        x = self.embedding(src)
        x = self.pos(x)#הוספת מידע על המיקום במשפט
        x = self.emb_norm(x)#נרמול
        for layer in self.layers:
            x = layer(x, mask)#העברת הקלט דרך שכבת קידוד אחת
        return x

class RobertaLMHead(nn.Module):#הוספת ראש ניבוי טוקן חסר
    def __init__(self, d_model, vocab_size):
        super().__init__()
        self.dense = nn.Linear(d_model, d_model)#שכבה ליניארית
        self.gelu = nn.GELU()
        self.layer_norm = nn.LayerNorm(d_model, eps=1e-5)#נרמול
        self.decoder = nn.Linear(d_model, vocab_size, bias=False)#ממירה מווקטור בגודל 768 לציונים עבור כל מילה באוצר המילים
        self.bias = nn.Parameter(torch.zeros(vocab_size))

class RobertaForMaskedLM(nn.Module):#המודל מנבא מילים
    def __init__(self, transformer_model, vocab_size, d_model):
        super().__init__()
        self.transformer = transformer_model
        self.lm_head = RobertaLMHead(d_model, vocab_size)#יצירת ראש חיזוי מילים
    def forward(self, src):
        hidden_states = self.transformer(src)
        logits = self.lm_head(hidden_states)
        return logits

def load_roberta_weights(my_model, path):#טעינת משקלים חיצוניים למודל
    hf_model = HFRoberta.from_pretrained(path)
    hf_sd = hf_model.state_dict()#שמירת המשקלים שנטענו במילון
    my_sd = my_model.state_dict()#שמירת המשקלים שלי במילון
    is_wrapper = hasattr(my_model, 'transformer')
    prefix = "transformer." if is_wrapper else ""
    mapping = {
        "roberta.embeddings.word_embeddings.weight": f"{prefix}embedding.weight",
        "roberta.embeddings.position_embeddings.weight": f"{prefix}pos.pe.weight",
        "roberta.embeddings.LayerNorm.weight": f"{prefix}emb_norm.weight",
        "roberta.embeddings.LayerNorm.bias": f"{prefix}emb_norm.bias",
    }
    for i in range(12):
        p_hf, p_my = f"roberta.encoder.layer.{i}", f"{prefix}layers.{i}"
        mapping.update({
            f"{p_hf}.attention.self.query.weight": f"{p_my}.self_attn.W_q.weight",
            f"{p_hf}.attention.self.query.bias": f"{p_my}.self_attn.W_q.bias",
            f"{p_hf}.attention.self.key.weight": f"{p_my}.self_attn.W_k.weight",
            f"{p_hf}.attention.self.key.bias": f"{p_my}.self_attn.W_k.bias",
            f"{p_hf}.attention.self.value.weight": f"{p_my}.self_attn.W_v.weight",
            f"{p_hf}.attention.self.value.bias": f"{p_my}.self_attn.W_v.bias",

```

התאמת שאלות למשרה וזיהוי מאפיינים חשובים לחברה

הפונקציה מקבלת תיאור משרה ורשימת שאלות ממאגר השאלות, מחשבת התאמה בין דרישות המשרה לבין כל שאלה, ומחזירה רשימת שאלות מדורגת.

באמצעות דירוג השאלות ניתן להבין אילו מאפיינים חשובים למשרה, לדוגמה ניסיון, יכולת טכנית, איכות חשיבה, יכולת עבודה עצמאית או התאמה לסביבת עבודה.
קלט:

- שם המשרה.
- תיאור המשרה.
- רשימת שאלות מתוך מאגר השאלות.
- מספר שאלות רצוי להחזרה.
- פלט:
- רשימת שאלות מדורגת לפי התאמה.
- אופציה להוספת שאלה.
- בניית הטקסט שנכנס למודל:

```
def build_input_text(job_title, job_description, question):
    #שליפת תגית תוכן של השאלה
    q_tags = tags_to_text(question.get("questionContentTags", {}))
    #שליפת רשימת מודלי האבחון המומלצים לשאלה
    q_models = list_to_text(question.get("questionRecommendedScoringModels", []))
    #ברשימה יישמרו חלקי הטקסט שיכנסו למודל
    parts = [
        f"Job title: {job_title}",
        f"Job description: {job_description}",
        "",
        f"Question: {question.get('questionText', '')}",
        f"Question diagnostic target: {question.get('questionDiagnosticTarget', '')}",
    ]
    #תגיות תוכן
    if q_tags:
        parts.append(f"Question content tags: {q_tags}")
    #מודלי ניקוד
    if q_models:
        parts.append(f"Question scoring models: {q_models}")
    return "\n".join(parts)
```

פונקציה זו בונה את הקלט הטקסטואלי שנשלח למודל התאמת השאלות למשרה.
היא משלבת את שם המשרה, תיאור המשרה, נוסח השאלה, יעד האבחון של השאלה, תגיות התוכן ומודלי הניקוד המומלצים.
מטרת הפונקציה היא ליצור טקסט מאוחד וברור, שמאפשר למודל להשוות בין דרישות המשרה לבין מאפייני השאלה ולחשב ציון התאמה מדויק יותר.
דירוג השאלות לפי התאמתן למשרה:

```
def rank_questions(job_title, job_description, raw_questions, top_k=20):
    model, tokenizer = load_model_and_tokenizer()
    #רשימת השאלות שימצאו מתאימות
    ranked = []
    for raw_question in raw_questions:
        #נרמול השאלות למבנה אחיד
        question = normalize_question(raw_question)
        #טקסט ריק
        if not question["questionText"].strip():
            continue
        #פונקציה שמחשבת ציון התאמה בין שאלה למשרה
        score = predict_score(model, tokenizer, job_title, job_description, question)
        #מוסיף לרשימת אובייקט חדש שמייצג את השאלה עם ציון ההתאמה שלה
        ranked.append({
            "questionBankItemId": question["questionBankItemId"],
            "questionCode": question["questionCode"],
            "questionText": question["questionText"],
            "diagnosticTarget": question["questionDiagnosticTarget"],
            "matchScore": round(score, 4),
            "questionContentTags": question["questionContentTags"],
            "questionRecommendedScoringModels": question["questionRecommendedScoringModels"]
        })
    #מיון רשימת השאלות לפי ציון ההתאמה מהגבוה לנמוך
    ranked = sorted(ranked, key=lambda x: x["matchScore"], reverse=True)
    return ranked[:top_k]
```

פונקציה זו אחראית על דירוג שאלות הראיון לפי התאמתן לתיאור המשרה. תחילה נטענים המודל וה-`tokenizer`, ולאחר מכן הפונקציה עוברת על כל השאלות ממאגר השאלות. כל שאלה עוברת נרמול למבנה אחיד, ולאחר מכן מחושב עבורה ציון התאמה באמצעות המודל. השאלה נשמרת יחד עם הציון, יעד האבחון, תגיות התוכן ומודלי הניקוד המומלצים. בסיום, השאלות ממוינות לפי ציון ההתאמה מהגבוה לנמוך, והמערכת מחזירה רק את מספר השאלות המתאימות ביותר.

הבנת הסיבה שהתשובה לא ענתה על השאלה

הפונקציה מופעלת כאשר תשובת המועמד אינה רלוונטית לשאלה. היא מנתחת את הקשר בין השאלה לבין התשובה, מזהה את סיבת הכשל ומחזירה למועמד הערה לשיפור התשובה.

קלט:

- טקסט השאלה.
- תשובת המועמד.
- פלט:
- סטטוס הדיאגנוזה.
- סיבת הכשל, לדוגמה תשובה כללית מדי, תשובה בנושא אחר, חוסר דוגמה קונקרטית או חוסר התאמה לשאלה.
- הודעת משוב למועמד.
- הסבר פנימי למערכת.
- רשימת הסיבות:

```
class Reason(str,Enum):
    topic_drift="topic_drift"
    task_mismatch="task_mismatch"
    abstraction_mismatch="abstraction_mismatch"
    constraint_or_assumption_mismatch="constraint_or_assumption_mismatch"
    ambiguity_resolution="ambiguity_resolution"
    incomplete_coverage="incomplete_coverage"
    concept_confusion="concept_confusion"
    knowledge_gap="knowledge_gap"
    evasive_nonanswer="evasive_nonanswer"
    no_example="no_example"
    none_match="none_match"
```

קטע קוד זה מגדיר את סוגי הסיבות האפשריות לכך שתשובת מועמד אינה עונה על השאלה. במקום להחזיר תשובה כללית כמו "לא רלוונטי", המערכת מסווגת את הכשל לסיבה מוגדרת, כגון סטייה מהנושא, חוסר דוגמה, תשובה מתחמקת, חוסר ידע או כיסוי חלקי בלבד של השאלה. טעינת המודלים:

```
class MisunderstandingDiagnoser:
    #חלקה שמייצגת מודל שאחראי להבין למה תשובת המועמד לא ענתה טוב על השאלה
    def __init__(self,models_dir:str,softmax_alpha:Optional[float]=None):
        self.device=torch.device("cuda"if torch.cuda.is_available()else"cpu")
        self.mnli_path=os.path.join(models_dir,"roberta-large-mnli")
        self.embed_path=os.path.join(models_dir,"all-MiniLM-L6-v2")
        default_alpha=float(CFG["thresholds"]["SOFTMAX_ALPHA"])
        self.softmax_alpha=float(default_alpha if softmax_alpha is None else softmax_alpha)
        if not os.path.isdir(self.mnli_path):
            raise FileNotFoundError(f"Mnli model folder not found: {self.mnli_path}")
        if not os.path.isdir(self.embed_path):
            raise FileNotFoundError(f"Embedder model folder not found: {self.embed_path}")
        self.tokenizer=AutoTokenizer.from_pretrained(self.mnli_path)
        self.model=AutoModelForSequenceClassification.from_pretrained(self.mnli_path)
        self.model.eval()
        self.model.to(self.device)
        self.enthailment_id=2
        if hasattr(self.model.config,"label2id")and isinstance(self.model.config.label2id,dict):
            #enthailment רק ערך התווית שמירת רק ערך התווית
            for k,v in self.model.config.label2id.items():
                if str(k).lower().startswith("entail"):
                    self.enthailment_id=int(v)
        self.embedder=SentenceTransformer(self.embed_path)
```

בקטע זה נטענים שני רכיבים מרכזיים של מודל הדיאגנוזה: מודל MNLI לבדיקת קשר לוגי בין טקסטים, ומודל SentenceTransformer לחישוב דמיון סמנטי. שילוב זה מאפשר למערכת להבין את הקשר בין השאלה לתשובה, לזהות סיבות אפשריות לאי-התאמה, ולבסס את ההחלטה על יותר מממד אחד. המערכת עוברת על כל ההשערות, מחשבת Entailment לכל סיבה, ואז מדרגת:

```
#מקבלת שאלה ותשובה ומחזירה אובייקט
def diagnose(self,question:str,answer:str)->Diagnosis:
    #צירוף השאלה והתשובה לטקסט אחד
    premise=CFG.get("premise_template","Question: {q}\nAnswer: {a}").format(q=question,a=answer)
    #ברשימה זו יישמרו ציוני ההתאמה לכל סיבת כשל
    scored=[]
    #במילון זה יישמר לכל סיבה ציון ה entailment שלה
    entail_map={}
    #מעבר על כל סיבות הכשל ועל ההשערה שמתארת אותה ובדיקת התאמה
    for reason,hyp in HYPOTHESES.items():
        s=self._entailment(premise,hyp)
        scored.append((reason,float(s)))
        entail_map[reason.value]=float(s)
    #מיון מהגבוה לנמוך
    scored.sort(key=lambda x:x[1],reverse=True)
    #בחירת הסיבה עם הציון הגבוה ביותר
    best_reason,best_entail=scored[0]
    #חישוב דמיון סמנטי בין שאלה לתשובה, אם הדמיון נמוך ייתכן שהתשובה בכלל לא קשורה לשאלה
    sim_qa=qa_similarity(question,answer,self.embedder)
    #בדיקה אם התשובה דומה לדוגמאות של חוסר ידע
    kg_sim=max_similarity_to_prototypes(answer,KNOWLEDGE_GAP_PROTOTYPES,self.embedder)
    #בדיקה אם התשובה דומה לדוגמאות של תשובה מתחמקת
    ev_sim=max_similarity_to_prototypes(answer,EVASIVE_PROTOTYPES,self.embedder)
    ans_l=answer.lower().strip()
    q_l=question.lower().strip()
```

זוהי תחילת פונקציית הדיאגנוזה המרכזית.

הפונקציה בונה טקסט משולב של השאלה והתשובה, ולאחר מכן בודקת עבור כל סיבת כשל אפשרית עד כמה היא מתאימה למקרה הנוכחי.

בנוסף, היא מחשבת דמיון סמנטי בין השאלה לתשובה, דמיון לדפוסי חוסר ידע ודמיון לדפוסי תשובה מתחמקת.

שילוב המדדים מאפשר למערכת לבחור את סיבת הכשל המתאימה ביותר.
 חוקי ההכרעה:

```
if kg_strong and not has_recovery_content:
    forced_reason=Reason.knowledge_gap
if forced_reason is None and sim_qa<HARD_OFFTOPIC_T:
    forced_reason=Reason.topic_drift
if forced_reason is None and sim_qa<SOFT_OFFTOPIC_T and scored[0][1]<WEAK_NLI_T:
    forced_reason=Reason.topic_drift
if forced_reason is None and asks_steps and looks_definitional and sim_qa>=SOFT_OFFTOPIC_T:
    forced_reason=Reason.task_mismatch
if forced_reason is None and sim_qa>=SOFT_OFFTOPIC_T and kg_sim>=KNOWLEDGE_GAP_T and scored[0][1]<WEAK_NLI_T:
    forced_reason=Reason.knowledge_gap
if forced_reason is None and no_example_lex:
    forced_reason=Reason.no_example
if forced_reason is None and sim_qa>=SOFT_OFFTOPIC_T and ev_sim>=EVASIVE_T and scored[0][1]<WEAK_NLI_T:
    forced_reason=Reason.evasive_nonanswer
if forced_reason is None and best_reason==Reason.topic_drift and sim_qa>=0.35 and len(scoded)>1:
    best_reason,best_entail=scoded[1]
```

קטע זה מציג שכבת החלטה לוגית מעל תוצאות המודלים.

במקרים מסוימים המערכת מזהה דפוסים ברורים, כגון חוסר ידע, סטייה מהנושא, תשובה הגדרתית במקום ביצועית, היעדר דוגמה או תשובה מתחמקת.
 במקרים כאלה נקבעת סיבת הכשל באופן מפורש, כדי לשפר את יציבות ההחלטה ולא להסתמך רק על ציון המודל.

ראוטר ניתוב מודלי האבחון בשילוב התייחסות למודלי ניתוב המומלצים של השאלה (מטא-דטא שלה)

הפונקציה מקבלת שאלה ותשובת מועמד, ומחליטה אילו מודלי אבחון מתאימים להפעלה.
 לדוגמה, אם התשובה עוסקת בניסיון קודם, יופעל מודל ניסיון; אם היא כוללת פתרון בעיה, יופעל מודל איכות חשיבה; ואם היא מתארת התנהגות אישית, יופעל מודל תכונות.

בנוסף ניתנת התייחסות לביטחון מודל הניתוב ולתוכן המטא- דטא של השאלה.

```
HashSet<string> recommendedModels = ParseRecommendedModels(recommendedScoringModelsJson);
// קורה מתוך המטא את המודלים המומלצים
if (recommendedModels.Count == 0)
{
    Console.WriteLine("No recommended models found in metadata. Using raw router decision.");
    return rawRouting;
}

EnsurePredictedKeys(rawRouting);
if (!HasEnoughDiagnosticSignal(answer))
{
    // אם התשובה קצרה מדי או כללית לא מחזקים לפי המטא כדי לא להריץ מודלים על תשובה לא מספיקה
    Console.WriteLine("Answer is too short/vague for metadata boosting. Keeping raw router decision.");
    return rawRouting;
}

ApplyMetadataAwareDecision(rawRouting, recommendedModels);
// שינוי החלטת הניתוב לפי המודלים המומלצים
rawRouting.RawMetadataJson = JsonConvert.SerializeObject(new
{
    routingMode = _settings.RoutingMode,
    recommendedModels = recommendedModels.ToList(),
    diagnosticCoverageJson,
    contentCoverageJson,
    answerSignalsJson,
    answerLength = answer?.Length ?? 0,
    createdAtUtc = DateTime.UtcNow
});
```

```
private void ApplyOneModel(RoutingResult routingResult, string modelKey, bool isRecommended)
{
    //1 קורה את החלטה 0 או 1
    int rawPrediction = routingResult.Predicted.TryGetValue(modelKey, out int value) ? value : 0;
    double confidence = GetProbability(routingResult, modelKey);
    routingResult.Predicted[modelKey] = isRecommended
        ? (rawPrediction == 1 || confidence >= _settings.BoostThreshold ? 1 : 0)
        : (rawPrediction == 1 && confidence >= _settings.SecondaryThreshold ? 1 : 0);
}
```

אלגוריתם לחישוב ציון סופי

הפונקציה מחשבת את ציון ההתאמה הסופי של המועמד למשרה.

היא אוספת את כל הראיות שנוצרו במהלך הראיון: תשובות, בדיקות רלוונטיות, תוצאות אבחון, כיסוי ראיות ודפוסים חוזרים, ומשקללת אותן לציון אחד מוסבר.

קלט:

- מזהה ראיון.
- תשובות המועמד.
- תוצאות מודלי האבחון.
- נתוני השאלות והמשרה.
- החלטות רלוונטיות ודיאגנוזה.

פלט:

- ציון התאמה סופי.
 - חוזקה מרכזית.
 - חולשה יחסית.
 - שיטת חישוב.
 - סיכום התאמה לחברה.
- פונקציה לחישוב ושמירת ציון סופי לראיון מסוים:


```

public FinalCandidateScore? CalculateAndSaveInterviewScore(Guid interviewId)
{
    //שליפת הראיון מהמסד
    var interview = _db.Interviews
        .Include(i => i.Candidate)
        .Include(i => i.Job)
        .ThenInclude(j => j.Company)
        .FirstOrDefault(i => i.Id == interviewId);
    if (interview == null)
    {
        return null;
    }
    //בניית גרף הראיות של הראיון
    var graph = _graphBuilder.Build(interviewId);
    //ניתוח דפוסים חוזרים על הגרף
    graph.Patterns = _patternAnalyzer.Analyze(graph);
    //חישוב ציון סופי
    graph.FinalBreakdown = _scoreComposer.Compose(graph);
    //שולף תשובות מייצגות מתוך הגרף
    var representativeAnswers = GetRepresentativeAnswers(graph).ToList();
    //בניית אובייקט שמסביר איך הציון חושב
    var rawJson = BuildRawJson(graph, representativeAnswers);
    var finalCandidateScore = new FinalCandidateScore
    {
        //שמירות
        InterviewId = interview.Id,
        FinalScore = graph.FinalBreakdown.FinalScore,
        Summary = BuildSummary(graph, interview),
        RawJson = JsonConvert.SerializeObject(rawJson, Formatting.Indented),
        ModelVersion = "weighted-candidate-evidence-graph-v1"
    };
    _db.FinalCandidateScores.Add(finalCandidateScore);
    interview.Status = "Completed";
    interview.FinishedAtUtc = DateTime.UtcNow;
    _db.SaveChanges();
}
    
```

פונקציה שיוצרת סיכום מילולי לחברה על תוצאות הראיון:

```
private static string BuildSummary(CandidateEvidenceGraph graph, Interview interview)
{
    // בחירת המאפיינים שנבדקו
    var topFeatures = graph.FeatureEvidence.Values
        .OrderByDescending(f => f.PriorityWeight)
        .Take(3).Select(f => $"{f.FeatureName}: {Math.Round(f.WeightedScore, 1)}").ToList();
    // חיפוש חוזקה הבולטת של המועמד
    var strongestFeature = graph.FeatureEvidence.Values
        .Where(f => f.EvidenceCoverage > 0)
        .OrderByDescending(f => f.WeightedScore).FirstOrDefault();
    // חיפוש חולשה יחסית במאפיין חשוב
    var weakestImportantFeature = graph.FeatureEvidence.Values
        .Where(f => f.PriorityWeight >= 0.12 && f.EvidenceCoverage > 0).OrderBy(f => f.WeightedScore).FirstOrDefault();
    // איסוף דפוסים שליליים לאורך הראיון
    var negativePatterns = graph.Patterns
        .Where(p => p.ScoreImpact < 0).OrderBy(p => p.ScoreImpact).Take(2).Select(p => p.Description).ToList();
    // איסוף דפוסים חיוביים
    var positivePatterns = graph.Patterns
        .Where(p => p.ScoreImpact > 0).OrderByDescending(p => p.ScoreImpact).Take(1).Select(p => p.Description).ToList();
    // יצירת רשימת משפטים שמהם יורכב הסיכום הסופי
    var summaryParts = new List<string>
```

הפונקציה בונה סיכום מילולי של הציון.

היא מוצאת מאפיינים מרכזיים, חוזקה בולטת, חולשה במאפיין חשוב, ודפוסים חיוביים או שליליים לאורך הראיון. כך החברה מקבלת הסבר ברור ולא רק מספר.

חישוב קטגוריות נגזרות באבחון אופי (CultureFit, Motivation)

קלט:

- תוצאות אבחון של תשובת מועמד.

- מזהה תשובה או ראיון.

פלט:

- תוצאות אבחון נוספות מסוג מוטיבציה והתאמה תרבותית.

פונ' המגדירה אילו ציונים מחשבים

```
// רשימת אותות לחישוב מוטיבציה
var motivationSignals = new List<Signal>{
    new("C", 0.28, C), // אחריות ומשמעת
    new("O", 0.18, O), // פתיחות
    new("responsibility", 0.18, responsibility), // אחריות מנסיון קודם
    new("ownership", 0.12, ownership), // לקיחת בעלות
    new("impact", 0.10, impact), // השפעה מעשית
    new("depth", 0.08, depth), // עומק חשיבה
    new("exposure", 0.06, exposure), // חשיפה
};
// רשימת אותות לחישוב התאמה תרבותית
var cultureFitSignals = new List<Signal>{
    new("A", 0.30, A), // הסכמה
    new("E", 0.18, E), // הקצנה
    new("clarity", 0.16, clarity), // בהירות תקשורתית
    new("emotional_stability", 0.12, emotionalStability), // יציבות רגשית
    new("ownership", 0.10, ownership), // לקיחת אחריות
    new("responsibility", 0.08, responsibility), // אחריות מנסיון
    new("logic", 0.06, logic), // לוגיקה
};
```

חישוב הציון:

```
// חישוב סכום כל המשקלים האפשריים
double totalPossibleWeight = signals.Sum(s => s.Weight);
// חישוב כמה משקל נוצל
double usedWeight = 0.0;
// הסכום המשוקלל של הציונים
double weightedSum = 0.0;
// רשימת האותות שהשתתפו בחישוב
var usedSignals = new List<string>();
// אותות שלא השתתפו בחישוב
var missingSignals = new List<string>();
foreach (var signal in signals)
{
    if (!signal.Value.HasValue) {missingSignals.Add(signal.Name); continue;}
    usedWeight += signal.Weight;
    // מוסיפים את הציון כפול המשקל שלו
    weightedSum += signal.Weight * signal.Value.Value; usedSignals.Add(signal.Name);
}
if (usedWeight == 0)
{
    return new WeightedScoreResult
    {
        Score = null, Coverage = 0.0, UsedSignals = usedSignals, MissingSignals = missingSignals
    };
}
// חישוב הציון הסופי כממוצע משוקלל
double score = weightedSum / usedWeight;
score = ClipScore(score);
// חישוב הכיסוי: כמה מתוך המידע האפשרי היה באמת זמין
double coverage = totalPossibleWeight > 0
    ? usedWeight / totalPossibleWeight
    : 0.0;
return new WeightedScoreResult
```

הקוד מחשב שתי קטגוריות נגזרות: Motivation ו-CultureFit. במקום להריץ מודל חדש עבור קטגוריות אלו, המערכת משתמשת בציונים שכבר הופקו ממודלי האבחון הקיימים: תכונות אישיות, איכות חשיבה, יכולות מעשיות וניסיון. לכל קטגוריה מוגדרת רשימת אותות עם משקלים שונים, והפונקציה WeightedScore מחשבת ממוצע משוקלל רק לפי האותות שקיימים בפועל. בנוסף, היא מחשבת Coverage, שמייצג כמה מידע היה זמין לצורך החישוב. כך המערכת מפיקה תובנות נוספות מתוך מידע שכבר נאסף, בלי להעמיס בהרצת מודלים נוספים.

הפעלת שרת "מודלים חמים"

כדי לשפר ביצועים, המודלים מופעלים במקביל, ולאחר שכל התוצאות חוזרות הן נשמרות במסד הנתונים. פונקציה הזו שולחת לשרת Python בקשות:

```
public sealed class PythonModelServerClient
{
    private readonly HttpClient _httpClient;
    private readonly ModelServerEndpointSettings _endpoints;

    public PythonModelServerClient(HttpClient httpClient, IConfiguration configuration)
    {
        _httpClient = httpClient;
        _endpoints = LoadSettings(configuration).Endpoints;
    }

    public string CheckRelevance(string question, string answer)
    {
        return PostAndRead(_endpoints.Relevance, new { question, answer });
    }

    public string Diagnose(string question, string answer)
    {
        return PostAndRead(_endpoints.Diagnosis, new { question, answer });
    }

    public string Route(string question, string answer)
    {
        return PostAndRead(_endpoints.Routing, new { question, answer });
    }

    public string RunDiagnostic(string diagnosticType, string question, string answer)
    {
        return PostAndRead(_endpoints.Diagnostic, new
```

בניית גרף הראיות של המועמד

```
private void BuildJobProfile(CandidateEvidenceGraph graph)
{
    //בניית משקל המאפיינים של המשרה
    var rawPriorities = new Dictionary<string, double>(StringComparer.OrdinalIgnoreCase);
    foreach (var question in graph.Questions.Values)
    {
        foreach (var feature in question.ExpectedFeatures)
        {
            if (!rawPriorities.ContainsKey(feature.Key))
            {
                rawPriorities[feature.Key] = 0;

                //חשיבות יכולת לפי: משקל היכולת בשאלה, התאמת השאלה למשרה, חשיבות השאלה
                rawPriorities[feature.Key] += feature.Value * question.MatchToJobScore * question.QuestionImportance;
            }
        }
    }
    if (rawPriorities.Count == 0 || rawPriorities.Values.Sum() <= 0)
    {
        foreach (var item in _settings.DefaultFeaturePriorities)
        {
            rawPriorities[item.Key] = item.Value;
        }
    }
    NormalizeDictionary(rawPriorities);
    foreach (var item in rawPriorities.OrderByDescending(x => x.Value))
```

**קוד שמזהה דפוסים בין תשובות
חולשה במאפיין קריטי:**

```
private void DetectWeakCriticalFeatures(CandidateEvidenceGraph graph, List<CrossAnswerPattern> patterns)
{
    // חולשה במאפיין קריטי
    var settings = _settings.Patterns.WeakCriticalFeature;
    foreach (var criticalFeature in graph.JobProfile.CriticalFeatures)
    {
        // אם הציון המשוקלל נמוך מהסף, אבל יש מספיק כיסוי ראיות – אז זו לא סתם חוסר מידע, אלא חולשה אמיתית במאפיין חשוב
        if (!graph.FeatureEvidence.TryGetValue(criticalFeature, out var evidence))
        {
            continue;
        }

        if (evidence.WeightedScore < settings.WeightedScoreThreshold &&
            evidence.EvidenceCoverage >= settings.EvidenceCoverageThreshold)
        {
            patterns.Add(new CrossAnswerPattern
            {
                PatternType = settings.PatternType,
                Severity = settings.Severity,
                Description = settings.DescriptionTemplate.Replace("{feature}", criticalFeature),
                ScoreImpact = settings.Impact,
                AffectedFeatures = new Dictionary<string, double>(StringComparer.OrdinalIgnoreCase)
                {
                    [criticalFeature] = settings.Impact
                },
                RelatedQuestionIds = evidence.WeakQuestionIds
            });
        }
    }
}
```

ביצוע חזק אבל הסבר חלש:

```
private void DetectStrongExecutionWeakExplanation(CandidateEvidenceGraph graph, List<CrossAnswerPattern> patterns)
{
    // ביצוע חזק אבל הסבר חלש
    var settings = _settings.Patterns.StrongExecutionWeakExplanation;
    // אם היכולות המעשיות גבוהות, אבל איכות החשיבה/ההסבר נמוכה – המערכת מזדהה דפוס כזה.
    var hasPractical = graph.FeatureEvidence.TryGetValue(settings.PracticalAbilitiesFeature, out var practical);
    var hasThinking = graph.FeatureEvidence.TryGetValue(settings.ThinkingQualityFeature, out var thinking);

    if (!hasPractical || !hasThinking)
    {
        return;
    }

    if (practical!.WeightedScore >= settings.PracticalAbilitiesScoreThreshold &&
        thinking!.WeightedScore < settings.ThinkingQualityScoreThreshold)
    {
        patterns.Add(new CrossAnswerPattern
        {
            PatternType = settings.PatternType,
            Severity = settings.Severity,
            Description = settings.Description,
            ScoreImpact = settings.Impact,
            AffectedFeatures = new Dictionary<string, double>(StringComparer.OrdinalIgnoreCase)
            {
                [settings.ThinkingQualityFeature] = settings.Impact
            }
        });
    }
}
```

ציונים גבוהים ואמינות נמוכה:

```
private void DetectLowReliabilityDespiteScores(CandidateEvidenceGraph graph, List<CrossAnswerPattern> patterns)
{
    // ציונים גבוהים ואמינות נמוכה
    var settings = _settings.Patterns.LowReliabilityDespiteHighScores;

    var highScoreLowReliability = graph.FeatureEvidence.Values
        .Where(f => f.WeightedScore >= settings.WeightedScoreThreshold &&
            f.Reliability < settings.ReliabilityThreshold)
        .ToList();

    if (highScoreLowReliability.Count == 0)
    {
        return;
    }

    patterns.Add(new CrossAnswerPattern
    {
        PatternType = settings.PatternType,
        Severity = settings.Severity,
        Description = settings.Description,
        ScoreImpact = settings.Impact,
        AffectedFeatures = highScoreLowReliability.ToDictionary(
            f => f.FeatureName,
            _ => settings.AffectedFeatureImpact,
            StringComparer.OrdinalIgnoreCase)
    });
}
```

עקביות גבוהה במאפיינים חשובים:

```
private void DetectHighConsistency(CandidateEvidenceGraph graph, List<CrossAnswerPattern> patterns)
{
    // עקביות גבוהה במאפיינים חשובים
    var settings = _settings.Patterns.ConsistentHighPriorityAlignment;
    var importantFeatures = graph.FeatureEvidence.Values
        .Where(f => f.PriorityWeight >= settings.PriorityWeightThreshold)
        .ToList();

    if (importantFeatures.Count < settings.MinimumImportantFeatures)
    {
        return;
    }

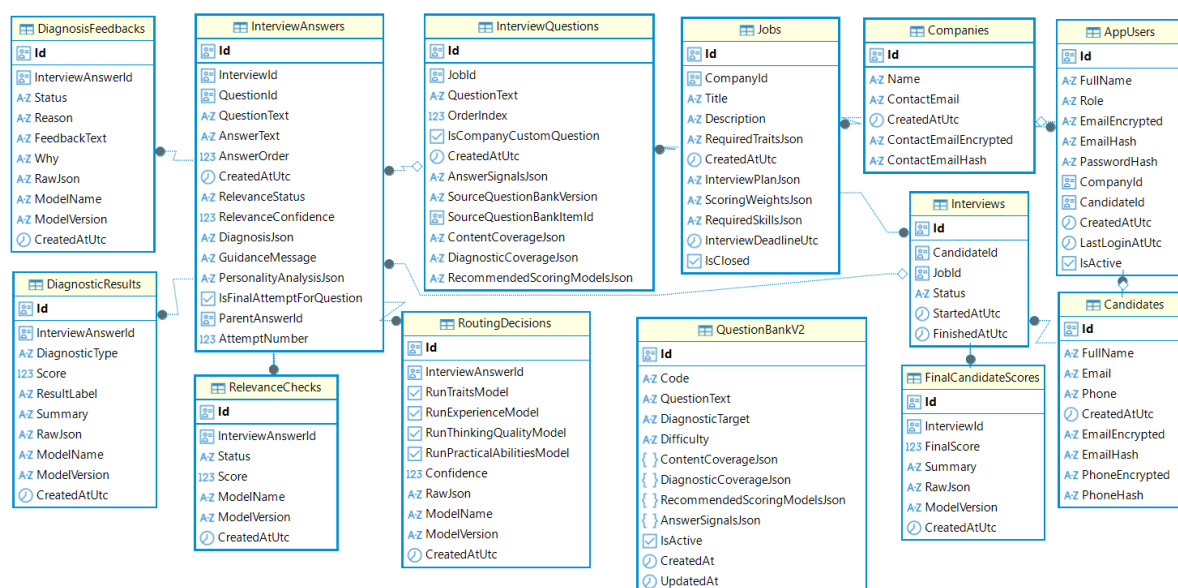
    var allStrong = importantFeatures.All(f =>
        f.WeightedScore >= settings.WeightedScoreThreshold &&
        f.Reliability >= settings.ReliabilityThreshold &&
        f.EvidenceCoverage >= settings.EvidenceCoverageThreshold);

    if (allStrong)
    {
        patterns.Add(new CrossAnswerPattern
        {
            // בכל דפוס נשמרים:
            PatternType = settings.PatternType,
            Severity = settings.Severity,
            Description = settings.Description,
            ScoreImpact = settings.Impact,
            AffectedFeatures = importantFeatures.ToDictionary(
                f => f.FeatureName,
                _ => settings.AffectedFeatureImpact,
                StringComparer.OrdinalIgnoreCase)
        });
    }
}
```

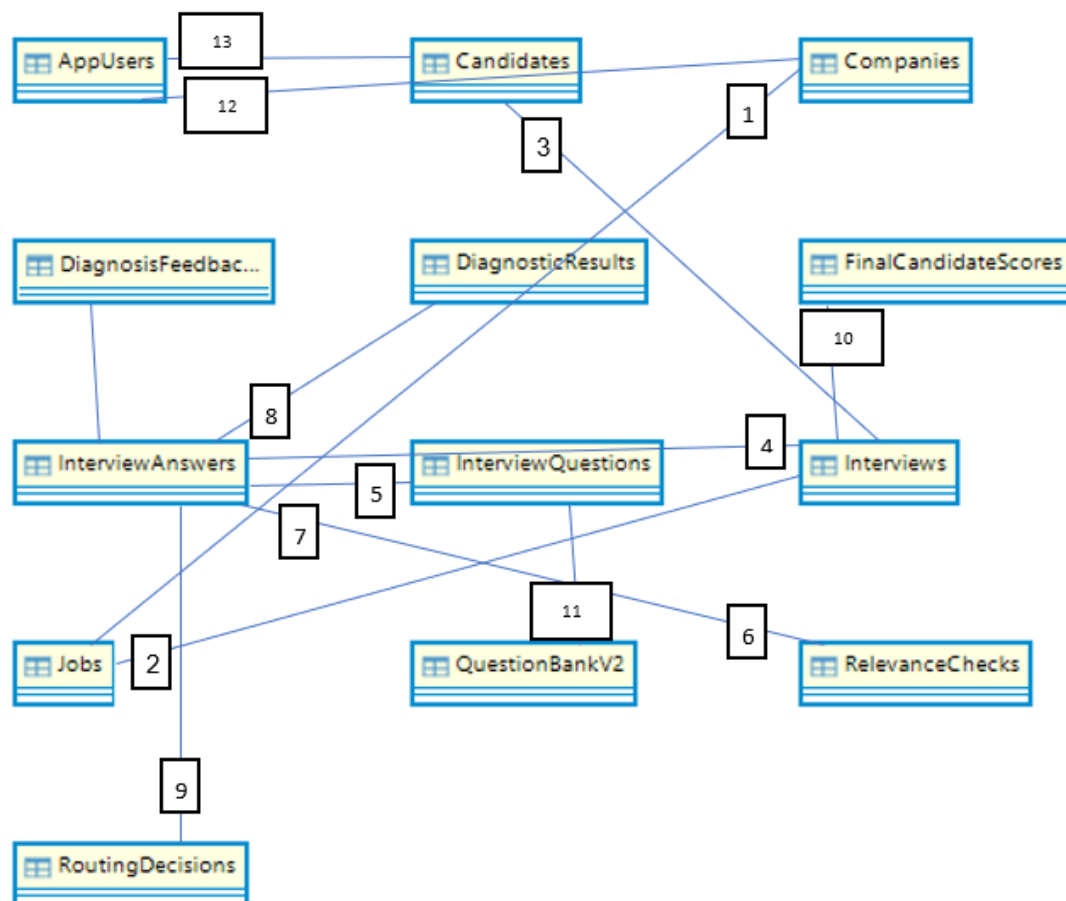
22. תיאור מסד הנתונים

מסד הנתונים של מערכת MindMatch AI שומר את כלל הנתונים הדרושים להפעלת המערכת. במסד נשמרים משתמשים, חברות, מועמדים, משרות, שאלות ראיון, ראיונות, תשובות מועמדים, בדיקות רלוונטיות, משובי דיאגנוזה, החלטות ניתוב למודלי אבחון, תוצאות אבחון, ציונים סופיים ומאגר שאלות. המסד מאפשר לנהל את כל תהליך הראיון: החל מיצירת משרה על ידי חברה, דרך בחירת שאלות מתאימות, ביצוע ראיון על ידי מועמד, ניתוח תשובות המועמד באמצעות מודלי למידת מכונה, ועד שמירת ציון התאמה סופי עבור המועמד. תרשים ה-DSD הבא נוצר מתוך מסד הנתונים באמצעות DBBeaver.

התרשים מציג את הטבלאות הקיימות במסד, את העמודות המרכזיות בכל טבלה, ואת הקשרים הפיזיים שהוגדרו בין הטבלאות באמצעות מפתחות זרים.



תרשים ERD-ה



1. Companies → Jobs

חברה אחת יכולה לפרסם מספר משורות, וכל משרה שייכת לחברה אחת. קשר N:1.

2. Jobs → Interviews

משרה אחת יכולה לכלול מספר ראיונות, וכל ראיון משוּיך למשרה אחת. קשר N:1.

3. Candidates → Interviews

מועמד אחד יכול להשתתף במספר ראיונות, וכל ראיון שייך למועמד אחד. קשר 1:N.

4. Interviews → InterviewAnswers

ראיון אחד כולל מספר תשובות, וכל תשובה שייכת לראיון אחד. קשר 1:N.

5. InterviewQuestions → InterviewAnswers

שאלה אחת יכולה לקבל מספר תשובות, וכל תשובה מתייחסת לשאלה אחת. קשר N:1.

6. InterviewAnswers → RelevanceChecks

לכל תשובה יכולות להישמר בדיקות רלוונטיות, וכל בדיקה שייכת לתשובה אחת. קשר N:1.

7. InterviewAnswers → DiagnosisFeedbacks

לכל תשובה יכול להישמר משוב אבחוני אחד או יותר, וכל משוב שייך לתשובה אחת. קשר N:1.

8. InterviewAnswers → DiagnosticResults
 לכל תשובה יכולות להישמר תוצאות אבחון ממודלים שונים, וכל תוצאת אבחון שייכת לתשובה אחת.
 קשר N:1.

9. InterviewAnswers → RoutingDecisions
 לכל תשובה יכולות להישמר החלטות ניתוב למודלי אבחון, וכל החלטת ניתוב שייכת לתשובה אחת.
 קשר N:1.

10. Interviews → FinalCandidateScores
 לכל ראיון יכול להישמר ציון סופי אחד או יותר, וכל ציון סופי שייך לראיון אחד. קשר N:1.

11. QuestionBankV2 → InterviewQuestions
 שאלה מתוך מאגר השאלות יכולה לשמש כמקור ליצירת שאלות ראיון.
 בטבלת InterviewQuestions נשמר מזהב מקור השאלה מתוך המאגר בשדה SourceQuestionBankItemId. זהו קשר לוגי של N:1.

12. AppUsers → Companies
 משתמש מסוג חברה מקושר לחברה אחת במערכת. דרך קשר זה ניתן לדעת לאיזו חברה שייך המשתמש המחובר. קשר 1:1 או N:1, בהתאם לאפשרות של כמה משתמשים לאותה חברה.

13. AppUsers → Candidates
 משתמש מסוג מועמד מקושר למועמד אחד במערכת. דרך קשר זה ניתן לדעת לאיזה מועמד שייך המשתמש המחובר. קשר 1:1 או N:1, בהתאם לאפשרות של כמה משתמשים לאותו מועמד.

תרשים ה-ERD מציג את הישויות המרכזיות במערכת ואת הקשרים ביניהן.
 ניתן לראות כי משתמש במערכת יכול להיות חברה או מועמד.
 חברה יוצרת משרות, לכל משרה נשמרות שאלות ראיון, ומועמדים מבצעים ראיונות עבור המשרות.
 לכל ראיון נשמרות תשובות המועמד, ועל כל תשובה נשמרות תוצאות עיבוד שונות כגון בדיקת רלוונטיות, דיאגנוזה, החלטת ניתוב ותוצאות אבחון.
 בסיום הראיון נשמר ציון התאמה סופי.

טבלאות:

טבלת AppUsers:

עמודה	תפקיד	טיפוס נתונים	חובה?
Id	מפתח ראשי	Guid	כן
FullName	שם משתמש	string	כן
Role	סוג המשתמש	string	כן
EmailEncrypted	אימייל מוצפן	string	כן
EmailHash	של האימייל Hash לצורך חיפוש	string	כן
PasswordHash	של הסיסמה Hash	string	כן
CompanyId	קישור לחברה אם המשתמש הוא חברה	?Guid	לא
CandidateId	קישור למועמד אם המשתמש הוא מועמד	?Guid	לא
CreatedAtUtc	תאריך יצירה	DateTime	כן
LastLoginAtUtc	זמן התחברות אחרון	?DateTime	לא
IsActive	האם המשתמש פעיל	bool	כן

טבלת Companies:

עמודה	תפקיד	טיפוס נתונים	חובה?
Id	מפתח ראשי	Guid	כן
Name	שם החברה	string	כן
ContactEmail	אימייל איש קשר	string	כן
CreatedAtUtc	תאריך יצירת החברה	DateTime	כן
ContactEmailEncrypted	אימייל מוצפן	string	כן
ContactEmailHash	של אימייל לצורך Hash חיפוש	string	כן

טבלת Candidates:

עמודה	תפקיד	טיפוס נתונים	חובה
Id	מפתח ראשי	Guid	כן
FullName	שם המועמד	string	כן
Email	אימייל	string	כן
Phone	טלפון	string	לא
CreatedAtUtc	תאריך יצירת מועמד	DateTime	כן
EmailEncrypted	אימייל מוצפן	string	כן
EmailHash	של אימייל Hash	string	כן
PhoneEncrypted	טלפון מוצפן	string	לא
PhoneHash	של טלפון Hash	string	לא

טבלת Jobs:

עמודה	תפקיד	טיפוס נתונים	חובה
Id	מפתח ראשי	Guid	כן
CompanyId	מפתח זר לחברה	Guid	כן
Title	שם המשרה	string	כן
Description	תיאור המשרה	string	כן
RequiredTraitsJson	תכונות נדרשות בפורמט JSON	text / JSON string	לא
CreatedAtUtc	תאריך יצירת המשרה	DateTime	כן
InterviewPlanJson	תוכנית ראיון	text / JSON string	לא
ScoringWeightsJson	משקלי ניקוד	text / JSON string	לא
RequiredSkillsJson	כישורים נדרשים	text / JSON string	לא
InterviewDeadlineUtc	תאריך סיום הראיון	?DateTime	לא
IsClosed	האם המשרה סגורה	bool	כן

טבלת Interviews:

עמודה	תפקיד	טיפוס נתונים	חובה
Id	מפתח ראשי	Guid	כן
CandidateId	מפתח זר למועמד	Guid	כן
JobId	מפתח זר למשרה	Guid	כן
Status	סטטוס הראיון	string	כן
StartedAtUtc	זמן התחלת ראיון	DateTime	כן
FinishedAtUtc	זמן סיום ראיון	?DateTime	לא

טבלת InterviewQuestions:

עמודה	תפקיד	טיפוס נתונים	חובה
Id	מפתח ראשי	Guid	כן
JobId	מפתח זר למשרה	Guid	כן
QuestionText	נוסח השאלה	string	כן
OrderIndex	סדר השאלה	int	כן
IsCompanyCustomQuestion	האם זו שאלה שהחברה הוסיפה	bool	כן
CreatedAtUtc	תאריך יצירת השאלה	DateTime	כן
AnswerSignalsJson	אותות נדרשים בתשובה	text / JSON string	לא
SourceQuestionBankVersion	גרסת מאגר השאלות	string	לא
SourceQuestionBankItemId	מזהה מקור ממאגר השאלות	?Guid	לא
ContentCoverageJson	כיסוי תוכן	text / JSON string	לא
DiagnosticCoverageJson	כיסוי אבחוני	text / JSON string	לא
RecommendedScoringModelsJson	מודלים מומלצים	text / JSON string	לא

טבלת InterviewAnswers:

עמודה	תפקיד	טיפוס נתונים	חובה
Id	מפתח ראשי	Guid	כן
InterviewId	מפתח זר לראיון	Guid	כן
QuestionId	מפתח זר לשאלה	Guid	כן
QuestionText	טקסט השאלה בזמן המענה	string	כן
AnswerText	תשובת המועמד	string	כן
AnswerOrder	סדר התשובה	int	כן
CreatedAtUtc	זמן יצירה	DateTime	כן
RelevanceStatus	סטטוס רלוונטיות	string	לא
RelevanceConfidence	ציון ביטחון רלוונטיות	?double	לא
DiagnosisJson	פלט דיאגנוזה	text / JSON string	לא
GuidanceMessage	הערה למועמד	string	לא
PersonalityAnalysisJson	פלט ניתוח אישיות	text / JSON string	לא
IsFinalAttemptForQuestion	האם זה ניסיון אחרון	bool	כן
ParentAnswerId	תשובה קודמת במקרה של תיקון	?Guid	לא
AttemptNumber	מספר ניסיון	int	כן

טבלת RelevanceChecks:

עמודה	תפקיד	טיפוס נתונים	חובה
Id	מפתח ראשי	Guid	כן
InterviewAnswerId	מפתח זר לתשובה	Guid	כן
Status	Relevant / Irrelevant	string	כן
Score	ציון רלוונטיות	double	כן
ModelName	שם המודל	string	כן

ModelVersion	גרסת המודל	string	כן
CreatedAtUtc	זמן יצירה	DateTime	כן

טבלת DiagnosisFeedbacks:

עמודה	תפקיד	טיפוס נתונים	חובה
Id	מפתח ראשי	Guid	כן
InterviewAnswerId	מפתח זר לתשובה	Guid	כן
Status	סטטוס הדיאגנוזה	string	כן
Reason	סיבת הכשל	string	כן
FeedbackText	הערה למועמד	string	כן
Why	הסבר פנימי	string	לא
RawJson	פלט מלא מהמודל	text / JSON string	לא
ModelName	שם המודל	string	כן
ModelVersion	גרסת המודל	string	כן
CreatedAtUtc	זמן יצירה	DateTime	כן

טבלת RoutingDecisions:

עמודה	תפקיד	טיפוס נתונים	חובה
Id	מפתח ראשי	Guid	כן
InterviewAnswerId	מפתח זר לתשובה	Guid	כן
RunTraitsModel	האם להריץ מודל תכונות	bool	כן
RunExperienceModel	האם להריץ מודל ניסיון	bool	כן
RunThinkingQualityModel	האם להריץ מודל איכות חשיבה	bool	כן
RunPracticalAbilitiesModel	האם להריץ מודל יכולות מעשיות	bool	כן
Confidence	ביטחון החלטת הניתוב	double	כן

לא	text / JSON string	פלט מלא	RawJson
כן	string	שם המודל	ModelName
כן	string	גרסה	ModelVersion
כן	DateTime	זמן יצירה	CreatedAtUtc

טבלת DiagnosticResults:

חובה	טיפוס נתונים	תפקיד	עמודה
כן	Guid	מפתח ראשי	Id
כן	Guid	מפתח זר לתשובה	InterviewAnswerId
כן	string	סוג האבחון	DiagnosticType
לא	?double	ציון האבחון	Score
לא	string	תווית תוצאה	ResultLabel
לא	string	סיכום קצר	Summary
לא	text / JSON string	פלט מלא של המודל	RawJson
כן	string	שם המודל	ModelName
כן	string	גרסה	ModelVersion
כן	DateTime	זמן יצירה	CreatedAtUtc

טבלת FinalCandidateScores:

חובה	טיפוס נתונים	תפקיד	עמודה
כן	Guid	מפתח ראשי	Id
כן	Guid	מפתח זר לראיון	InterviewId
כן	double	ציון סופי	FinalScore
לא	string	סיכום מילולי	Summary
לא	text / JSON string	פירוט חישוב מלא	RawJson
כן	string	גרסת שיטת החישוב	ModelVersion
כן	DateTime	זמן יצירה	CreatedAtUtc

טבלת 2:QuestionBankV:

עמודה	תפקיד	טיפוס נתונים	חובה
Id	מפתח ראשי	Guid	כן
Code	קוד שאלה	string	כן
QuestionText	נוסח השאלה	string	כן
DiagnosticTarget	יעד אבחון	string	לא
Difficulty	רמת קושי	string	כן
ContentCoverageJs on	כיסוי תוכן	text / JSON string	לא
DiagnosticCoverage Json	כיסוי אבחוני	text / JSON string	לא
RecommendedScori ngModelsJson	מודלי ניקוד מומלצים	text / JSON string	לא
AnswerSignalsJson	אותות נדרשים בתשובה	text / JSON string	לא
IsActive	האם השאלה פעילה	bool	כן
CreatedAt	תאריך יצירה	DateTime	כן
UpdatedAt	תאריך עדכון	?DateTime	לא

23. מדריך למשתמש

התחברות למערכת
 יש להיכנס למסך ההתחברות ולהקליד אימייל וסיסמה.
 לאחר מכן יש ללחוץ על כפתור הכניסה למערכת.
 אם הפרטים תקינים, המשתמש יועבר למסך המתאים לפי סוג המשתמש שלו: חברה או מועמד.

משתמש א' – חברה

1. לאחר ההתחברות החברה נכנסת לאזור החברה.
2. ליצירת משרה חדשה יש ללחוץ על יצירת משרה, למלא שם משרה, תיאור משרה ותאריך סיום לראיון.
3. לאחר הזנת תיאור המשרה, המערכת מציעה שאלות ראיון מתאימות למשרה.
4. החברה בוחרת ושומרת את שאלות הראיון הרצויות.
5. לצפייה בתוצאות יש להיכנס למסך תוצאות המועמדים.
6. במסך זה החברה בוחרת משרה, לאחר מכן בוחרת מועמד, וצופה בציון ההתאמה ובסיכום הראיון שלו.

הדרישות ממשתמש זה:

1. הרשמה והתחברות למערכת.
2. הזנת פרטי משרה.
3. כתיבת תיאור משרה ברור ומפורט.
4. בחירת תאריך סיום לראיון.
5. בחירת שאלות ראיון מתוך ההצעות שהמערכת מציגה.

משתמש ב' – מועמד

1. לאחר ההתחברות המועמד נכנס לאזור המועמד.
2. לצפייה במשרות יש להיכנס למסך המשרות הפתוחות.
3. כדי להתחיל ראיון יש לבחור משרה וללחוץ על התחלת ראיון.
4. במהלך הראיון יש לענות על השאלות שמוצגות במסך.
5. לאחר כתיבת תשובה יש ללחוץ על שליחת תשובה.
6. אם התשובה אינה רלוונטית לשאלה, המערכת מציגה הערה למועמד ומאפשרת לו לנסות לענות שוב.
7. לאחר סיום כל השאלות, הראיון נשמר במערכת והחברה יכולה לצפות בתוצאות.

הדרישות ממשתמש זה:

1. הרשמה והתחברות למערכת.
2. בחירת משרה פתוחה.
3. כתיבת תשובות ברורות ורלוונטיות לשאלות הראיון.
4. אם נעשה שימוש בקלט קולי, יש לאפשר לדפדפן גישה למיקרופון.

24. בדיקות והערכה

הבדיקות במערכת בוצעו על ידי הזנה חוזרת של נתונים במצבים שונים, כדי לבדוק כיצד המערכת מגיבה לקלט תקין ולקלט שגוי, והאם היא מצליחה לשמור על יציבות לאורך כל תהליך הראיון. נבדקה הרשמה והתחברות של חברה ושל מועמד, כולל ניסיון התחברות עם פרטים שגויים. בנוסף, נבדקה יצירת משרה עם תיאור תקין, עם תיאור חסר, עם תאריך שכבר עבר ועם תאריך החורג מ-30 יום. נבדק תהליך התאמת השאלות למשרה על ידי הזנת תיאורי משרה שונים, כדי לוודא שהמערכת מחזירה שאלות שמתאימות לדרישות התפקיד. כמו כן, נבדק תהליך הראיון של המועמד באמצעות תשובות מסוגים שונים: תשובות רלוונטיות, תשובות קצרות מדי, תשובות כלליות ותשובות שאינן קשורות לשאלה. במקרים שבהם התשובה לא הייתה רלוונטית, נבדק שהמערכת מציגה הערה מתאימה ומאפשרת לו לענות שוב. בנוסף, נבדק שהמערכת מפעילה את מודלי האבחון, שומרת את תוצאותיהם במסד הנתונים, ומחשבת בסיום הראיון ציון התאמה סופי למועמד. מטרת הבדיקות הייתה לוודא שהמערכת מתמודדת עם סוגי קלט שונים, שומרת נתונים בצורה תקינה, מחזירה הודעות מתאימות למשתמש, וממשיכה לפעול באופן יציב גם במצבים לא תקינים.

25. מסקנות

פיתוח הפרויקט MindMatch AI היה עבורי תהליך משמעותי מבחינה מקצועית ואישית. במהלך העבודה למדתי לבנות מערכת מלאה הכוללת צד לקוח, שרת, מסד נתונים ומודלי בינה מלאכותית, ולחבר ביניהם למערכת אחת פעילה. מבחינה מקצועית, הפרויקט חיזק אצלי את הידע ב-React, ב-ASP.NET Core, בעבודה מול PostgreSQL, ובשילוב מודלי Python בתוך מערכת C#. בנוסף, למדתי להתמודד עם בעיות מורכבות כמו בדיקת רלוונטיות של תשובות, ניתוב למודלי אבחון, חישוב ציון סופי ושיפור ביצועים בעזרת שרת מודלים חם.

מבחינה אישית, הפרויקט דרש ממני התמדה, למידה עצמאית ויכולת לפתור תקלות גם כשהן היו מורכבות ולא ברורות מיד. היו שלבים שבהם נתקלתי בשגיאות, באיטיות של מודלים ובבעיות חיבור בין רכיבי המערכת, אך ההתמודדות איתם חיזקה אצלי את תחושת המסוגלות ואת הביטחון ביכולת שלי לפתח מערכת מורכבת. לסיכום, הפרויקט תרם לי רבות בהבנת תהליך פיתוח מערכת אמיתית מקצה לקצה, ובמיוחד בשילוב בין פיתוח תוכנה, מסדי נתונים ולמידת מכונה.

26. פיתוחים עתידיים

בעתיד ניתן לשדרג את המערכת במספר כיוונים:

1. שיפור מודלי האבחון באמצעות הרחבת מאגר הנתונים, כדי לשפר את דיוק בדיקת הרלוונטיות, הדיאגנוזה והציונים.
2. הוספת אפשרות לשליחת סיכום אוטומטי לחברה לאחר סיום הראיונות למשרה מסוימת.
3. הוספת אפשרות למחיקת נתוני מועמדים לאחר תקופה מסוימת, כדי לשפר את שמירת הפרטיות.
4. שיפור ביצועי המודלים באמצעות הרצה על שרת חזק יותר או GPU, כדי לקצר עוד יותר את זמן התגובה בזמן ראיון.

27. ביבליוגרפיה

אתרים:

MDN Web Docs

Stack Overflow

GitHub

huggingface

IBM Natural Language Processing

The Employment Interview: A Review of Recent Research and Recommendations for Future Research

Are we asking the right questions? Predictive validity comparison of four structured interview question types

Scikit-learn User Guide — Supervised Learning

מאמרים:

Semantic Answer Similarity for Evaluating Question Answering Models

Attention Is All You Need

An introduction to the five-factor model and its applications

סרטוני יוטיוב:

Deep Learning Chapter

Let's build GPT: from scratch, in code, spelled out

Building a Transformer Model from Scratch: Complete Step-by-Step Guide

Complete Transformers For NLP Deep Learning One Shot With Handwritten Notes

Let's build the GPT Tokenizer

